

REPIF1-2: Indoor Weather Station Project

Student: Maciej Stankowski

Class: *1TPIF12-DI*

Date: *27 September – 07 June 2026*

Contents

REPIFI1: SPRINT L0 Report Preparation	3
REPIFI1: SPRINT L1 Report Raspberry Pi	6
REPIFI1: SPRINT L2 Report Database	26
REPIFI1: SPRINT L3 Report Website	37
REPIFI1: SPRINT L4 Report Web database server	47
REPIFI1: SPRINT L5 Report Data Transfer	59
REPIFI1: SPRINT L6 Report Backup Server	67
REPIFI1: SPRINT A1 Report PVC Holder	80
REPIFI1: SPRINT A2 Report Printed Circuit Board	91
REPIFI1: SPRINT A3 Report Power Supply	99
REPIFI1: SPRINT A4 Report Sensors 1	104
REPIFI1: SPRINT A5 Report Sensors 2	109
REPIFI1: SPRINT A6 Report Assembly	116
REPIFI1: SPRINT A7 Report Lighting	125

REPIFI1: SPRINT L0 Report Preparation

Project Preparation and Planning

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 27 September – 9 October 2025

Contents

Sprint 0 of the Project	4
Objective	4
Selected Planning Tool	4
Initial Project Setup	5

Sprint 0 of the Project

Objective

The objective of this sprint was to prepare for the project before starting the technical implementation. This included selecting a suitable planning tool to organize tasks and monitor project progress using an agile workflow.

Selected Planning Tool

Trello

For project planning, I selected **Trello** as the planning tool. I chose it because it is free to use, easy to understand, and allows project tasks to be organized visually. Its card-based structure made it practical for tracking completed tasks, ongoing work, and upcoming sprints throughout the project.



Initial Project Setup

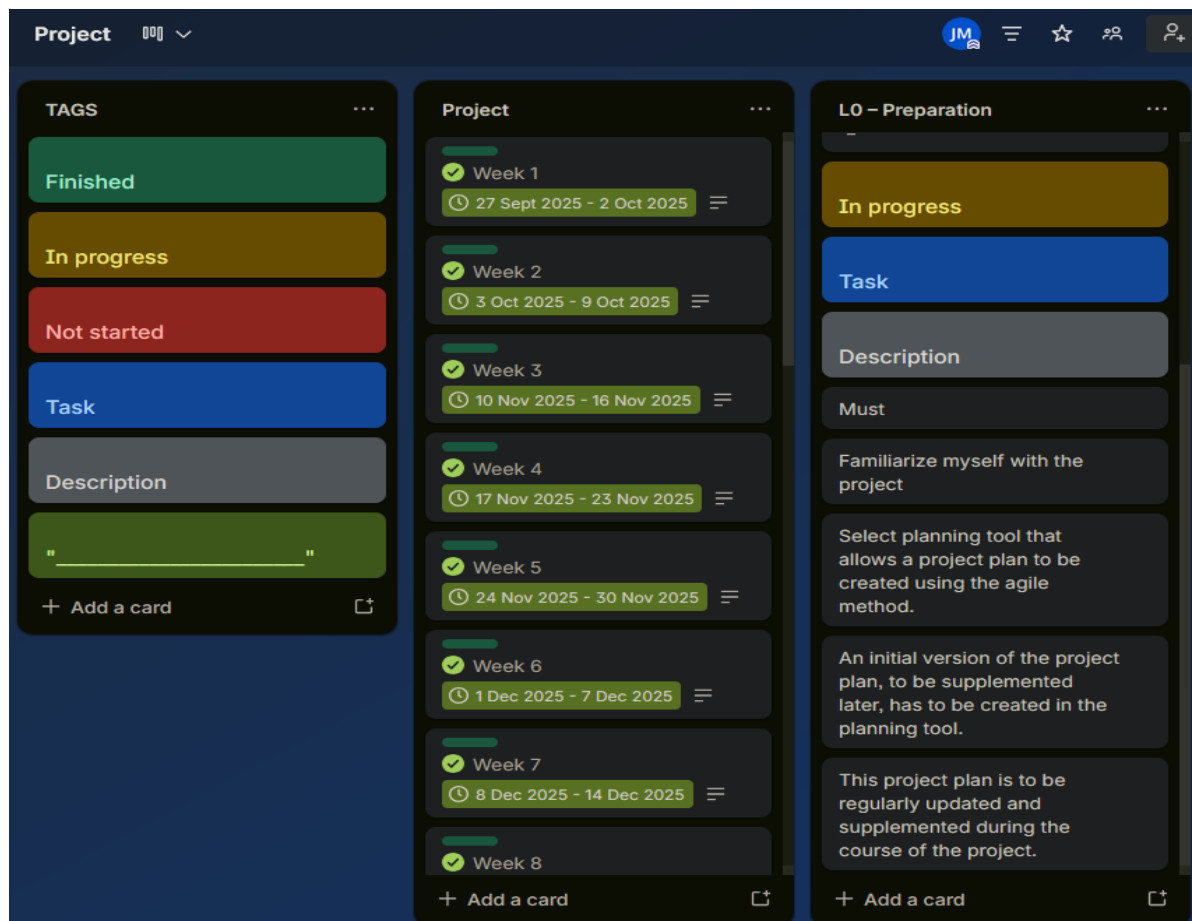
Initial project board was created and divided into several sections to manage daily project work.

The first section, **Tags**, was used to classify task cards using labels such as *Finished*, *In Progress*, *Not Started*, *Task*, and *Description*.

The second section, **Project**, contained date-based checkpoints. This was used to monitor weekly progress and verify whether planned tasks for a specific sprint were completed within the expected timeframe.

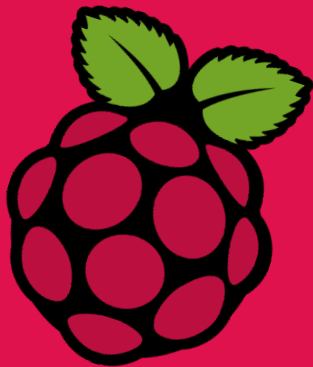
The remaining sections were organized by individual sprints, both for Workstation and Workshop. Inside each sprint, task cards were categorized using the previously created labels:

- **Finished** → tasks that were successfully completed
- **In Progress** → tasks currently being worked on
- **Task** → upcoming activities to be completed
- **Description** → the original *Must*, *Should*, and *Could* requirements from the user story



REPIFI1: SPRINT L1 Report

Raspberry Pi



Installation and Configuration of Raspberry Pi

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 10 November - 22 November 2025

Contents

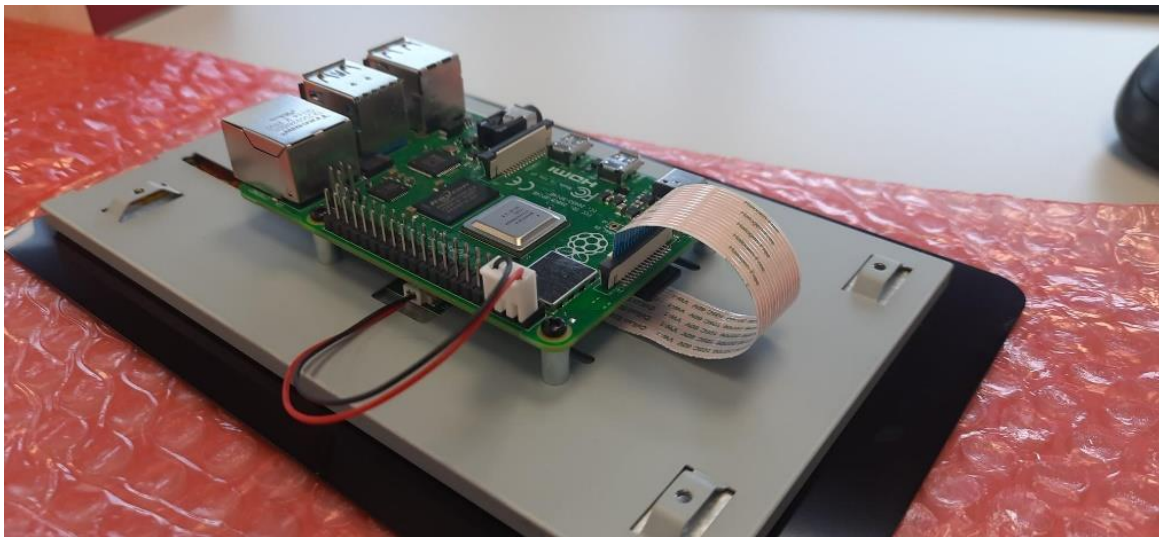
REPIFI1: SPRINT L1 Report Raspberry Pi	6
Sprint 1 of the Project	8
Goal	8
Objective	8
Components.....	10
Raspberry Pi 4 model B	10
Raspberry Pi 7 inch Touchscreen Display PI Touch 2	10
Additional parts/tools needed	11
Softwares.....	11
Raspberry Pi Imager.....	11
RealVNC Viewer	11
WinSCP	11
Installation and Configuration of Raspberry Pi.....	12
Installation of an operating system	12
<i>Installing Raspberry Pi Imager</i>	12
<i>Using Raspberry Pi Imager</i>	13
Assembly	16
Setting up RealVNC Viewer	17
<i>Connecting to our Raspberry Pi through VNC Viewer.....</i>	18
First Setup	19
<i>Changing screen orientation and disabling on-screen keyboard.....</i>	19
<i>Setting Up SSH Key Authentication</i>	20
<i>Setting up static IP address for the Ethernet interface</i>	21
<i>Enabling I2C Interface</i>	22
<i>Installing Firmware Files and configuring them</i>	22
<i>Installing Docker and PHP with Apache2.....</i>	23
<i>Autostart function configuration</i>	24

Sprint 1 of the Project

Goal

The goal of this first sprint was to prepare the Raspberry Pi with all essential base-level installations and configurations required for the later stages of the project.

This report ensures that the device provides a stable foundation for displaying measurements menu on the connected touchscreen, the moment we turn the micro-controller on. Showing values that will be gathered from sensors which will be installed and configured later on in the project's cycle. Eventually transmitting those informations to a web server.



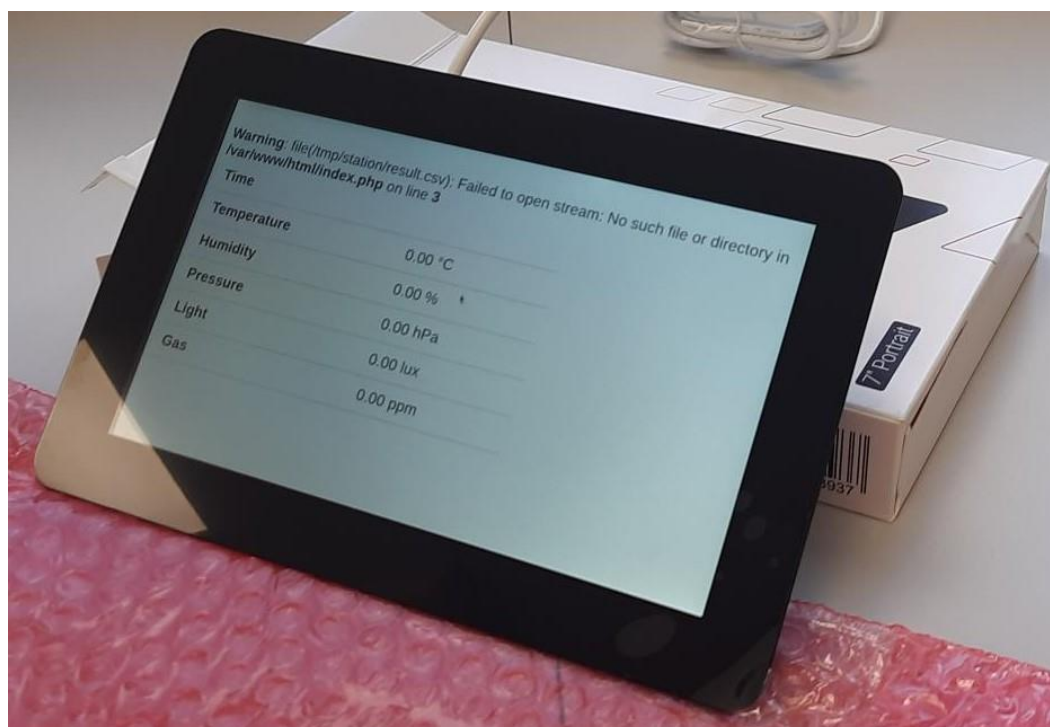
Picture1: Raspberry Pi after assembly

Objective

The objective of this sprint was to fully prepare the Raspberry Pi hardware and software environment so that it is ready for use in the upcoming development phases. This work included mounting the Raspberry Pi to the display and ensuring that the screen was properly connected and configured in landscape orientation. The Raspberry Pi OS was installed by flashing it onto a microSD card, establishing the base system required for further configuration.

Both network interfaces were configured: the Raspberry Pi received a static LAN IP address set within its network settings, and it also obtained a WLAN IP address through the classroom's Wi-Fi connection. Necessary system interfaces, such as I²C, were enabled to allow sensor data to be read. Secure remote access was prepared by enabling SSH and disabling password-based authentication. The supplied project files were copied into the appropriate directories, and the corresponding configuration files were updated as required. Additionally, the firmware was set to start automatically during the boot process.

To support future functionality, Docker was installed and used to host the provided website responsible for displaying the current measurement values. This website was configured to run in kiosk mode on the connected display, ensuring that the interface loads automatically and continuously.



Picture2: Final result of the first Sprint

Components

Raspberry Pi 4 model B

The Raspberry Pi 4 Model B is a small single-board computer that serves as the main processing unit the system. It will run the operating system, communicate with sensors, host the necessary services (such as Docker containers), allow connection between webserver and data, and display measurement data.



of

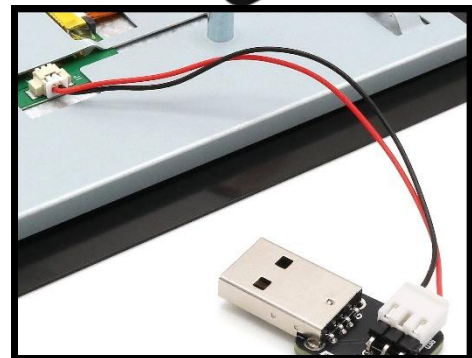
Raspberry Pi 7 inch Touchscreen Display PI Touch 2

This touchscreen display is the primary output device for the project. It will be used to show live measurement values in kiosk mode and allow simple user interaction.



GPIO power cable

The custom made GPIO power cable supplies the touchscreen with power directly from the Raspberry Pi's GPIO pins. By connecting the 5V and GND pins, the display receives a stable power source without needing an external adapter.



15-way to 15-way FFC (Flat Flexible Cable)

The FFC cable is used to connect the touchscreen display to the Raspberry Pi. It ensures that display data and touch control signals are transferred correctly between both devices.

Additional parts/tools needed

Charging cable / power supply – Provides stable power to the Raspberry Pi.

MicroSD card – Used as the Raspberry Pi's storage, containing the operating system and project files.

SD card adapter – Needed to insert the microSD card into a PC when flashing the OS.

Screwdriver – Used to mount the Raspberry Pi onto the display frame or case.

Softwares

Raspberry Pi Imager

Raspberry Pi Imager is a tool used to download and write an operating system onto a microSD card.

I used it to install *Raspberry Pi OS* on the SD card before booting the device for the first time.



RealVNC Viewer

RealVNC Viewer is a remote desktop application that allows you to access and control the Raspberry Pi's graphical interface from another computer. Just like SSH, it needs the devices assigned IP address to connect to it.

I used it for remote configuration and troubleshooting without needing a physical keyboard or mouse attached to the Pi.



WinSCP

WinSCP is a file transfer tool for Windows that connects to the Raspberry Pi using SSH. It allows you to copy files, modify directories, and manage project data easily.

I used it to transfer configuration files, firmware, scripts, and website assets from my personal device—where the project materials were stored—to the Raspberry Pi, placing them into the specific directories required for this sprint.



Installation and Configuration of Raspberry Pi

Introduction

This section describes the installation and base configuration of the Raspberry Pi used in the project. The goal was to prepare a stable and functional system environment that allows sensor data to be collected, displayed, and later transmitted to a server.

The following parts outline the setup process, including operating system installation, network configuration, enabling required interfaces, deploying project files, and preparing the device for further development phases.

Installation of an operating system

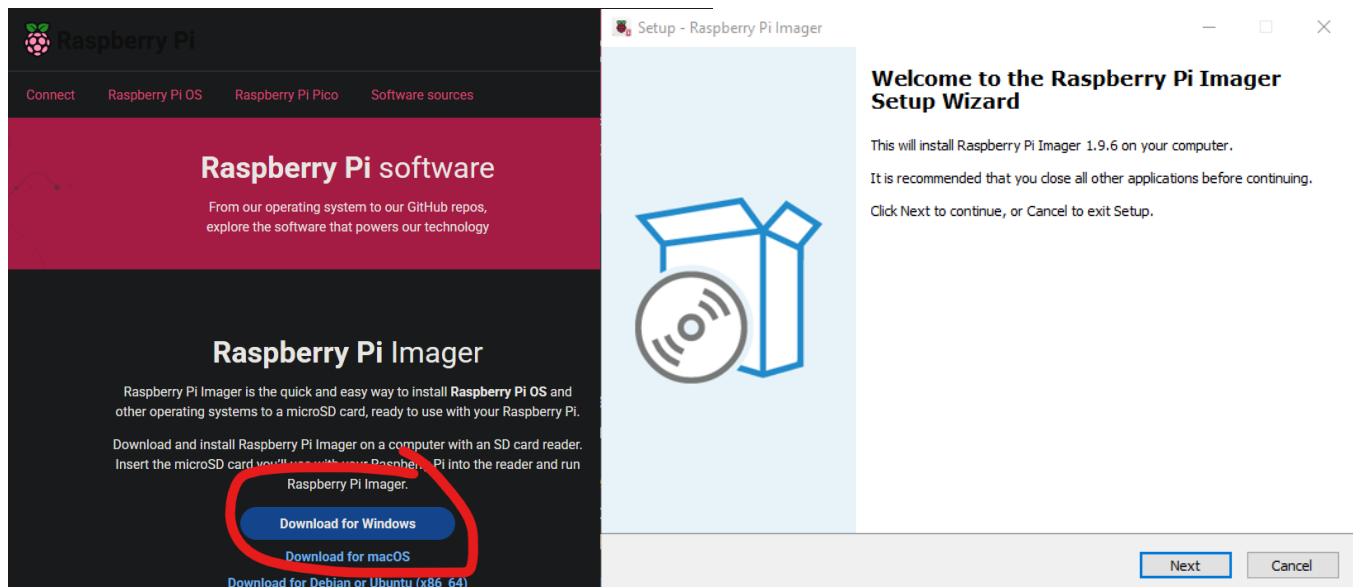
To install the Raspberry Pi OS, the operating system first needs to be flashed onto a microSD card, which will later be used to boot the Raspberry Pi. The official tool recommended for this process is the **Raspberry Pi Imager**, available from the Raspberry Pi Foundation's website.

Installing Raspberry Pi Imager

To begin, the Raspberry Pi Imager must be downloaded and installed on the workstation:

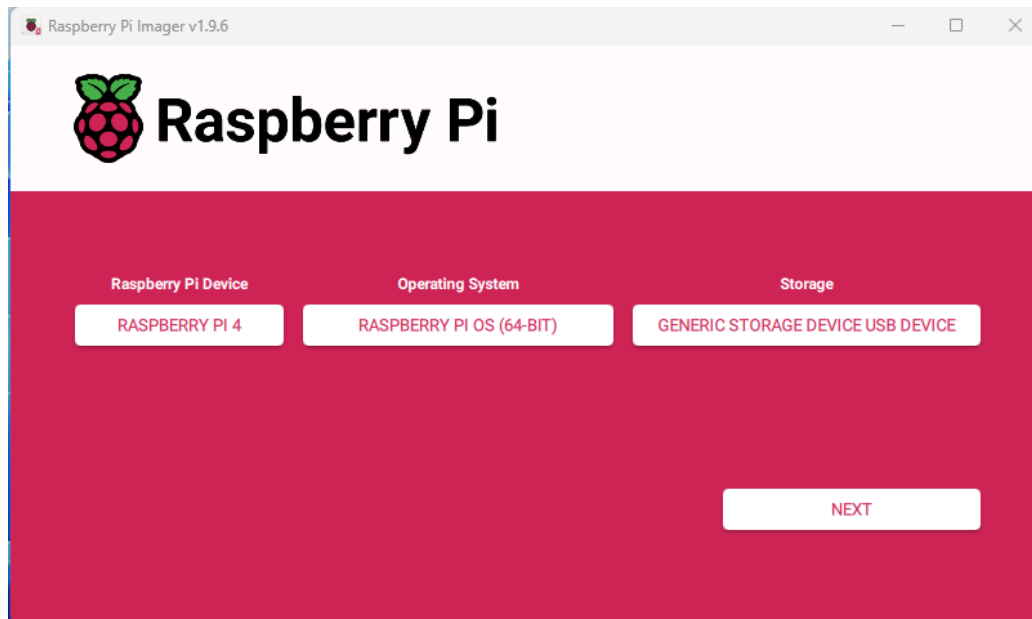
1. Navigate to the official Raspberry Pi software page:
<https://www.raspberrypi.com/software>
2. Download the Raspberry Pi Imager for your operating system and follow the installation instructions.

The setup process is straightforward and mainly consists of selecting the preferred language and clicking *Next* until the installation completes.



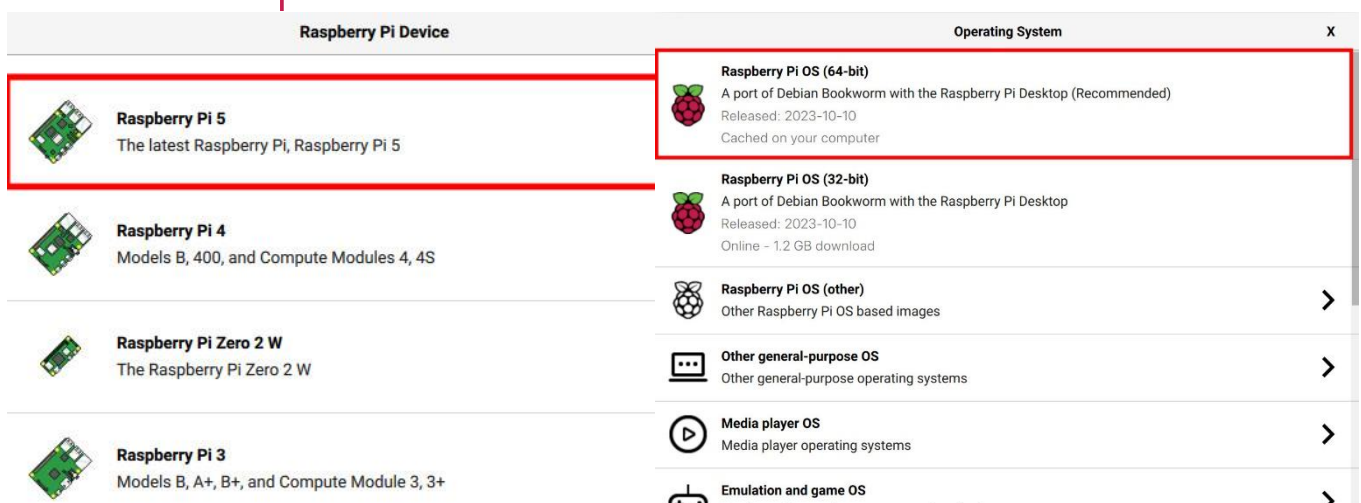
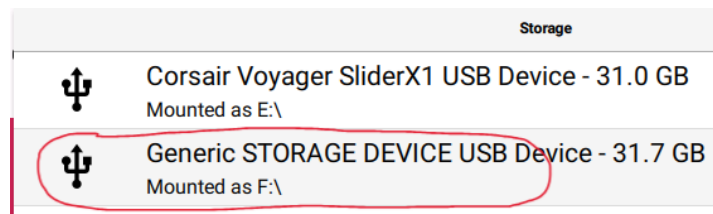
Using Raspberry Pi Imager

Once installed, the Raspberry Pi Imager can be used to prepare the microSD card.



1. Start the program and select the appropriate options:

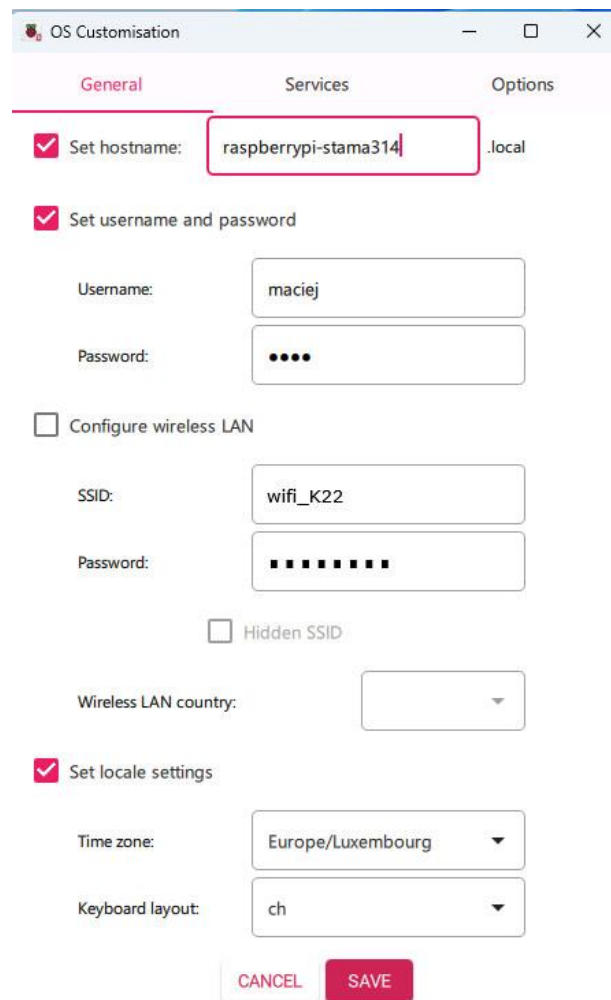
- **Device** → Raspberry Pi 4
- **Operating System** → Raspberry Pi OS (64-bit)
- **Storage** → the microSD card connected through a card reader



2. Before writing the image, open **OS Customisation** to configure essential settings.

In this customisation menu, the following items should be configured:

- **Hostname** → to allow SSH access via local network naming
- **Username and password**
- **Wireless LAN settings** (SSID and password)
- **Keyboard layout** and **timezone**, if needed
- **Enable SSH**, which is required for remote access



The screenshot shows the 'OS Customisation' window with the 'General' tab selected. The settings are as follows:

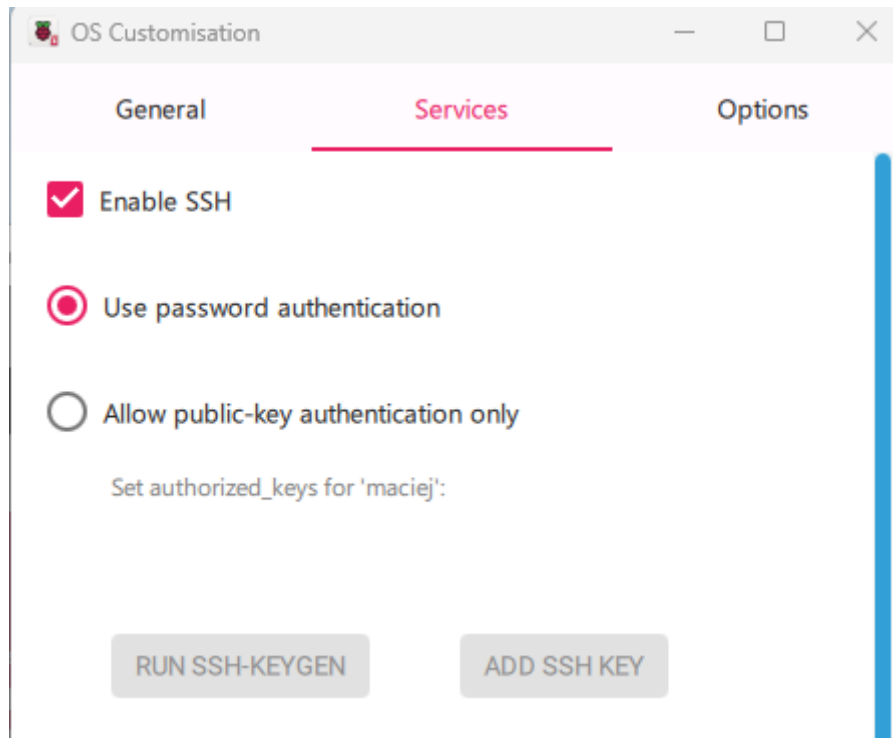
- Set hostname:** ☒ The text field contains 'raspberrypi-stama314' and '.local'.
- Set username and password:** ☒ The 'Username' field contains 'maciej' and the 'Password' field is masked with dots.
- Configure wireless LAN:** ☐ The 'SSID' field contains 'wifi_K22' and the 'Password' field is masked with dots. Below it, 'Hidden SSID' is ☐. The 'Wireless LAN country' dropdown is empty.
- Set locale settings:** ☒ The 'Time zone' dropdown is set to 'Europe/Luxembourg' and the 'Keyboard layout' dropdown is set to 'ch'.

At the bottom, there are 'CANCEL' and 'SAVE' buttons.

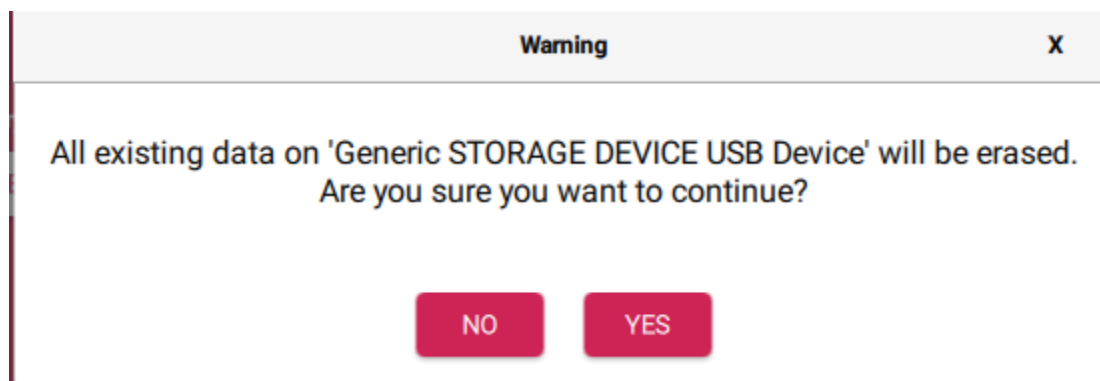
In the OS customisation screen, I set a unique hostname for the project, created the user account, and configured the time zone and keyboard layout.

I enabled Wi-Fi settings here because we are going to use the classroom's network during the project's progression when working on my Raspberry Pi.

I enabled SSH so I could access the Raspberry Pi remotely during development. I also activated password authentication for the initial setup, which I later replaced with key-based SSH authentication for security.



After confirming the settings, click **Save**, then apply the customisation. If the tool warns about erasing the storage, select **Yes**. *(only if the microSD has no important data on it. otherwise, make backup of the content inside of the card)* The flashing process will begin, and after a short wait, the microSD card will be ready for use.



Once the operating system has been successfully written to the microSD card, the next steps involve assembling and configuring the Raspberry Pi hardware, including mounting the touchscreen and preparing the device for the remaining setup tasks.

Assembly

Before starting the first boot and connecting the Raspberry Pi to its power supply, I assembled all required hardware components. This involved attaching the Raspberry Pi 4 to the 7-inch Raspberry Pi touchscreen and connecting the necessary cables to their correct ports.

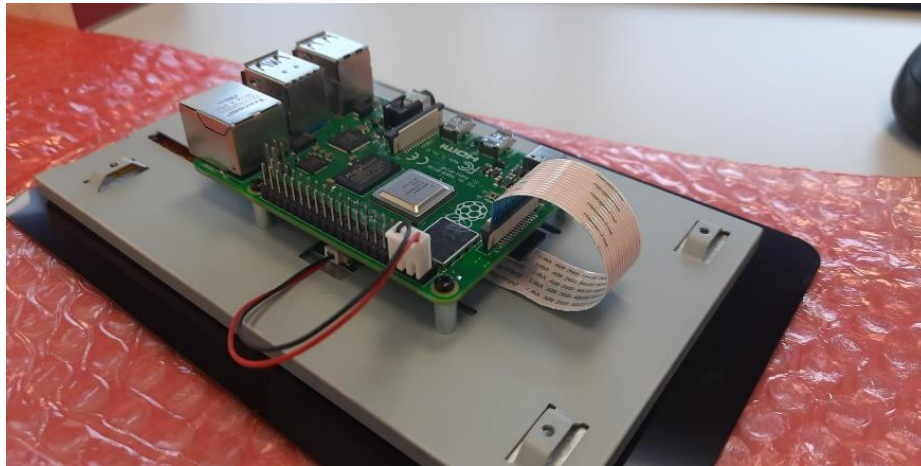


I first connected the **FFC (flat-flex cable)** between the Raspberry Pi and the touchscreen. The Raspberry Pi uses the **DSI (Display Serial Interface) port**, and the touchscreen has a matching DSI ribbon connector on its controller board. This cable is responsible for transmitting the display and touch control signals.



Next, I connected the **GPIO power cable**, which provides power from the Raspberry Pi directly to the touchscreen. The cable was attached to the **5V and GND GPIO pins** on the Raspberry Pi, and the corresponding power header on the touchscreen's controller board, ensuring the display receives stable power without an external adapter.

After both cables were connected securely, I mounted the Raspberry Pi onto the back of the touchscreen using the designated screw holes and spacers included with the kit. Once the assembly was complete and all connections were checked, the system was ready for its first boot.



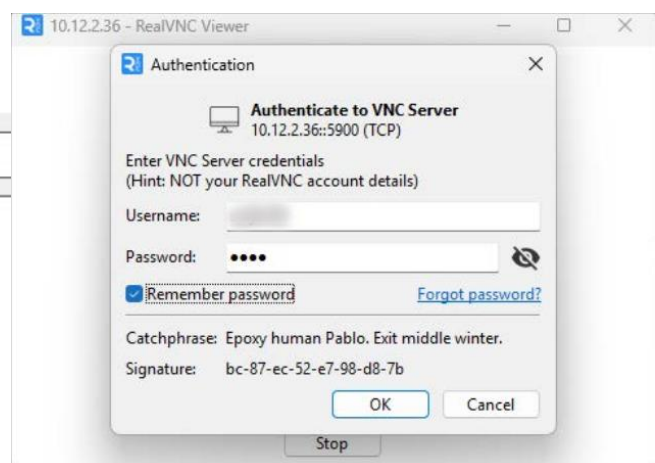
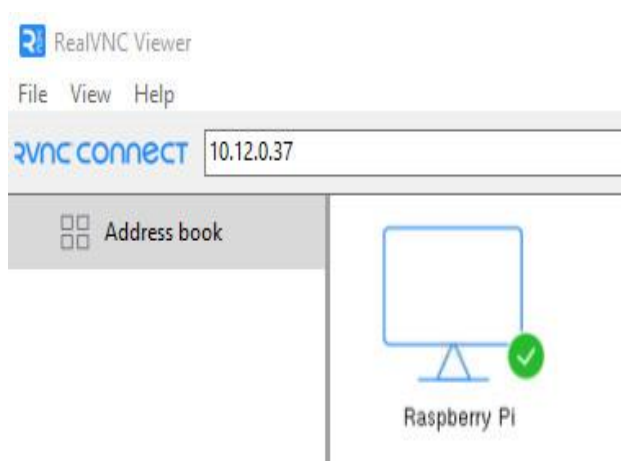
Setting up RealVNC Viewer

Once the Raspberry Pi was running, it was only usable through the touchscreen. For easier configuration throughout the project, I decided to enable remote desktop access first.

First, I retrieved the IP address directly on the Raspberry Pi using Terminal and typing in:

```
maciej@:~$hostname -I
```

With the IP address noted, I prepared RealVNC Viewer on my computer so I could connect remotely. I simply entered the Raspberry Pi's IP address into the VNC Viewer search bar and logged in with my Raspberry Pi credentials.

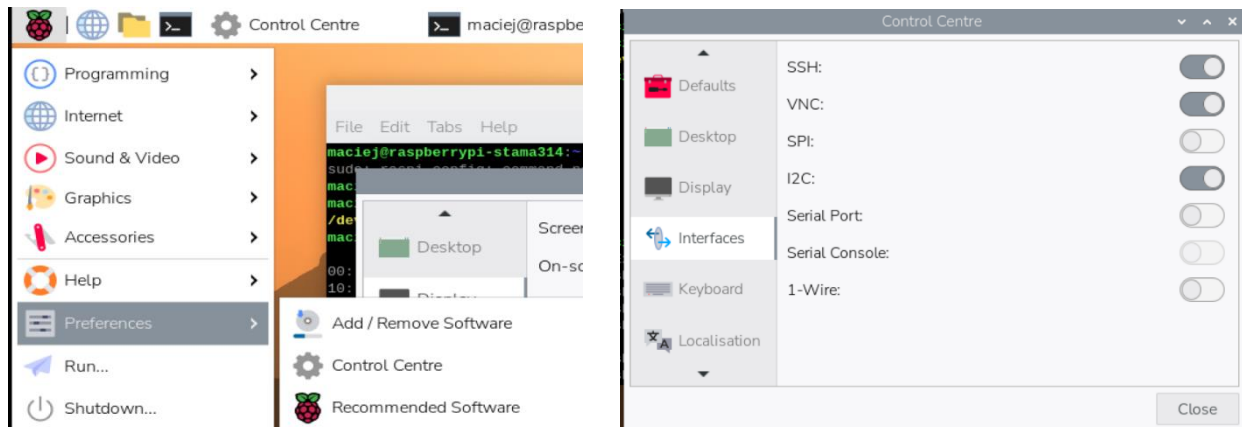


Connecting to our Raspberry Pi through VNC Viewer

Before I could remotely connect to my Raspberry Pi, I first had to enable the VNC Server on the device. There are two ways to do this:

1. Through the Raspberry Pi GUI

I opened the *Raspberry Pi Configuration* menu, went to the **Interfaces** tab, and enabled **VNC** by checking the corresponding option.

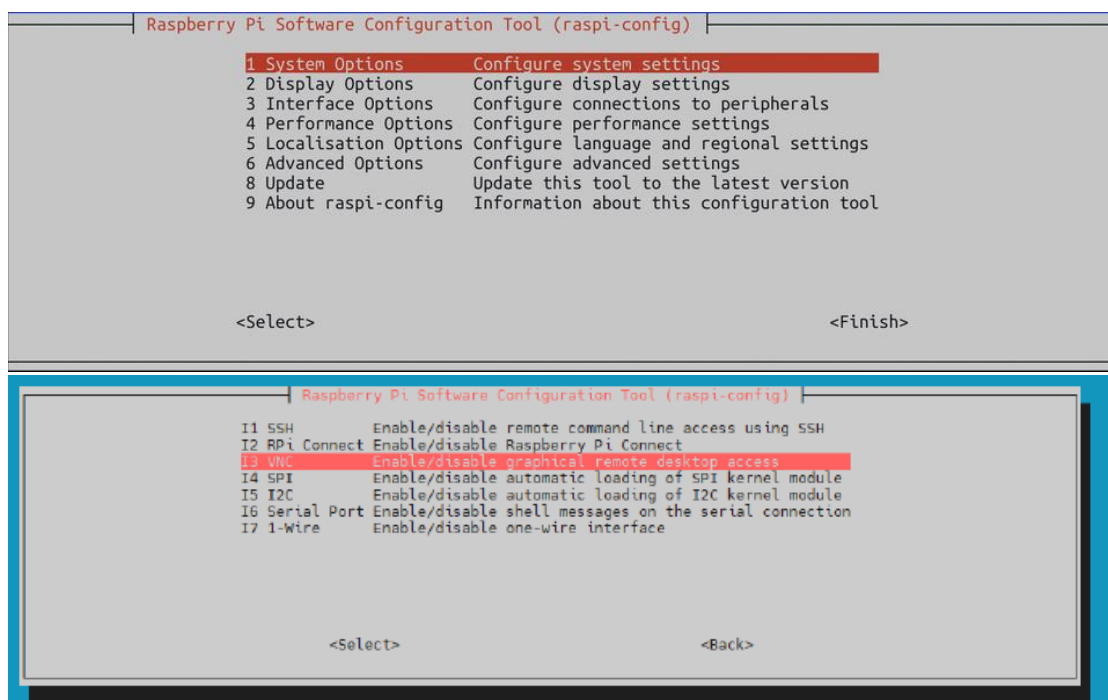


2. Through the terminal using raspi-config

Alternatively, I could enable it via:

```
~$sudo raspi-config
```

Then I navigated to **Interface Options** → **VNC** and selected *Enable*.



First Setup

With remote access working, I continued with the initial system configuration.

This included applying updates, adjusting the desktop settings, and preparing the device for later stages of the project.

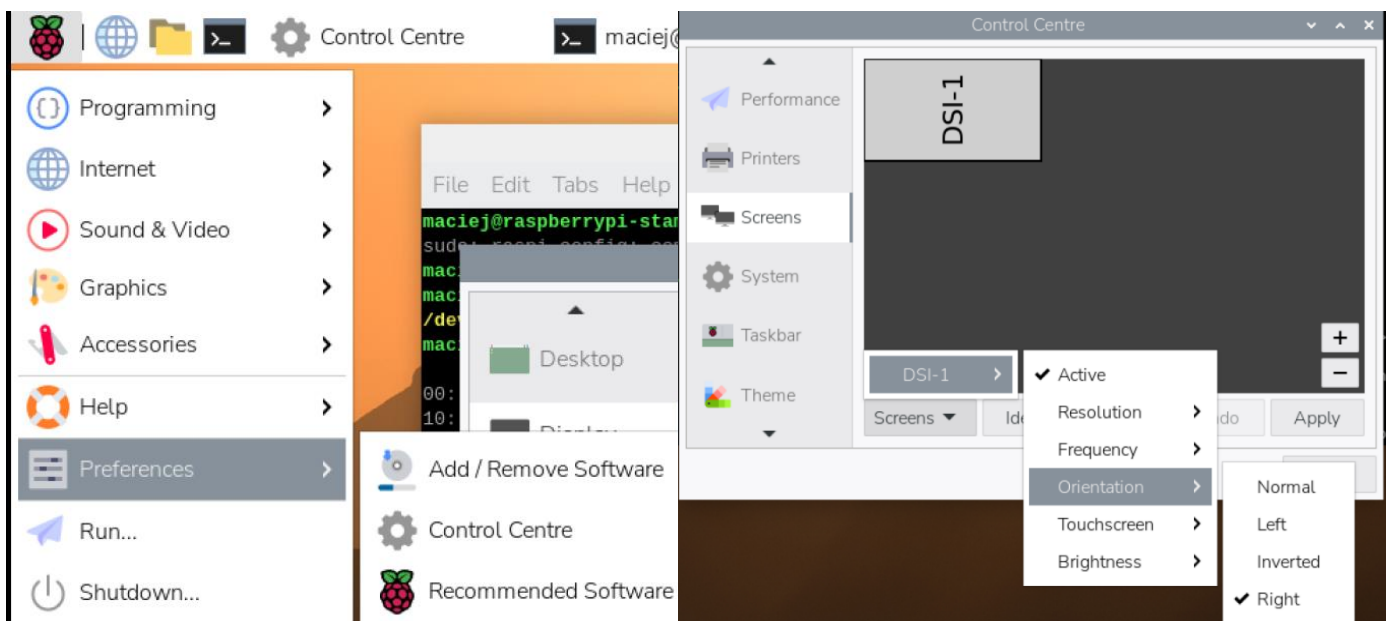
Changing screen orientation and disabling on-screen keyboard

Because the default touchscreen orientation is portrait mode, I changed it to landscape mode for usability.

I did this through:

**Raspberry Pi Settings → Screens → DSI-1 → Orientation → Right/Left
(depending on how the screen is positioned)**

A reboot ensured the changes were applied properly.



There is also an automatic on-screen keyboard feature which activates whenever there is a prompt or input place like for example Terminal, that is clicked on or activated. To disable that feature, it is almost the same way as for the screen orientation settings.

I did this through:

**Raspberry Pi Settings → Display → On-screen Keyboard → Disabled
(no reboot needed)**

Setting Up SSH Key Authentication

To enable secure password-based SSH access for the project, I generated an SSH key pair on my physical computer and added the public key to the Raspberry Pi. This ensures that I can connect to the Pi through SSH without manually typing a password each time.

First, I created the SSH key pair on my Windows machine in Command Prompt using the command:

```
C:\Users\stama314>ssh-keygen
Generating public/private ed25519 key pair.
Enter file in which to save the key (C:\Users\stama314\.ssh/id_ed25519):
Enter passphrase (empty for no passphrase):
Enter same passphrase again:
Your identification has been saved in C:\Users\stama314\.ssh/id_ed25519
Your public key has been saved in C:\Users\stama314\.ssh/id_ed25519.pub
The key fingerprint is:
SHA256:L2wVFxERD2ul+7PJ+G6U4Mk20hsA7gFxn1riJd2F2SI ltpemdomain\stama314@k22-02
The key's randomart image is:
+--[ED25519 256]--+
|      . . B0o |
|    o oEo+O. |
|  . + B.*.. |
| + B + . . |
|  S o . . + |
| o + . . O |
| = . . =o. |
| . . o+.+ |
|      o** |
+-----[SHA256]-----+
```

This generated two files inside my user profile directory, typically located in:
C:\Users\<my-username>\.ssh\

The public key file (for example: id_rsa.pub) is the one that needs to be transferred to the Raspberry Pi.

To do that, I used **WinSCP** and copied the id_rsa.pub file into a SSH directory on the Raspberry Pi,:

/home/<pi-user>/.ssh/

C:\Users\stama314\.ssh\				/home/maciej/.ssh/				
Name	Size	Type	Changed	Name	Size	Changed	Rights	Owner
..		Parent directory	18.11.25 08:55:31	..		24.11.25 10:50:04	rwx-----	maciej
config	1 KB	File	24.11.25 10:55:14	authorized_keys	1 KB	18.11.25 08:55:31	rw-rw-r--	maciej
id_ed25519	1 KB	File	18.11.25 08:55:31					
id_ed25519.pub	1 KB	Microsoft Publish...	18.11.25 08:55:31					
known_hosts	2 KB	File	24.11.25 10:39:12					
known_hosts.old	1 KB	OLD File	17.11.25 10:27:47					

Then I rename the public key into the SSH authorized keys file:

```
sudo mv id_rsa.pub authorized_keys
```

Finally, I ensured proper permissions so SSH would accept the file:

```
chmod 600 ~/.ssh/authorized_keys
```

```
chmod 700 ~/.ssh
```

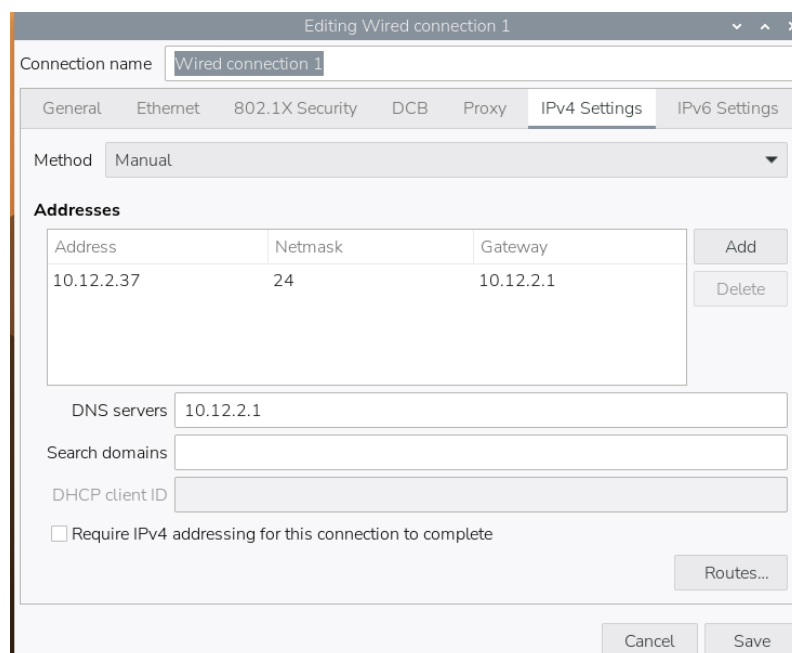
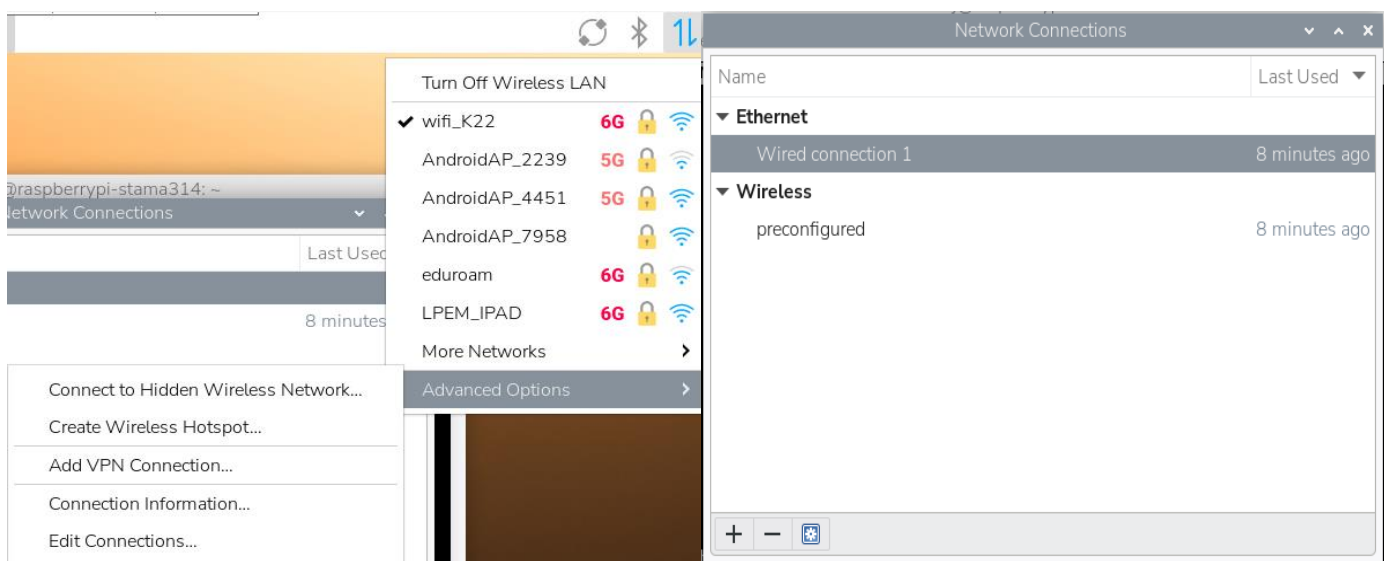
Setting up static IP address for the Ethernet interface

To ensure consistent network access during development, I configured a static IP address on the Ethernet interface.

I opened:

Network icon → Advanced Options → Edit Connections → Wired connection (click on it to access) → IPv4 Settings → Set Method to “Manual”

I then assigned an IP address where the last digits matched the information when using the “hostname -l” command and assigned also a DNS server with the same address as the main device has because most local networks use the router for both internet routing and DNS forwarding. Making Raspberry Pi resolve domain names reliably through the network’s main router.



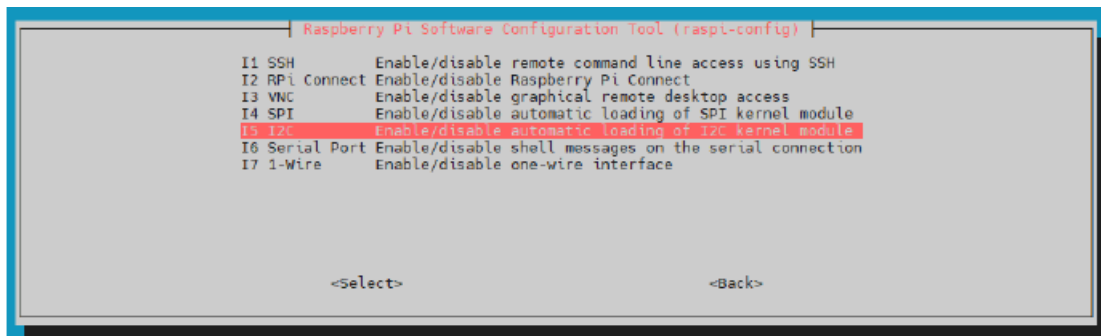
Enabling I2C Interface

Since the project uses I2C-based sensors, I enabled the I2C interface, the same way as enabling VNC, via:

```
~$sudo raspi-config
```

Then:

Interface Options → I2C → Enable



Installing Firmware Files and configuring them

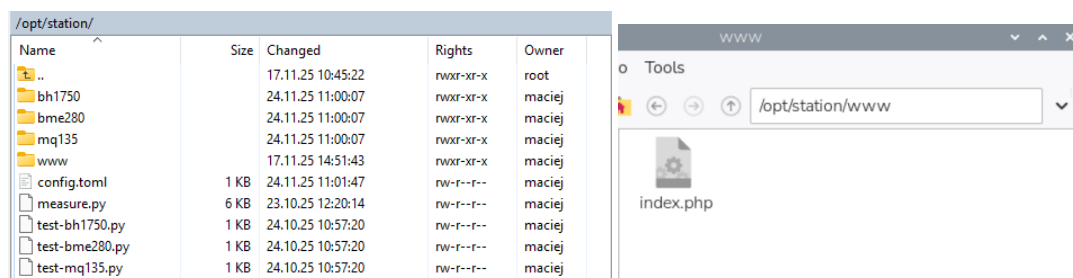
For this part, I needed to transfer firmware, script and website files to specific directories in Raspberry Pi's folders to ensure the correct functionality of our station. There are many ways to do this, but I chose to use WinSCP for this.

Link to the files provided for testing: [Ressourcen.zip](#)

Once the files were downloaded, I then created the required folders (*opt/station*, *var/station*, *tmp/station*) and set permissions appropriately to each of them

```
~$sudo mkdir -p /opt/station/www
~$sudo mkdir -p /var/station
~$sudo mkdir -p /tmp/station
~$sudo mkdir -p /var/www/html
~$sudo chown -R stama314:stama314 /opt/station /var/station
/tmp/station /var/www/html
~$sudo chmod -R 755 /opt/station /var/station /tmp/station
/var/www/html
```

I copied the firmware files and scripts to the */opt/station* directory and a simple website PHP script into both */var/html/www* and */opt/station/www*.



Installing Docker and PHP with Apache2

To run the project's web interface, I installed Docker using the official installation script:

```
~$curl -sSL https://get.docker.com | sh
```

Once the installation is complete, Docker displays the following confirmation message.

```
=====
To run Docker as a non-privileged user, consider setting up the
Docker daemon in rootless mode for your user:

    dockerd-rootless-setuptool.sh install

Visit https://docs.docker.com/go/rootless/ to learn about rootless mode.

To run the Docker daemon as a fully privileged service, but granting non-root
users access, refer to https://docs.docker.com/go/daemon-access/

WARNING: Access to the remote API on a privileged Docker daemon is equivalent
to root access on the host. Refer to the 'Docker daemon attack surface'
documentation for details: https://docs.docker.com/go/attack-surface/
=====
```

Then I created a Docker container running PHP 8.4 with Apache2:

```
~$docker run -d --name php -p 8080:80 -v /tmp/station:/tmp/station -v
opt/station/www:/var/www/html php:8.4-apache
```

docker run

Creates and starts a new Docker container.

-d

Runs the container in the background so the terminal remains available.

--name php

Assigns the name **php** to the container for easier management (for example `docker start php`).

-p 8080:80

Maps port **8080** of the Raspberry Pi to port **80** inside the container, making the website accessible via `http://localhost:8080`.

-v /tmp/station:/tmp/station

Shares the sensor data folder between the host system and the container. This allows the PHP website to access the measurement data created by the Python script.

-v /opt/station/www:/var/www/html

Connects the local website files to Apache's web directory inside the container, allowing files like `index.php` to be displayed.

php:8.4-apache

Uses the Docker image containing **PHP 8.4** and the **Apache** web server.

I then edited `config.toml` that is located in `/opt/station` and updated it (for now it's just these information added since we haven't configured sensors yet):

- `station_serial` to my assigned IAM number
- path to `/opt/station`

```

/opt/station/config.toml - Mousepad
File Edit Search View Document Help
station_serial = "STAMA314"      # Serial number of the station
interval = 2                     # Time between measurements
toggle_intensity = 100          # The light level at which the LED should be turned on
[i2c]                           # I2C-Addresses of the sensors
bme280 = 0x76
bh1750 = 0x23
mq135 = 0x48
[destination]                   # Information about the server
ip = "10.12.2.39"               # IP-Address or Hostname of the Server
path = "api/receive.php"        # Path to the script that accepts the measurements

```

Autostart function configuration

To make the station start automatically at boot, I created a small shell script:

```
~$sudo nano ~/.config/labwc/autostart
```

```

GNU nano 8.4                  .config/labwc/autostart
#!/bin/sh

# Create temp folder mkdir -p /tmp/station
mkdir -p /tmp/station

# Start your python script
sh -c "python3 /opt/station/measure.py -o > /tmp/station/result.csv" &

# Start docker container
sudo docker start php

# Wait for service to start
sleep 5

# Start measurement script using virtual environment
cd /opt/station
source venv/bin/activate
#sudo python3 measure.py &

# Wait for container to start
sleep 10

# Launch Chromium in kiosk mode
chromium --kiosk http://localhost:8080

```

The script makes a temporary folder, sets a python script `measure.py` (which will run it in the background while shell is running). To then redirect the output of it to `/tmp/station` with dummy data inside `result.csv`. Then it tells docker to start a “php” container. Next it is running the python script `measure.py` using virtual environment. Lastly, it launches the Chromium browser with measurement page in kiosk mode.

After saving the file, I made it executable:

```
~$chmod +x ~/.config/labwc/autostart
```

And then execute the bash script for testing:

```
~$ bash ~/.config/labwc/autostart
```

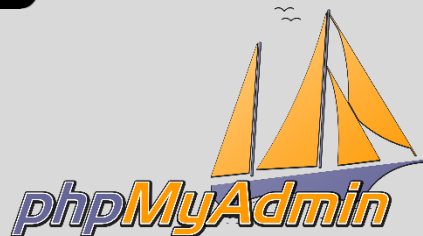
On reboot, the web interface launches automatically in Fullscreen.



Note:

If you see any errors at the beginning when executing the bash script, just ignore them, it is checking if a php docker container exist and that GPIO pins on Raspberry Pi are currently being used. When starting up a Raspberry Pi, it will take a bit for the Measurement Screen to show up.

REPIFI1: SPRINT L2 Report Database



Database Modelling and SQL Implementation

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 24 November – 5 December 2025

Contents

Sprint 2 of the Project.....	28
Goal	28
Objective.....	28
Requirements	29
<i>System Rules and Relationships</i>	29
Data Models	30
<i>Conceptual Data Model (CDM)</i>	30
<i>Logical Data Model (LDM)</i>	31
Database Implementation.....	32
Local Database Setup & Testing.....	35

Sprint 2 of the Project

Goal

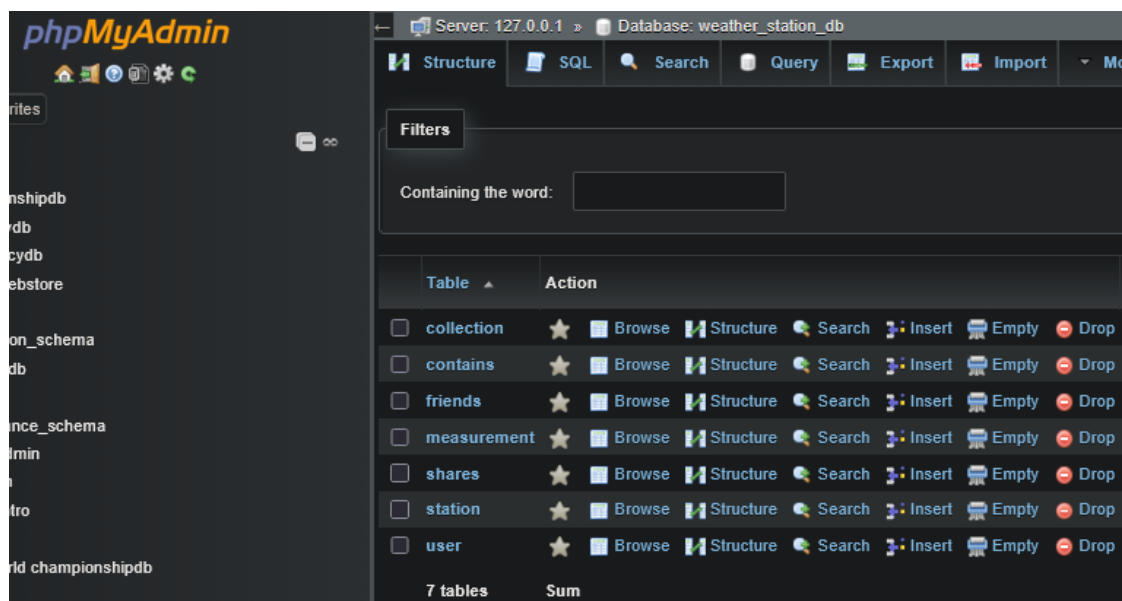
The goal of second sprint was to design and prepare a functional database for the upcoming phases of the project.

Based on the requirements specified in the project description, a Conceptual Data Model (CDM) and a Logical Data Model (LDM) were created to define the necessary entities, relationships, and data structures. Using these models as a foundation, the final SQL database was implemented to ensure correct storage and management of all user, station, and measurement data.

Objective

The objective of this sprint was to create a complete database structure capable of storing all data required for the project. As part of this process, the necessary entities and relationships were first analyzed and outlined in a Conceptual Data Model (CDM), which was then translated into a detailed Logical Data Model (LDM).

Using these models, the final SQL schema was developed and prepared for implementation in a local database environment.



Requirements

Introduction

To ensure that all measurement data from the individual stations can be stored and processed well, the database had to fulfill several requirements from the project description. Requirements describe which types of data must be stored, how the entities relate to one another, and how users and stations interact within the system.

The most important entities are *Users*, *Stations*, *Measurements*, and *Collections*, each with specific attributes and constraints.

Users

User data had to include unique **usernames**, **full names**, **email addresses**, and **information** about whether a user is a normal **user** or an **administrator**.

Stations

Stations required a unique **serial number** assigned by the company, along with an optional **station name** and **description**.

Measurements

For each station, measurements needed to store a **timestamp** and **multiple environmental sensor values** such as *temperature*, *humidity*, *air pressure*, *light intensity*, and *air quality*.

Collections

In addition, collections were required to group measurements, each with its own **name** and **description**, while also allowing measurements to appear in multiple collections.

System Rules and Relationships

The relationships between these entities were defined by the project specifications:

User–Station Relationship

- A station is created by an **administrator**.
- Once purchased, a customer can **register the station** under their account.
- A user can own **multiple** stations.
- Each station belongs to **exactly one** user.

Station–Measurement Relationship

- Measurements are recorded **by a station**.
- Each measurement belongs to **one station**.
- A station can produce **many** measurements.

Collections

- Users can create **any number of collections**.
- A collection can contain **zero or more measurements**.
- The same measurement can appear in **multiple** collections.
- A collection belongs to **exactly one user**.

User Social Features

- Users can add other users as **friends** (optional, many-to-many).
- Collections can be **shared** with other users:
 - One collection → many users
 - One user → access to many collections

Data Models

Conceptual Data Model (CDM)

The Conceptual Data Model represents the high-level structure of the database, identifying the main entities and their relationships without technical details. The model includes:

Users – project members like myself who register and manage weather stations.

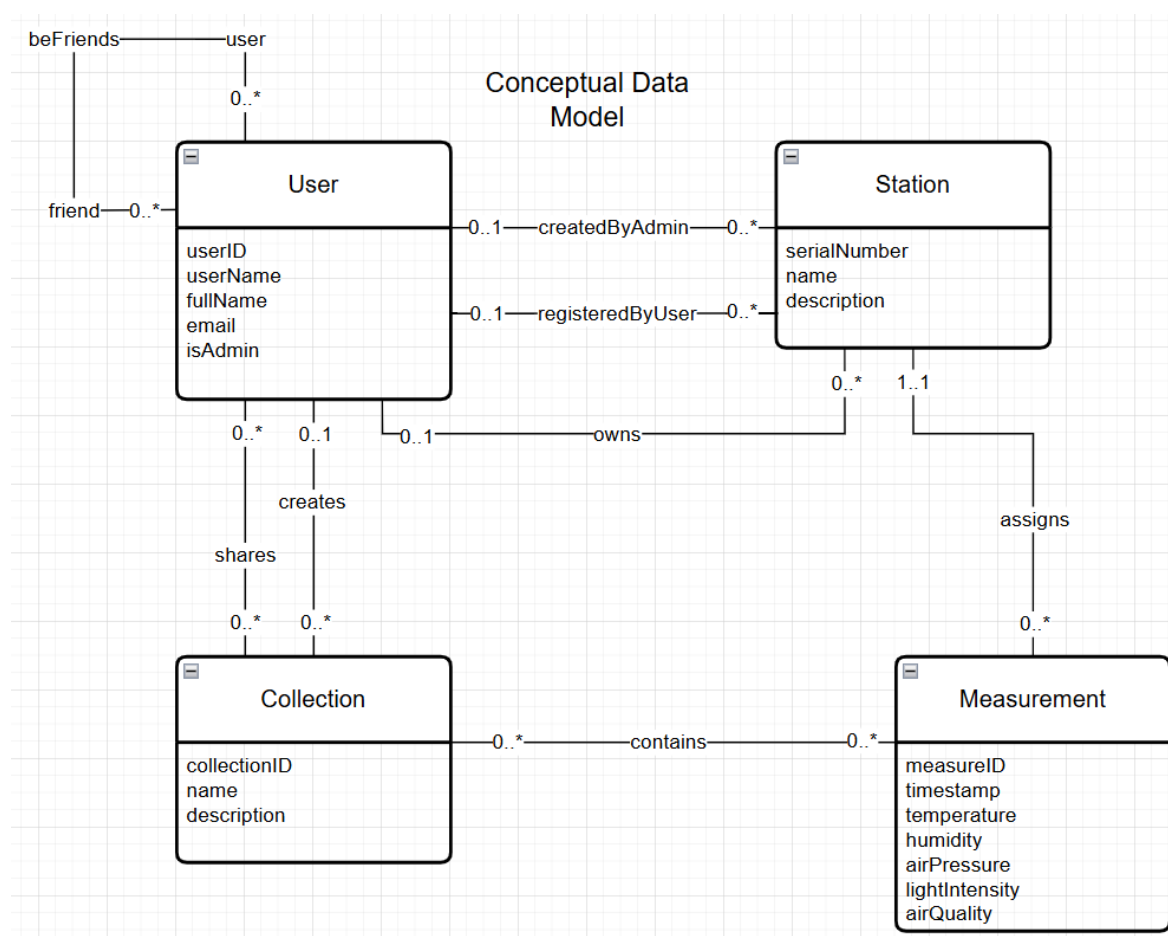
Stations – each Raspberry Pi 4 unit (e.g., SN-1001) installed in a specific location to collect environmental data.

Measurements – data collected through sensors every 2 seconds (temperature, humidity, pressure, light, air quality).

Collections – group of measurements gathered in a specific time period

Friends – user-to-user connections to enable data sharing.

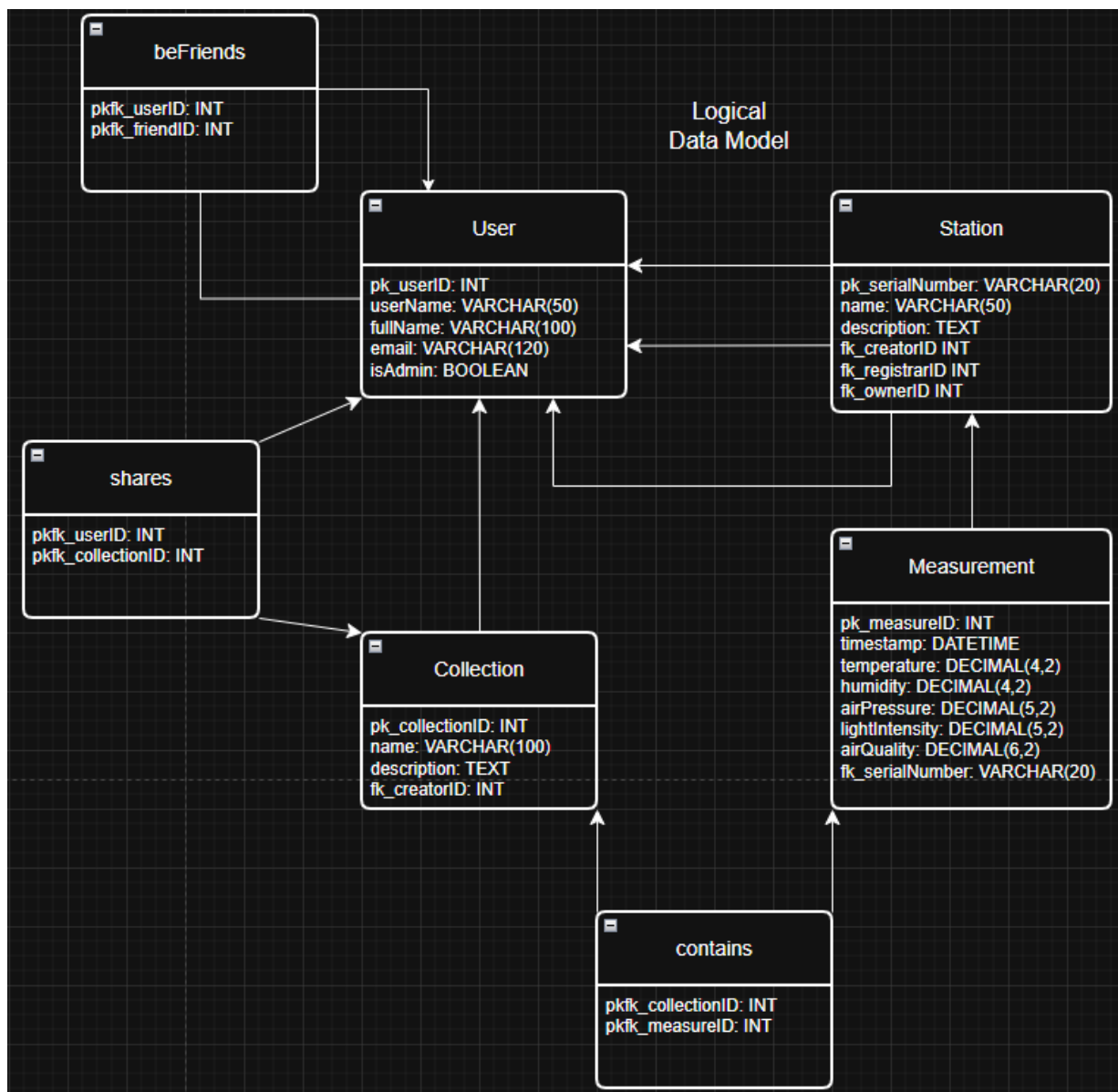
Collection Sharing – permissions that allow friends to view each other's collections.



Logical Data Model (LDM)

The Logical Data Model translates the conceptual entities into a structured, table based design suitable for implementation in a SQL database. It defines:

- Tables corresponding to each entity
- Primary and foreign keys to enforce relationships
- Specific data types, constraints, and indexes for performance and integrity
- Normalization to eliminate redundancy and ensure efficient storage



Database Implementation

The database was implemented using a **relational (SQL) database** designed to store and manage all data for the Portable Indoor Feedback system. The implementation includes table creation, relationship enforcement, and realistic sample data for testing.

SQL Code

Below is the complete SQL script used to create the database, tables, and insert sample data:

```
-- DATABASE CREATION
CREATE DATABASE weather_station_db;
USE weather_station_db;

CREATE TABLE User (
    pk_userID INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    fullName VARCHAR(100) NOT NULL,
    email VARCHAR(120) NOT NULL,
    isAdmin BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE Station (
    pk_serialNumber VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100),
    description TEXT,
    fk_creatorID INT NULL,
    fk_registrarID INT NULL,
    fk_ownerID INT NULL,
    FOREIGN KEY (fk_creatorID) REFERENCES User(pk_userID),
    FOREIGN KEY (fk_registrarID) REFERENCES User(pk_userID),
    FOREIGN KEY (fk_ownerID) REFERENCES User(pk_userID)
);

CREATE TABLE Measurement (
    pk_measurementID INT PRIMARY KEY AUTO_INCREMENT,
    timestamp DATETIME NOT NULL,
    temperature DECIMAL(4,2),
    humidity DECIMAL(4,2),
    airPressure DECIMAL(5,2),
    lightIntensity DECIMAL(5,2),
    airQuality DECIMAL(6,2),
    fk_stationSerialNumber VARCHAR(20) NOT NULL,
    FOREIGN KEY (fk_stationSerialNumber) REFERENCES Station(pk_serialNumber)
);
```



```
CREATE TABLE Measurement (
    pk_measurementID INT PRIMARY KEY AUTO_INCREMENT,
    timestamp DATETIME NOT NULL,
    temperature DECIMAL(4,2),
    humidity DECIMAL(4,2),
    airPressure DECIMAL(5,2),
    lightIntensity DECIMAL(5,2),
    airQuality DECIMAL(6,2),
    fk_stationSerialNumber VARCHAR(20) NOT NULL,
    FOREIGN KEY (fk_stationSerialNumber) REFERENCES Station(pk_serialNumber)
);

CREATE TABLE Collection (
    pk_collectionID INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    description TEXT,
    fk_creatorID INT NOT NULL,
    FOREIGN KEY (fk_creatorID) REFERENCES User(pk_userID)
);

CREATE TABLE friends (
    pkfk_userID INT NOT NULL,
    pkfk_friendID INT NOT NULL,
    PRIMARY KEY (pkfk_userID, pkfk_friendID),
    FOREIGN KEY (pkfk_userID) REFERENCES User(pk_userID),
    FOREIGN KEY (pkfk_friendID) REFERENCES User(pk_userID)
);

CREATE TABLE contains (
    pkfk_collectionID INT NOT NULL,
    pkfk_measurementID INT NOT NULL,
    PRIMARY KEY (pkfk_collectionID, pkfk_measurementID),
    FOREIGN KEY (pkfk_collectionID) REFERENCES Collection(pk_collectionID),
    FOREIGN KEY (pkfk_measurementID) REFERENCES Measurement(pk_measurementID)
);

CREATE TABLE shares (
    pkfk_collectionID INT NOT NULL,
    pkfk_userID INT NOT NULL,
    PRIMARY KEY (pkfk_collectionID, pkfk_userID),
    FOREIGN KEY (pkfk_collectionID) REFERENCES Collection(pk_collectionID),
    FOREIGN KEY (pkfk_userID) REFERENCES User(pk_userID)
);

-- Sample data insertion (see following sections)
```

Data Types & Constraints

Column Type	Data Type Used	Example	Constraint Applied
IDs	INT	pk_userID = 3	PRIMARY KEY, AUTO_INCREMENT
Names / Text	VARCHAR (50-120)	username = 'user_mike'	UNIQUE, NOT NULL
Measurements	DECIMAL (4,2-6,2)	temperature = 20.50	NOT NULL (optional)
Timestamps	DATETIME	timestamp = '2025-01-10 08:15:00'	NOT NULL
Foreign Keys	INT / VARCHAR	fk_stationSerialNumber = 'SN-1001'	FOREIGN KEY REFERENCES

Key Constraints Explained

- **PRIMARY KEY** – Uniquely identifies each row in a table.
Example: `pk_userID` in the `User` table.
- **FOREIGN KEY** – Ensures referential integrity by linking tables.
Example: `fk_creatorID` in `Station` references `pk_userID` in `User`.
- **UNIQUE** – Prevents duplicate values in a column.
Example: `username` must be unique across all users.
- **NOT NULL** – Ensures a column cannot be empty.
Example: `timestamp` in `Measurement` must always be provided.
- **AUTO_INCREMENT** – Automatically generates sequential IDs for new rows.
Example: New users receive an incremented `pk_userID`.
- **DEFAULT** – Assigns a default value if none is provided.
Example: `isAdmin` defaults to `FALSE`.

Sample Data Insertion

Realistic dummy data was inserted into each table to simulate real-world usage and enable immediate testing of queries, frontend integration, and relationship validation.

The inserted data includes:

- **6 users** (2 administrators, 4 regular users)
- **5 weather stations** with unique serial numbers
- **9 measurement records** with realistic sensor values
- **4 collections** for organizing stations
- **Friend relationships** and **collection sharing** permissions

Example of inserted measurement data:

```
INSERT INTO Measurement (timestamp, temperature, humidity, airPressure,
lightIntensity, airQuality, fk_stationSerialNumber)
VALUES ('2025-01-10 08:15:00', 18.5, 60.2, 1012.3, 200.0, 350.0, 'SN-1001');
```

Local Database Setup & Testing

Environment

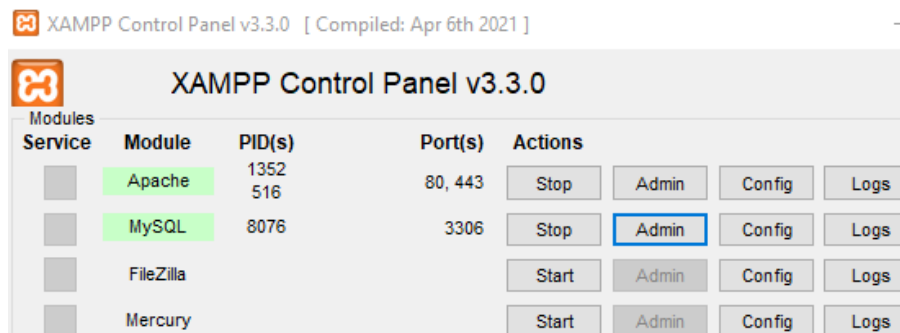
&

Tools

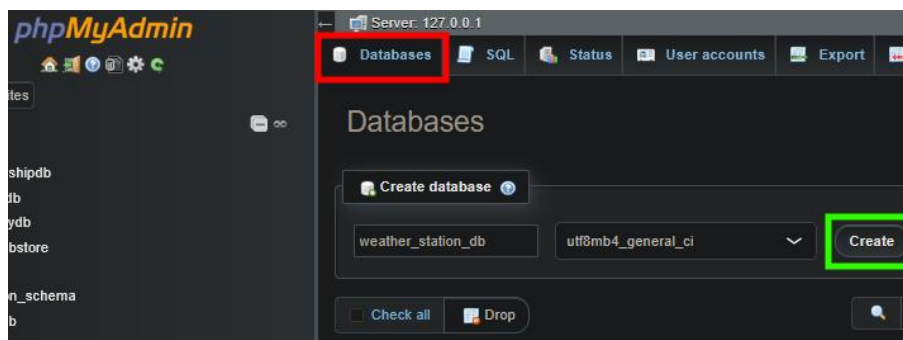
The database was set up and tested using **phpMyAdmin** running on a local **XAMPP** stack (Apache + MySQL). phpMyAdmin provided a graphical interface for executing SQL, managing tables, and verifying data integrity.

Setup Procedure

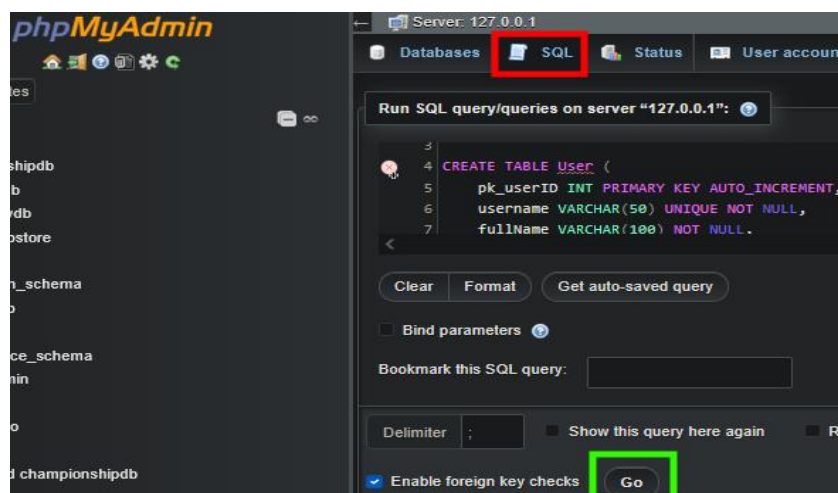
1. Access phpMyAdmin via <http://localhost/phpmyadmin> or by using build in feature in XAMPP by clicking on Actions buttons “Admin” on the *MySQL* row.



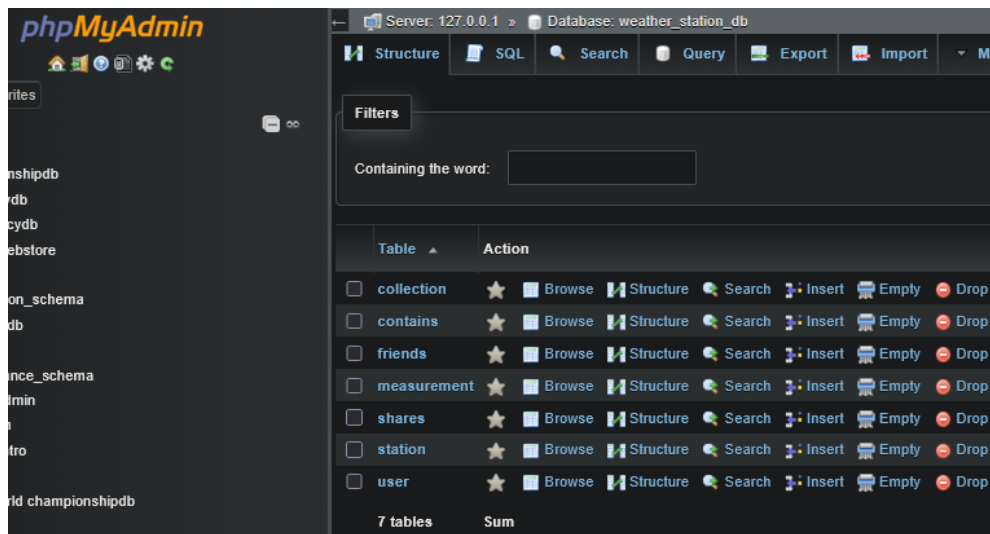
2. Create a new database named `weather_station_db` using the phpMyAdmin Database tab.



3. Import the SQL script (`database_setup.sql`) using the SQL tab.



4. Verify table creation in the database Structure view.



Export & Backup

In case of transferring the database from one machine to another or preserving the current state for later website integration. A full backup was created using phpMyAdmin's export function. The entire *weather_station_db*, including table structures, constraints, and all sample data, was exported as a single SQL file (*weather_station_backup.sql*). This file can be easily imported into any MySQL-compatible system, ensuring seamless deployment or recovery.

Using phpMyAdmin's **Export** feature:

1. Selected the *weather_station_db* database.
2. Chose **Quick** (minimal settings) or **Custom** (all settings) export method with **SQL** format.
3. Included all tables, structure, and data.
4. Saved as *weather_station_backup.sql* for portability and future deployment.

REPIFI1: SPRINT L3 Report Website



Website Application Development for User and Data Management

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 8 December 2025 – 5 February 2026

Contents

Sprint 3 of the Project	39
Goal	39
Objective	39
Planning & Development Process	40
Initial Planning (08/12/25 – 14/12/25)	40
First Implementation Phase (15/12/25 – 21/12/25)	42
Expansion of Website Pages (22/12/25 – 04/01/26)	42
Architecture Reorganising	44
Final Development Phase (05/01/26 – 25/01/26)	44
Final Cleanup & Testing (02/01/26)	45
Testing & Validation	46
Final Result	46
Conclusion	46

Sprint 3 of the Project

This sprint focuses on the development of the website for the Indoor Weather Station project. The purpose of this user story was to design and implement a functional web interface that allows users to view measurement data, interact with stations, and manage their account within the system.

The website sprint fits after the initial hardware-focused sprint (L1), where the Raspberry Pi and base system were prepared. While L1 ensured that data could be generated and displayed locally, this sprint extends the project by introducing a centralized web interface that structures, presents, and manages this data in a user-friendly way. The website also prepares the foundation for later project phases, primarily the integration of sensor data from the stations and the continued expansion of website functionality based on project requirements.

Goal

The goal of this sprint was to create a structured and usable website that serves as the main interaction point between the user and the Indoor Weather Station system on a browser. The website is intended to provide clear navigation, readable presentation of measurement data, and basic management features for users and stations.

The main focus areas of the website were:

- User interaction through a clear and consistent interface
- Viewing measurement data in a structured and accessible way
- Managing stations and user-related pages
- Preparing the system architecture for future project phases and extensions

Objective

The objective of this sprint was to translate the requirements from the User Story document into a working web application. This included building a functional website with multiple pages, implementing a login system with roles, and connecting the website to the provided database.

More specifically, the objectives were:

- To build a functional web interface using PHP, HTML, JS, and CSS
- To implement authentication and role-based access control
- To connect the website to the provided MySQL database using a configuration file
- To follow the User Story requirements, focusing primarily on the Must and Should requirements, while leaving the Could requirements for later implementation

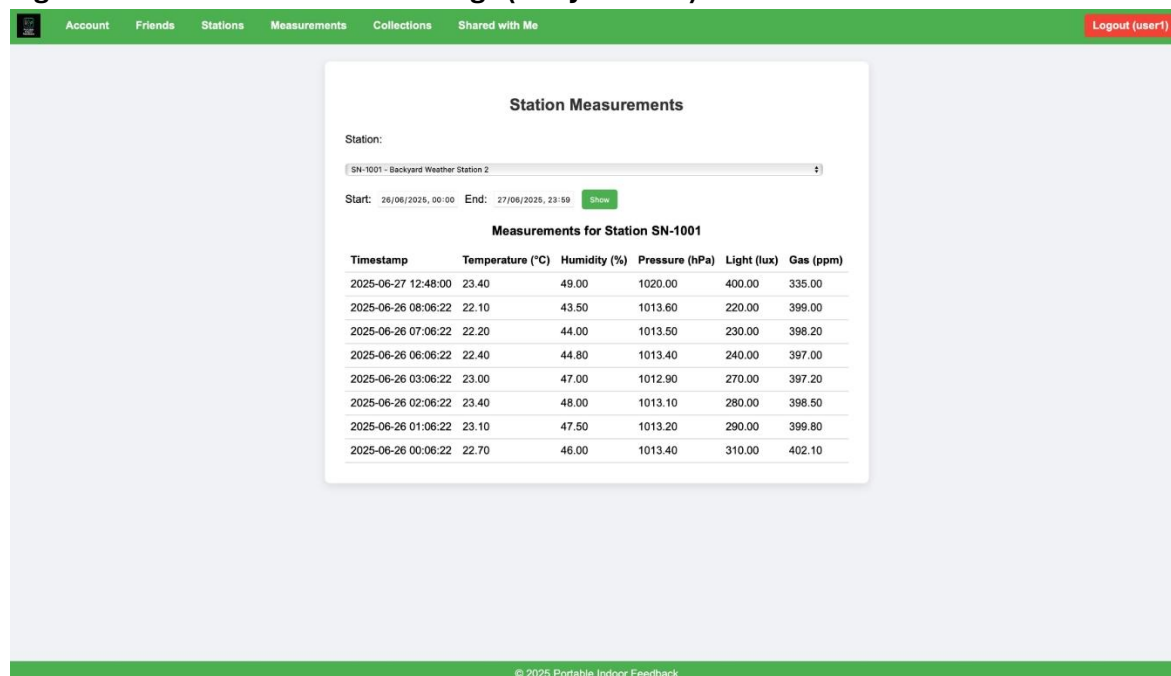
Planning & Development Process

Initial Planning (08/12/25 – 14/12/25)

During the first week, the focus was on planning the overall structure and appearance of the website. This included deciding how pages would be organized, how users would navigate through the website, and how the project files should be structured.

As a starting point, I used the example station measurements page provided in the Work Assignment document. This reference helped me define the basic layout, required input fields, date selectors, and station selection elements. At this stage, no database was available, so the focus remained on layout planning and structural decisions rather than data accuracy. Hard-coded dummy data was used for testing purposes. This allowed basic functionality and layout testing without blocking progress. Basic CSS styling was added, along with shared components such as a header and footer, to give the website a consistent look and feel.

Figure 1: Station Measurements Page (Early Version)



In this first image, it shows the example of what should a station measurements page in browser, provided in the Work Assignment document. Which was used as an idea concept for both structure and style of the pages for my website.

Portable Indoor Feedback

Station Measurements

Station: Living Room

Start Date & Time: dd / mm / yyyy , -- : --

End Date & Time: dd / mm / yyyy , -- : --

Show Measurements

Measurement Results (Concept)

Timestamp	Temp (°C)	Humidity (%)	Pressure (hPa)	Light (lux)	Gas (ppm)
—	—	—	—	—	—

Portable Indoor Feedback System

In this second picture, the main structural elements of the page can be seen, including the date and time selection inputs, station selector, and the area where measurement results are displayed. This page served as the reference point for later database integration and functionality expansion.

Figure 2: Dashboard Page (Early Version)

Portable Indoor Feedback

Welcome, User

This page provides an overview of your stations and measurements.

Current Measurements (Concept)

Temperature: — °C

Humidity: — %

Pressure: — hPa

Light: — lux

Gas: — ppm

Timestamp: —

Your Stations

- Living Room
- Bedroom
- Office

Portable Indoor Feedback System

The dashboard page was designed as the first page visible in theory after login. At this stage, it mainly contained placeholder content and dummy data, but its structure already reflected its future role as a summary and overview page for the user.

First Implementation Phase (15/12/25 – 21/12/25)

On the second week, the main database was provided to us. The project structure was then updated to support real data handling. A dedicated configuration file was created to manage the database connection, ensuring that database credentials and connection logic were centralized and reusable.

Dummy data was then added to the database using phpMyAdmin to allow testing of queries and page output. Existing pages were then remade so that they no longer relied on hard-coded values, but instead read and displayed data directly from the database.

This phase marked a transition from visual prototyping to functional data-driven development.

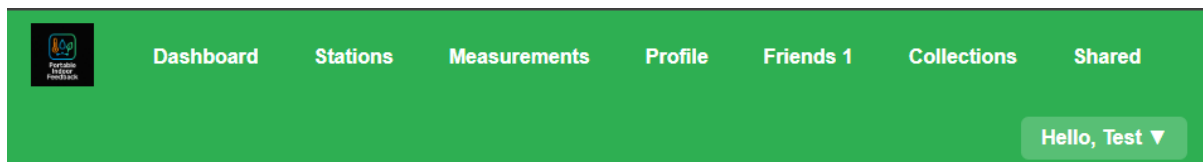
Expansion of Website Pages (22/12/25 – 04/01/26)

During the holiday period, the website was expanded with additional pages mainly intended for normal users. A navigation bar was implemented to allow easy movement between different sections of the website.

Pages such as Collections, Friends, Profile, Shared, and Stations were created one by one. The development process for each page followed a consistent workflow:

1. Creating the basic page structure and layout
2. Implementing the required functionality
3. Connecting the page to the database
4. Adding basic visual elements such as headers, footers, and labels
5. Testing and fixing errors before moving on to the next page

Figure 3: Main Navigation Bar



The navigation bar provides access to all main sections of the website and ensures consistent navigation across user pages. It was implemented early in this phase to support further page development. Later, a small User box was added to give the user an option to choose between different options like Logout or Preferences. A small thing I added to the website logo is that when clicking on it, it will send you to the main Dashboard page.

Figure 4: Example User Page – Collections

Collections

Create and manage collections of measurements from your stations.

Create New Collection

Collection name

Description (optional)

Station

-- select station --

Start date & time

dd/mm/yyyy --:--

End date & time

dd/mm/yyyy --:--

Create Collection

My Collections

Name	Creator	# Measurements	View	Actions	Share
collection 1	admin	10	View	New name Rename Delete	-- Select friend -- Share
collection 1	admin	10	View	New name Rename Delete	-- Select friend -- Share

This figure shows an example of one of user-focused page, where measurement data can be grouped into collections. User can set a name, add a small description, choose which station they want to make a collection from and set a time period from where it should take the measurements to store it in a collection. Once a collection is created, they can either view its contents, change the name of it or share it with their friends.

Architecture Reorganising

As the project grew, it became necessary to organise the architecture to improve readability and maintainability. The codebase was split into backend and frontend parts.

The backend handles data processing, database interaction, and core logic, while the frontend focuses on user-facing pages and layout. Reusable elements such as headers, footers, shared components, and helper functions were moved into dedicated include files.

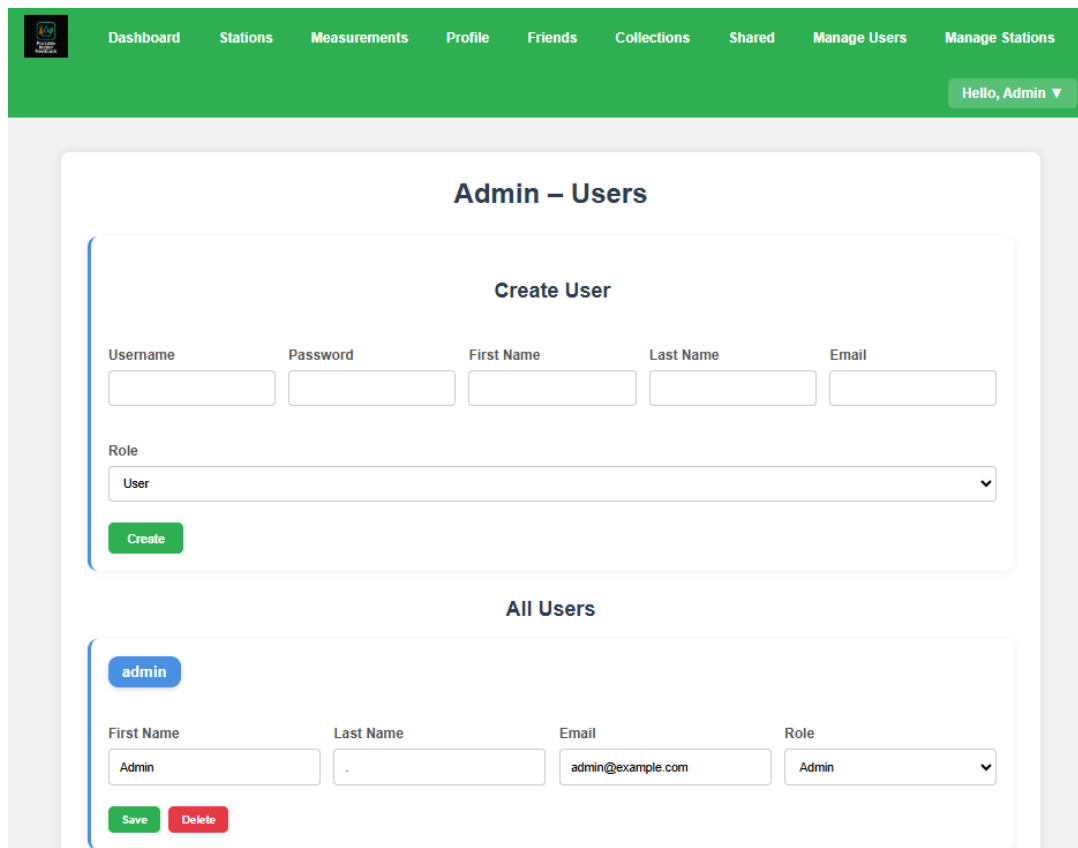
CSS files were organized into an assets directory, and a separate configuration directory was used for database connectivity. An empty admin directory was also prepared early on to support later implementation of administrator-specific functionality.

Final Development Phase (05/01/26 – 25/01/26)

After the holidays, development continued with a focus on more advanced functionality. A login system was implemented, along with role-based access control to differentiate between normal users and administrators.

Administrator pages were added to allow management of users and system-related data. Additional CSS improvements were applied to enhance usability and consistency. During this phase, a significant amount of time was spent on bug fixing, refactoring, and improving existing features based on testing results.

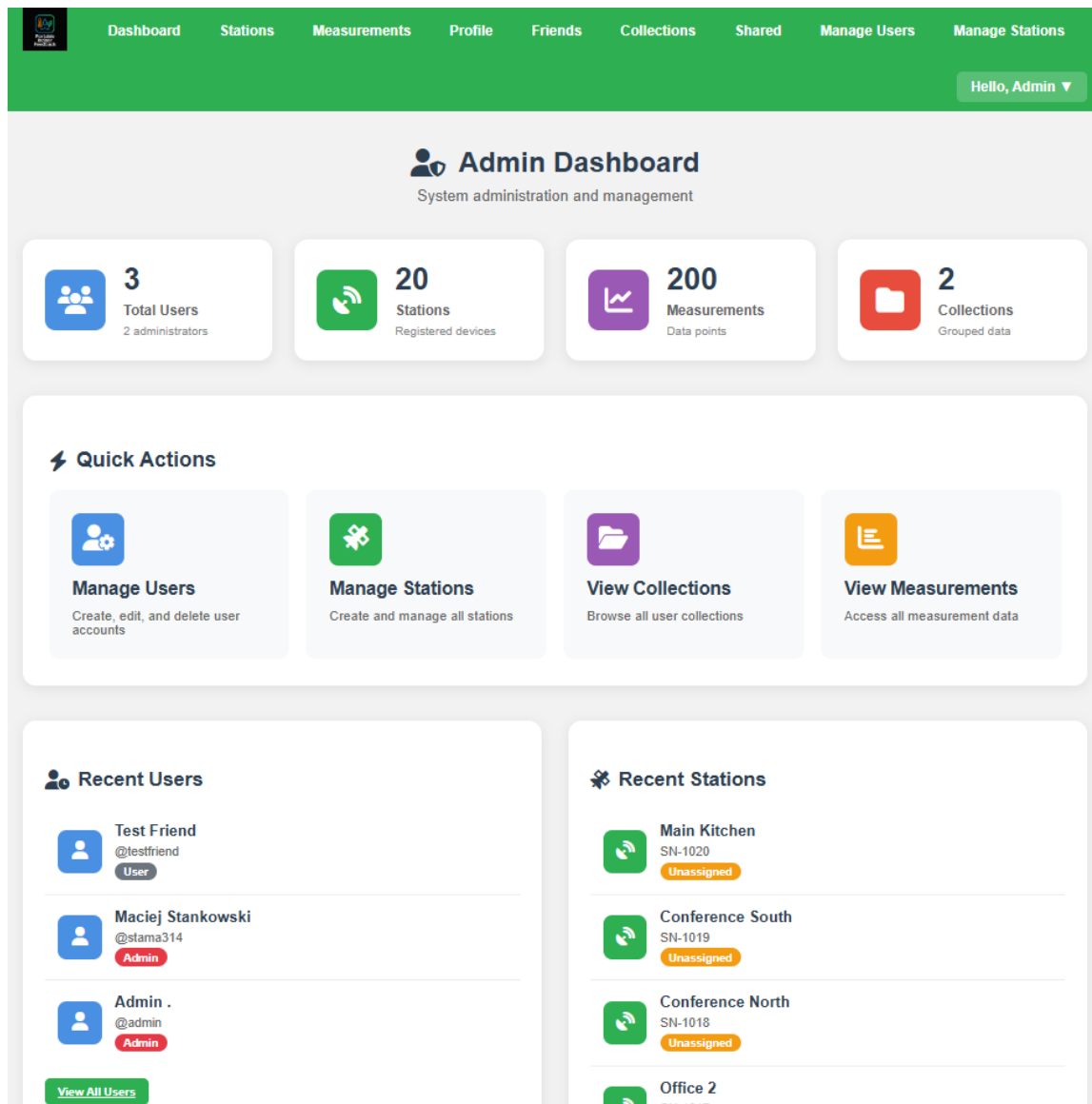
Figure 5: Admin User Management Page



The screenshot displays the 'Admin – Users' management interface. At the top, a green navigation bar contains links to Dashboard, Stations, Measurements, Profile, Friends, Collections, Shared, Manage Users, and Manage Stations. A user greeting 'Hello, Admin' is visible on the right. The main content area is titled 'Admin – Users' and is divided into two sections. The 'Create User' section includes input fields for Username, Password, First Name, Last Name, and Email, a Role dropdown menu currently set to 'User', and a green 'Create' button. The 'All Users' section shows a list of users, with the first entry 'admin' highlighted in a blue box. Below this entry are input fields for First Name (containing 'Admin'), Last Name (containing a dot), Email (containing 'admin@example.com'), and Role (a dropdown menu set to 'Admin'). At the bottom of this section are green 'Save' and red 'Delete' buttons.

This page allows administrators to manage user accounts, assign or revoke administrator rights, and remove users from the system. It demonstrates the separation between normal user functionality and administrator-only features.

Figure 6: Admin Dashboard Page



The admin dashboard serves as the central control panel for administrators, providing an overview of the system and access to administrative functionalities. From this page, administrators can manage users, stations, and measurement data across the entire platform.

Final Cleanup & Testing (02/01/26)

The final stage of the sprint focused on cleanup and validation. Unused files and redundant code were removed, and the overall structure was reviewed for consistency.

The website was tested against the User Story requirements to ensure that all Must and Should requirements were fulfilled. The Could requirements were reviewed but intentionally postponed for later development stages of the project since I had to prioritise in functionality of Must and Should parts.

Testing & Validation

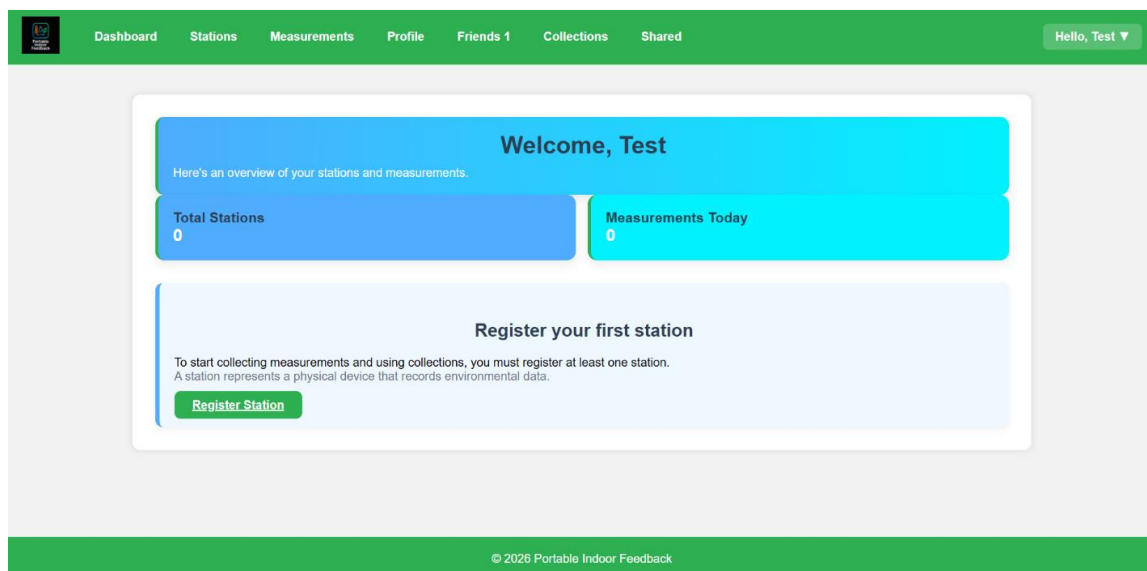
Testing was performed manually throughout the development process. This included checking page navigation, verifying correct behaviour of the login system and user roles, and confirming that database interactions worked as expected.

Error handling was validated by testing incorrect inputs and edge cases, and any issues found were addressed during the bug-fixing phases. Continuous testing during development helped reduce critical issues toward the end of the sprint.

Final Result

At the end of this sprint, the website provides a structured and functional browser interface for the Indoor Weather Station project. Users can log in, navigate through different pages, view measurement-related information, and interact with station-related data.

The website satisfies the core requirements defined in the User Story document and is designed in a way that allows future extensions, such as additional sensor data, improved visualizations, and expanded administrative functionality.



Conclusion

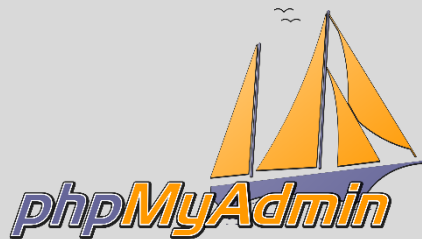
This sprint resulted in a complete and functional website developed by me with a bit of a help from internet and external resources. The project progressed from initial planning and visual prototyping to a fully database-driven application with authentication and role management.

What went well was the development process and early remaking of the project structure, which made later expansion somewhat easier. Areas for future improvement include implementing the remaining Could requirements and further optimizing the user experience. Of course, it is up to me to decide whether I should add additional settings and features or not.

Overall, for this sprint I successfully achieved its goals and provided a solid foundation for the next stages of the Indoor Weather Station project.

REPIF1: SPRINT L4 Report

Web database server



Web Server and Database Environment Configuration

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 2 February – 11 March 2026

Contents

Sprint 4 of the Project	49
Goal	49
Objective	49
System Components	50
Work Setup	51
Server Preparation and System Setup	51
Web Server Configuration	51
phpMyAdmin Configuration	54
Database Configuration	56
HTTPS/ Self-Signed Certificate Configuration	58
Reflection	58

Sprint 4 of the Project

Goal

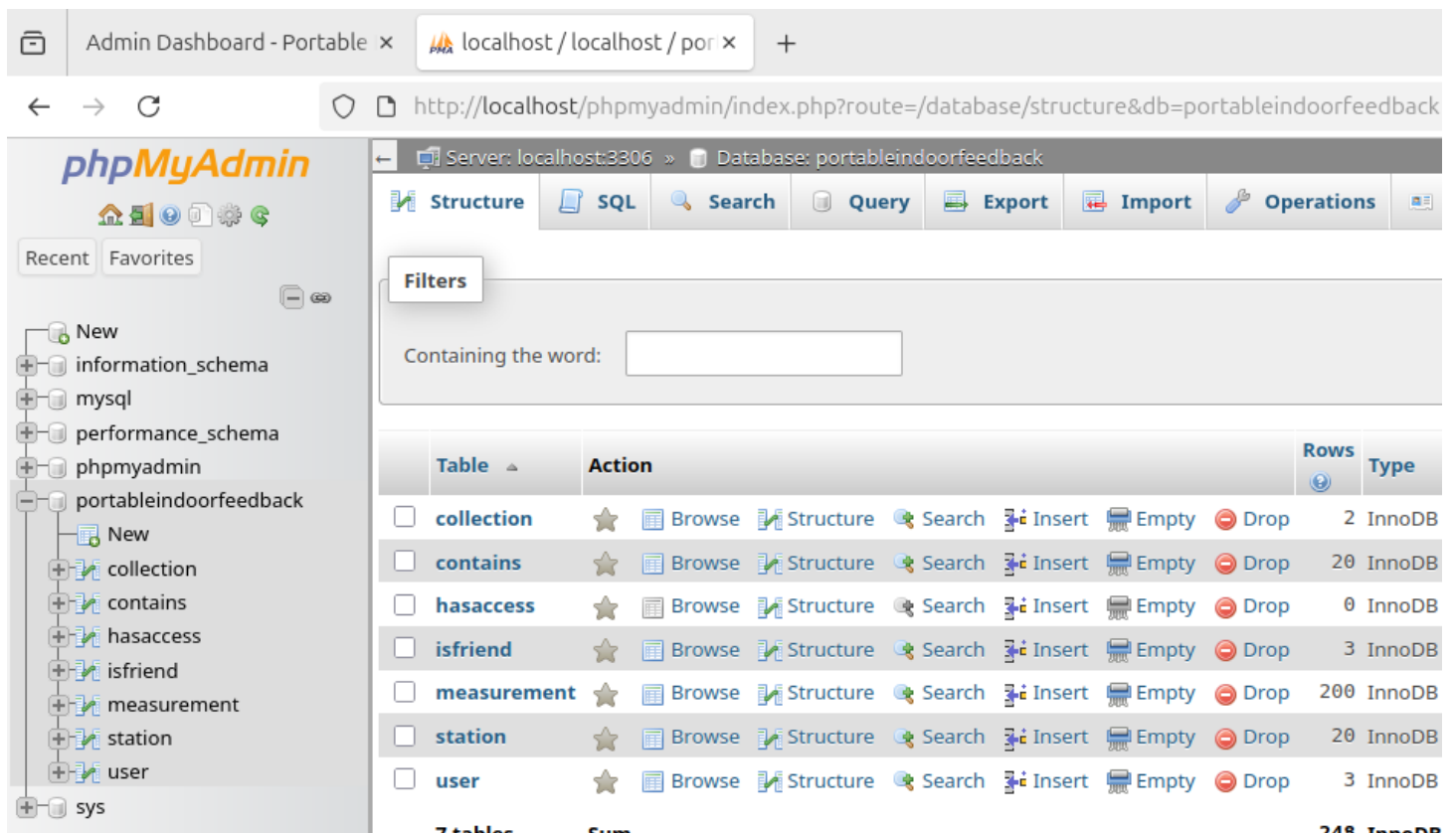
The goal of this sprint was to set up a functional web and database server environment that can be accessed over the network. The server should be capable of hosting a website and managing a database system that can later be used for storing and processing sensor data and other information.

Objective

The objective of this task was to set up a functional web and database server environment for the Portable Indoor Feedback (PIF) project. These components include Apache for the web server, MySQL for database management, PHP for server-side scripting, and phpMyAdmin for graphical database administration.

Additionally, the phpMyAdmin interface was installed to simplify database administration through a web-based graphical interface. The website files and the previously developed project database were then transferred to the server and integrated into the environment. OpenSSH was installed to enable secure remote access and file transfers between the main workstation and the virtual machine.

The main task of this Sprint was about us demonstrating our knowledge on setting up a webserver. Essentially, it was to ensure that the website could successfully connect to the database and run within the configured server environment for later steps of the project.



The screenshot shows the phpMyAdmin web interface. The browser address bar displays the URL: `http://localhost/phpmyadmin/index.php?route=/database/structure&db=portableindoorfeedback`. The interface includes a sidebar with a tree view of databases and tables. The main panel shows the 'Structure' tab for the 'portableindoorfeedback' database. A table with 7 columns is displayed, listing tables and their properties.

Table	Action	Rows	Type
☐ collection	★ Browse Structure Search Insert Empty Drop	2	InnoDB
☐ contains	★ Browse Structure Search Insert Empty Drop	20	InnoDB
☐ hasaccess	★ Browse Structure Search Insert Empty Drop	0	InnoDB
☐ isfriend	★ Browse Structure Search Insert Empty Drop	3	InnoDB
☐ measurement	★ Browse Structure Search Insert Empty Drop	200	InnoDB
☐ station	★ Browse Structure Search Insert Empty Drop	20	InnoDB
☐ user	★ Browse Structure Search Insert Empty Drop	3	InnoDB
7 tables	Sum	248	InnoDB

System Components

The web and database server environment was implemented using the following software components:

Ubuntu Desktop

Ubuntu Desktop was used as the operating system for the virtual server. It provides a stable Linux environment suitable for hosting web services and database applications. The Desktop version includes a graphical user interface, which simplifies system management and allows easier management of tools such as phpMyAdmin and the hosted website through a web browser.

Apache2 – Web Server

Apache2 is responsible for hosting and delivering web content over HTTP. It serves the website files located in the /var/www directory and handles requests coming from web browsers.

MySQL – Database Server

MySQL was installed to manage relational databases used by the website. It stores structured data such as user information and application data that can be accessed by the web application.

PHP – Server-Side Scripting Language

PHP enables dynamic website functionality and allows the web server to interact with the database.

phpMyAdmin – Database Management Interface

phpMyAdmin provides a graphical web interface for managing MySQL databases through a browser. It simplifies database administration tasks such as creating users, importing databases, and managing tables.

OpenSSH Server

OpenSSH was installed to allow secure remote access to the server and enable file transfers between the main workstation and the virtual machine using tools such as WinSCP.



Work Setup

Server Preparation and System Setup

System Updates and Base Configuration

The server environment was first prepared by updating the system and installing the required base components. This ensures that the system is running the latest packages and security updates.

```
sudo apt update  
sudo apt upgrade
```

OpenSSH Installation

The OpenSSH server was installed to allow secure remote access to the server and enable file transfers from the main workstation. After installation, the SSH service was enabled and started using systemctl, and the service status was verified.

```
sudo apt install openssh-server
```

```
sudo systemctl enable ssh  
sudo systemctl start ssh  
sudo systemctl status ssh
```

Web Server Configuration

Apache Installation

The Apache2 web server was installed to host the project website. This is necessary before creating the project website directory, as Apache serves files from `/var/www/html`.

```
sudo apt install apache2
```

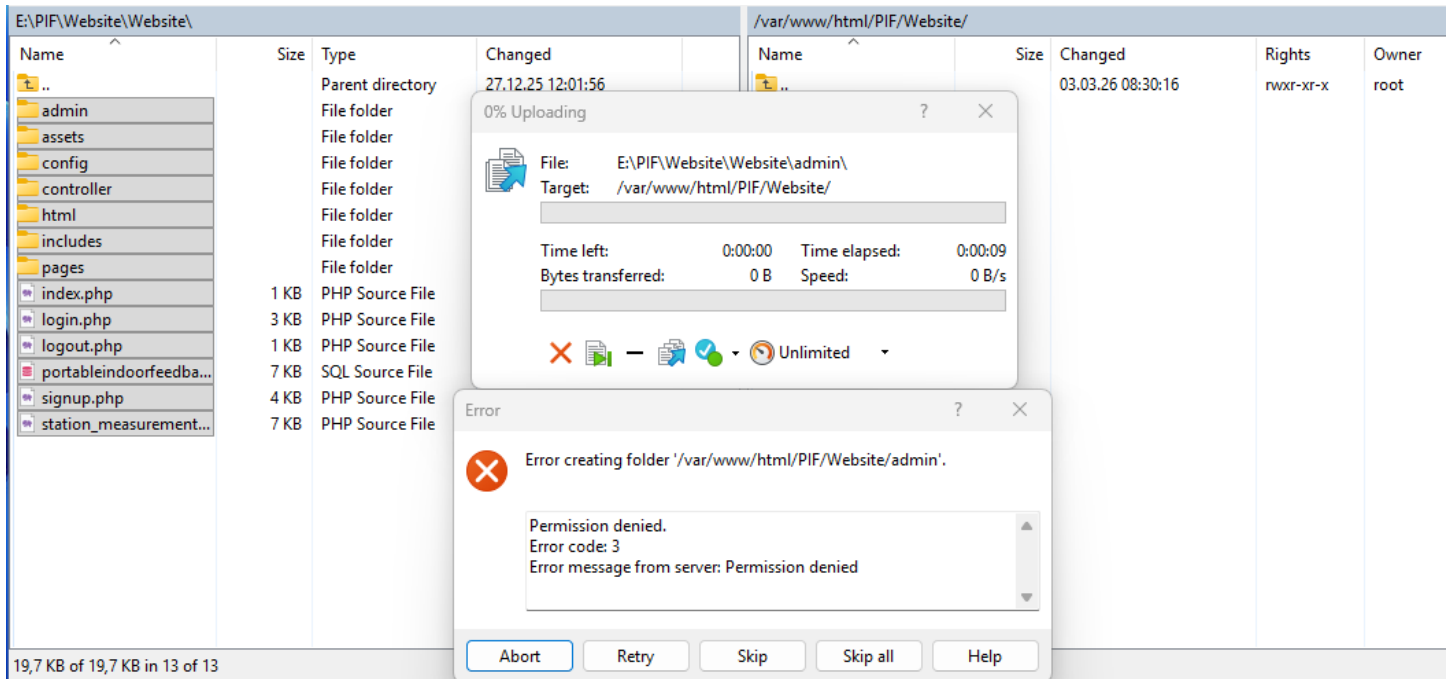
Project Website Directory

After installing Apache, a directory for the project website was then created:

```
sudo mkdir -p /var/www/html/PIF/Website
```

This directory serves as the storage location for the website files that were transferred from the previous sprint.

During the transfer of website files using WinSCP, a **permission denied** error occurred when uploading files to the web directory.



This issue was resolved by adjusting the directory ownership using the `chown` command so that my user account could modify the files during development.

```
sudo chown -R stama:stama /var/www/html/PIF/Website
```

After the website files were successfully transferred and tested, the directory permissions were restored to the default web server user to maintain proper security. The ownership of the project directory was set back to the Apache web server user `www-data`, and the directory permissions were adjusted so that the server could read and execute the files while preventing unauthorized modifications.

```
chown -R www-data:www-data /var/www/html/PIF/
chmod -R 755 /var/www/html/PIF
```

DirectoryIndex Verification

To verify the configuration of Apache and ensure that PHP files are prioritized when loading a website, the `DirectoryIndex` settings were checked using the following command:

```
grep -r "DirectoryIndex" /etc/apache2
```

Virtual Host Configuration

To host the project website with a custom domain name, an Apache **Virtual Host** configuration was created. This allows the web server to handle the website from a specific directory while responding to a defined hostname.

```
sudo cp /etc/apache2/sites-available/000-default.conf
/etc/apache2/sites-available/mywebsite.conf

sudo nano /etc/apache2/sites-available/mywebsite.conf
```

```
<VirtualHost *:80>

    ServerName website.local
    ServerAlias www.website.local
    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html/PIF/Website

    <Directory /var/www/html/PIF/Website>
        AllowOverride All
        Require all granted
    </Directory>

</VirtualHost>
```

The configuration defines the hostname **website.local** used to access the website and sets the document root to `/var/www/html/PIF/Website` directory from which the web server should provide the website files. The virtual host also included directory permissions allowing the web server to access and deliver files from this location.

After editing the configuration file, the new site configuration was enabled, and the Apache service was restarted to apply the changes.

```
sudo a2ensite mywebsite.conf

sudo systemctl restart apache2
```

Local Hostname Configuration

To allow the system to resolve the custom domain name, an entry was added to the `/etc/hosts` file. This file maps hostnames to IP addresses locally on the system.

```
sudo nano /etc/hosts
```

This configuration ensures that when the hostname **website.local** is entered in a browser, it points to the local web server running on the machine. The following line was added:

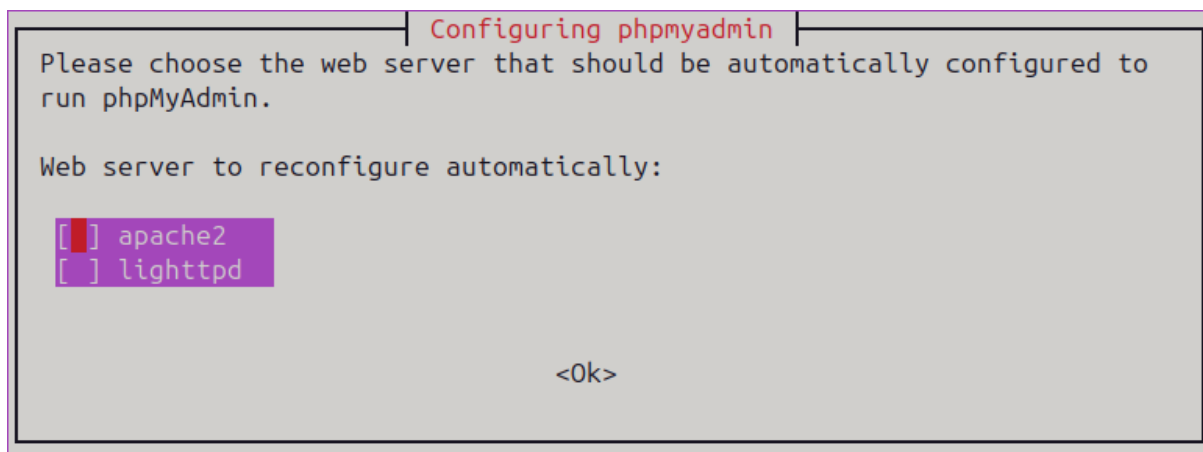
```
0.0.0.0    website.local www.website.local
```

phpMyAdmin Configuration

The phpMyAdmin tool was installed to allow graphical management of the MySQL database through a web browser.

```
sudo apt install phpmyadmin
```

During the installation process, several configuration options were defined. The installer prompted whether the database for phpMyAdmin should be configured automatically using **dbconfig-common**, which was confirmed. For the web server configuration, **Apache2** was selected so that phpMyAdmin could be integrated with the existing web server.



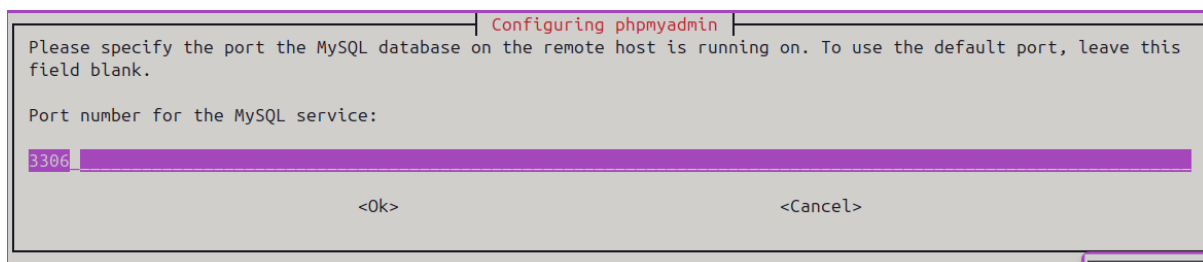
Configuring phpmyadmin

Please choose the web server that should be automatically configured to run phpMyAdmin.

Web server to reconfigure automatically:

☒ apache2
☐ lighttpd

<Ok>



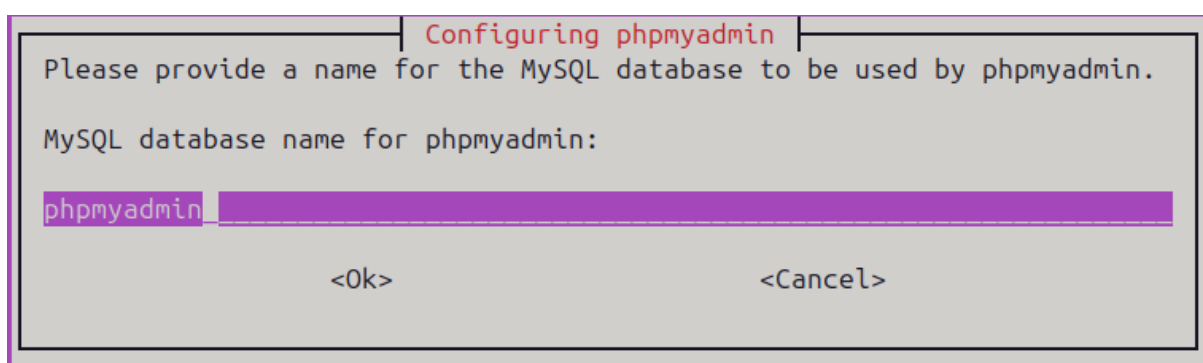
Configuring phpmyadmin

Please specify the port the MySQL database on the remote host is running on. To use the default port, leave this field blank.

Port number for the MySQL service:

3306

<Ok> <Cancel>



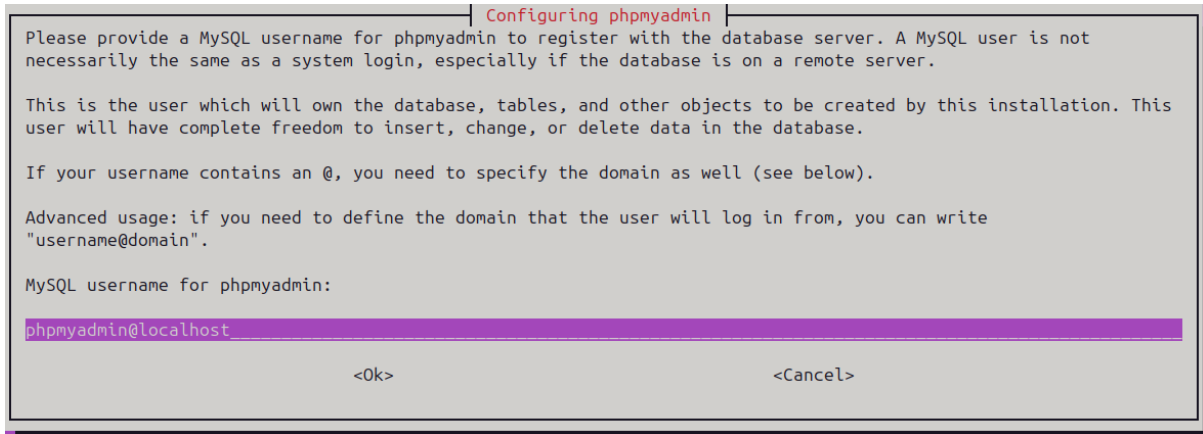
Configuring phpmyadmin

Please provide a name for the MySQL database to be used by phpmyadmin.

MySQL database name for phpmyadmin:

phpmyadmin

<Ok> <Cancel>



For the database connection settings, the **TCP/IP** connection method was used with **localhost** as the database host. The default MySQL port remained unchanged. The default authentication plugin was kept, and the phpMyAdmin database was created with the name **phpmyadmin**. The MySQL username remained unchanged, and an application password was defined during the setup process.

After completing the installation, phpMyAdmin became accessible through the web browser via the Apache web server.

However in my instance, an error occurred when attempting to enable the phpMyAdmin configuration using the **a2enconf phpMyAdmin** command or even when trying to enter the phpMyAdmin webpage. The system reported that the configuration file did not exist (ERROR: Conf phpMyAdmin does not exist). When trying to search "localhost/phpmyadmin" in URL gives a Not Found error message.

To resolve this issue, the required configuration file was manually created in the Apache configuration directory and the appropriate settings were added.

```
sudo nano /etc/apache2/conf-available/phpMyAdmin.conf
```

```
Alias /phpMyAdmin /usr/share/phpMyAdmin

<Directory /usr/share/phpMyAdmin>
    Options FollowSymLinks
    DirectoryIndex index.php
    AllowOverride All
    Require all granted
</Directory>
```

Afterward, the configuration was successfully enabled and Apache was restarted.

```
sudo a2enconf phpMyAdmin
sudo systemctl restart apache2
```

Database Configuration

The MySQL database server was installed to provide database functionality for the website application. MySQL is used to store and manage structured data required by the Portable Indoor Feedback (PIF) website.

```
sudo apt install mysql-server
```

After the installation, the MySQL security configuration script was executed to apply recommended security settings and configure the database environment.

```
sudo mysql_secure_installation
```

The `mysql_secure_installation` script guides the administrator through several security-related configuration steps. During this process, the following options were configured:

- The **password validation component** was configured to enforce password strength rules.
- The **root user authentication** was reviewed to ensure secure administrative access.
- **Anonymous MySQL users were removed**, preventing unauthorized users from accessing the database server.
- **Remote root login was disabled**, restricting administrative access to the local server only.
- The **test database was removed**, as it is not required for production environments and could pose a security risk.
- Finally, the **privilege tables were reloaded** to apply the configuration changes immediately.

These steps help secure the database server by limiting unnecessary access and removing default configurations that are not required for the project environment.

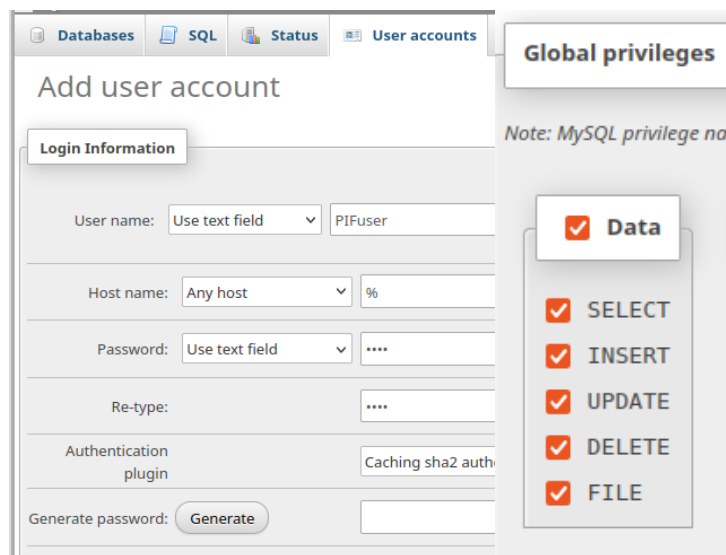
After completing the security configuration, the MySQL terminal was opened to continue with database user configuration and project database setup.

```
sudo mysql
```


For easier testing and administration during development, a new database user named **admin** was created in MySQL. This account was granted full privileges to simplify database management.

```
SET GLOBAL validate_password.policy = LOW;
SET GLOBAL validate_password.length = 4;
CREATE USER 'admin'@'localhost' IDENTIFIED BY '1234';
GRANT ALL PRIVILEGES ON *.* TO 'admin'@'localhost' WITH GRANT OPTION;
FLUSH PRIVILEGES;
EXIT;
```

Additionally, a separate database user named **PIFuser** was created through the phpMyAdmin interface. This user was assigned limited privileges and is intended to be used by the website application to improve security by restricting database access rights. In this case, **PIFuser** Global privileges are only set on in the Data table and nowhere else.



Finally, the website configuration file (config/database.php) was updated to reflect the new database credentials by changing the database username and password from the default root configuration to the newly created **admin** user.

```
portableindoorfeedba database.php portableindoorfeedba database.php x

<?php
// Base URL of the application for root-relative paths in links,
scripts, and CSS
define('BASE_URL', '');

// Database credentials
define('DB_HOST', 'localhost'); // Database server host
define('DB_USER', 'admin'); // Database username
define('DB_PASS', '1234'); // Database password
define('DB_NAME', 'portableindoorfeedback'); // Database name

// Function to create and return a PDO database connection
function getDbConnection() {
    try {
        // Create PDO connection using MySQL driver
        $conn = new PDO(
```

HTTPS/ Self-Signed Certificate Configuration

Although HTTPS was not implemented in this setup, knowledge about configuring secure connections was reviewed.

A self-signed SSL certificate can be created using the OpenSSL tool. This certificate allows encrypted communication between the server and a web browser.

Example command for generating a self-signed certificate:

```
sudo openssl req -x509 -nodes -days 365 -newkey rsa:2048 \  
-keyout /etc/ssl/private/site1.key \  
-out /etc/ssl/certs/site1.crt
```

Key parameters:

- **-x509** – Creates a self-signed certificate
- **-nodes** – Disables password protection for the private key
- **-newkey rsa:2048** – Generates a new 2048-bit RSA key
- **-keyout** – Defines the private key location
- **-out** – Defines the certificate file location

This certificate can later be used to configure HTTPS and redirect HTTP traffic to a secure connection.

Reflection

During this task, a complete web and database server environment was successfully prepared and configured. The installation of Apache, MySQL, and phpMyAdmin provided the necessary infrastructure for hosting the website and managing the project database.

Several configuration challenges occurred during the setup process. For example, permission issues initially prevented file transfers into the web directory, which required adjusting directory ownership. Another issue occurred during the phpMyAdmin configuration when the Apache configuration file was missing, which required manual creation and activation. And also, allowing the webserver to properly connect to the website work so that I could be able to access it.

Solving these issues improved my understanding of server configuration, Linux file permissions, and the interaction between the web server and the database system. The completed setup now allows the website to run correctly while providing convenient database management through phpMyAdmin.

REPIFI1: SPRINT L5 Report Data Transfer



Data Transfer and Server Communication Implementation

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 9 March – 22 March 2026

Contents

Sprint 5 of the Project	61
Goal	61
Objective	61
System Overview	61
Network Configuration	61
Web Server – Data Receiving Implementation	62
Test Interface (send.php)	62
Data Processing (receive.php)	62
Raspberry Pi – Data Sending Configuration	63
Configuration File (config.toml)	63
Data Collection (measure.py)	64
Troubleshooting and Adjustments	65
Result	66
Reflection	66

Sprint 5 of the Project

Goal

The goal of this sprint was to enable communication between the Raspberry Pi and the web server so that measured sensor data can be transferred, stored in the database, and later displayed on the website. This step connects the hardware and software components of the project, enabling the system to function as a complete data acquisition solution.

Objective

The objective was to implement and test a data transfer using HTTP POST requests. This included creating PHP scripts on the web server to receive and process incoming data, as well as configuring the Raspberry Pi to send measurement data automatically.

Additionally, both systems had to be configured within the same network environment to ensure reliable communication and correct data transmission.

System Overview

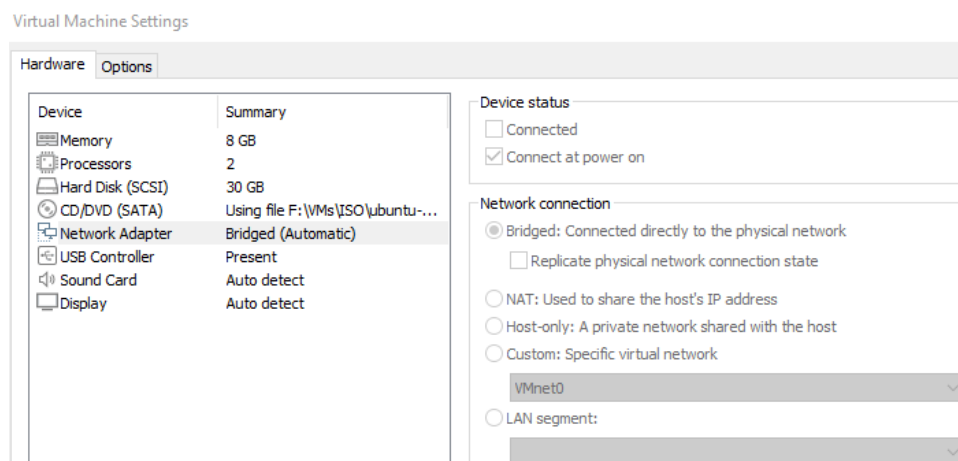
The system consists of two main parts:

- **Raspberry Pi (Data Sender)**
Collects sensor data and sends it to the server.
- **Web Server (Data Receiver)**
Receives incoming data, processes it, and stores it in the database.

The communication between both systems is performed using **HTTP POST requests**, allowing structured data transfer over the network.

Network Configuration

To allow communication between the Raspberry Pi and the web server, both systems needed to be in the same network. Initially, the virtual machine running the web server was configured using **NAT**, which created an isolated network. This prevented direct communication between the Raspberry Pi and the server. To resolve this issue, the network adapter was changed to **Bridged Mode**, allowing the virtual machine to obtain an IP address from the same network as the Raspberry Pi. This configuration enabled both systems to communicate directly using their respective IP addresses.



Web Server – Data Receiving Implementation

To test the receiving functionality of the web server, two PHP scripts were created in the `/PIF/Website/api` directory. These scripts were used to simulate data transmission and verify that the server can correctly receive, process, and store incoming measurement data before integrating the Raspberry Pi:

- `send.php` – test interface
- `receive.php` – data processing script

Test Interface (`send.php`)

The **`send.php`** file contains a simple HTML form used to simulate sensor data and send it to the server.

```
<form method="POST" action="receive.php">
```

Example input field:

```
<input type="number" step="0.01" name="temperature">
```

This interface was used to verify that the server correctly receives POST requests before integrating the Raspberry Pi.

Data Processing (`receive.php`)

The **`receive.php`** script receives, validates, and stores incoming data.

Receiving POST Data

```
$station_serial = trim($_POST['station_serial'] ?? '');  
$temperature = trim($_POST['temperature'] ?? '');
```

Station Validation

```
$stmt = $conn->prepare("SELECT pk_serialNumber FROM station WHERE pk_serialNumber = ?");  
$stmt->execute([$station_serial]);
```

This ensures that only valid stations can insert data into the database. Cannot be injected itself into the database because only admin can add new stations and normal user can only register them. Therefore, admin will have to manually add stations in the database for them to exist and to be registered later.

Database Insertion

```
INSERT INTO measurement  
(temperature, humidity, pressure, light, gas, timestamp, fk_station_records)  
VALUES (?, ?, ?, ?, ?, ?, ?)
```

Testing Output

```
echo "Temperature: $temperature <br>";
```

This output confirms that the data was successfully received by showing it on the receive.php.

Raspberry Pi – Data Sending Configuration

The data transmission from the Raspberry Pi is handled inside the `/opt/station/` directory using two main files:

`config.toml` – configuration file

`measure.py` – main data collection and transmission script

Configuration File (**config.toml**)

General Settings

```

/opt/station/config.toml - Mousepad
File Edit Search View Document Help
station_serial = "STAMA314"          # Serial number of the station
interval = 2                         # Time between measurements
toggle_intensity = 100               # The light level at which the LED should be turned on
[i2c]                                # I2C-Addresses of the sensors
bme280 = 0x76
bh1750 = 0x23
mq135 = 0x48
[destination]                        # Information about the server
ip = "10.12.2.39"                    # IP-Address or Hostname of the Server
path = "api/receive.php"             # Path to the script that accepts the measurements

```

The configuration file defines the key parameters required for the operation of the measurement system. The **station_serial** serves as a unique identifier for the device, ensuring that all transmitted data can be correctly assigned in the database. The **interval** determines how frequently measurements are taken and sent to the server.

The **toggle_intensity** value is used to control the LED on the Raspberry Pi, which is activated when the measured light intensity falls below the defined threshold.

In the **[i2c]** section, the addresses of the connected sensors are specified. These addresses are required for communication between the Raspberry Pi and the sensors via the I²C interface.

Finally, the **[destination]** section defines the target server for data transmission. The **ip** specifies the address of the web server, while the **path** determines the exact location of the **receive.php** script that processes incoming measurement data.

Data Collection (**measure.py**)

The `measure.py` script is responsible for collecting sensor data, processing it, and transmitting it to the web server. It runs continuously and performs all core operations of the measurement system.

Configuration Loading

At the beginning of the script, the configuration file is loaded. This allows the script to access parameters such as sensor addresses, measurement interval, and server destination dynamically.

```
with open("/opt/station/config.toml", "rb") as f:
    cfg = toml.load(f)
```

This approach ensures that configuration changes can be made without modifying the main program code.

Sensor Readings

The script reads measurement values from all connected sensors using their respective libraries.

```
light = bh1750.read_light()
temperature = bme280.read_temperature()
pressure = bme280.read_pressure()
humidity = bme280.read_humidity()
gas = mq135.read_CO2()
```

These values represent the environmental conditions measured by the station, including light intensity, temperature, air pressure, humidity, and air quality.

Data Transmission

After collecting the sensor data, the script sends it to the web server using an HTTP POST request.

```
requests.post(f"http://{host}{path}", data, timeout=1)
```

In this request:

- **host** refers to the IP address or hostname of the web server, which is defined in the **config.toml** file
- **path** specifies the location of the server-side script (**receive.php**) that is responsible for handling incoming data
- **data** contains the actual measurement values collected from the sensors

The full request URL is dynamically constructed using the configuration values:

```
http://<server-ip>/<path-to-receive.php>
```

This makes the system flexible, as the destination server and API endpoint can be changed without modifying the script itself.

The **config.toml** file plays a critical role here, as it provides the connection details used by the script. Without correct configuration of the **ip** and **path**, the data cannot be transmitted to the server.

On the receiving side, the **receive.php** script processes the incoming POST request. It extracts the transmitted values, validates them, and inserts them into the database. This establishes the connection between the Raspberry Pi (data sender) and the web server (data receiver).

Data structure

Before sending, the measurement values are organized into a structured format.

```
data = {  
    'station_serial': station_serial,  
    'timestamp': time,  
    'temperature': temperature,  
    'humidity': humidity,  
    'pressure': pressure,  
    'light': light,  
    'gas': gas  
}
```

This structure ensures that all relevant data is transmitted in a consistent and readable format, allowing the server to process and store it correctly.

Continuous Execution

The script runs in an infinite loop, continuously collecting and sending data at defined intervals.

```
while True:  
    ...  
    time.sleep(cfg['interval'])
```

The delay between measurements is controlled by the interval value defined in the configuration file. This ensures regular data updates without overloading the system.

Troubleshooting and Adjustments

Normally at this point the data transfer should work but in my case during implementation, several issues were encountered and resolved:

- **Incorrect data transfer path**
The POST request did not initially reach the correct server location. This issue was resolved by verifying and correcting the path defined in the **config.toml** file.
- **Script execution permissions**
The **measure.py** script did not execute properly during system startup due to missing permissions. When initially trying to search for the main reason as to why the Raspberry Pi couldn't send the data in the first place. I checked it by executing a command into the terminal: `"sudo python3 measure.py"` to see if it shows any output. This was resolved by modifying the **autostart** script and adding **sudo** to ensure proper execution.

Example adjustment:

```
sudo python3 measure.py &
```

This ensured that the script runs correctly and continuously sends measurement data.

Result

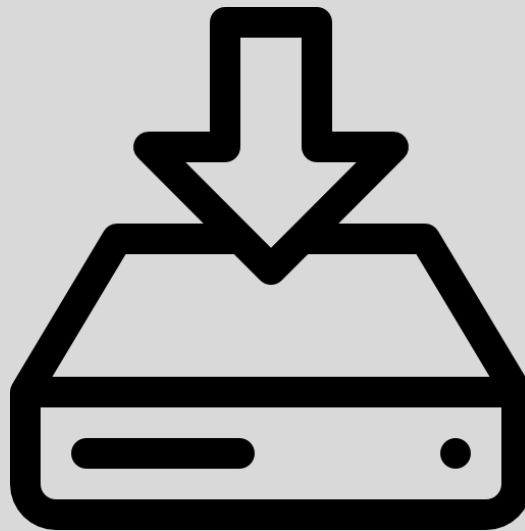
The system successfully established communication between the Raspberry Pi and the web server. Measurement data could be sent via HTTP POST requests, received by the PHP script, and stored correctly in the database.

Reflection

During this sprint, the main challenge was establishing reliable communication between two separate systems. Network configuration played its role, as the initial NAT setup prevented direct communication.

Additionally, debugging issues related to file paths and script permissions required careful verification of configuration files and system behaviour. Since it wasn't just the configurations on the main Raspberry Pi device but also the code that was implemented for the database to run and for the website to take the data from. Many checks and inspections of data path had to be done in order to ensure that the data arrived to the database correctly.

REPIFI1: SPRINT L6 Report Backup Server



Backup Server Configuration

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 23 March – 19 April 2026

Contents

Sprint 6 of the Project	69
Goal	69
Objective	69
Components and Tools	69
System Configuration	70
Database Server Preparation	70
Backup Server Setup	71
Backup Directory Setup	73
Backup Script.....	73
Automation with Cron	75
Secure MySQL Authentication with MySQL Config Editor	76
FTP Configuration	76
Reflection	78

Sprint 6 of the Project

Goal

The goal of this sprint was to set up a backup server capable of storing regular backups of the project database. This ensures that data can be restored in case of data loss or system failure.

Objective

The objective was to configure a separate Ubuntu Server system that connects to the main database server, creates automated daily backups using a shell script, and provides access to these backups via FTP.

Components and Tools

Ubuntu Server (Backup Server VM)

A lightweight Linux operating system used to host the backup system. It runs the backup scripts, stores the database backups, and manages services such as SSH and FTP.

MySQL / phpMyAdmin (Database Server)

The database server stores all project data. MySQL handles the database operations, while phpMyAdmin provides a graphical interface for managing users, databases, and permissions.

OpenSSH (Remote Access)

Used to securely access and manage the backup server remotely. It also allows file transfers and system administration from another machine.

MySQL Client

A command-line tool installed on the backup server that allows it to connect to the remote database server and execute commands such as database queries and backups.

mysqldump Utility

A tool used to export the database into a .sql file. It creates a complete backup of the database structure and data, which can later be restored if needed.

Cron (Task Scheduling)

A scheduling service used to automate tasks. In this project, it is used to run the backup script automatically every day at 08:00.

vsftpd (FTP Server)

A file transfer service that allows access to the backup files from another machine. It was configured with restricted access so the user can only access the backup directory (FTP jail).

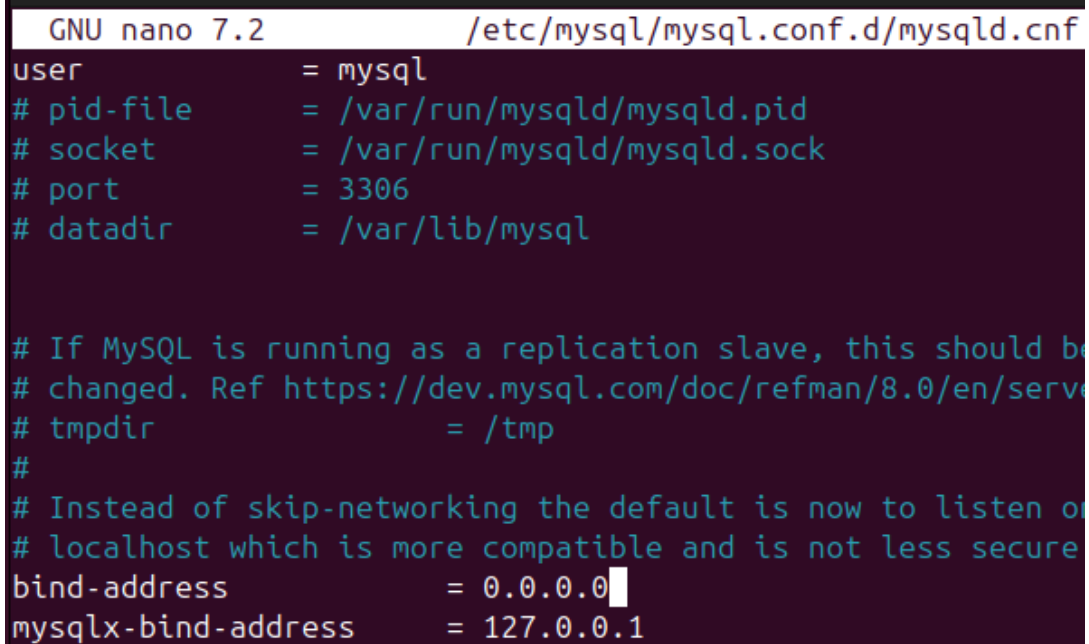
System Configuration

Database Server Preparation

To enable communication between the database server and the backup server, the network configuration of the virtual machine was adjusted. The network adapter was changed from **NAT** to **Bridged** mode, allowing both machines to operate within the same network. This is necessary because NAT isolates the virtual machine, preventing external systems from directly accessing the database service.

To allow external connections to the PHMyAdmin database, the configuration file was modified:

```
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
```



```
GNU nano 7.2 /etc/mysql/mysql.conf.d/mysqld.cnf
user = mysql
# pid-file = /var/run/mysqld/mysqld.pid
# socket = /var/run/mysqld/mysqld.sock
# port = 3306
# datadir = /var/lib/mysql

# If MySQL is running as a replication slave, this should be
# changed. Ref https://dev.mysql.com/doc/refman/8.0/en/serve
# tmpdir = /tmp
#
# Instead of skip-networking the default is now to listen on
# localhost which is more compatible and is not less secure
bind-address = 0.0.0.0
mysqlx-bind-address = 127.0.0.1
```

- bind-address was set to 0.0.0.0 to allow external access
- mysqlx-bind-address remained 127.0.0.1

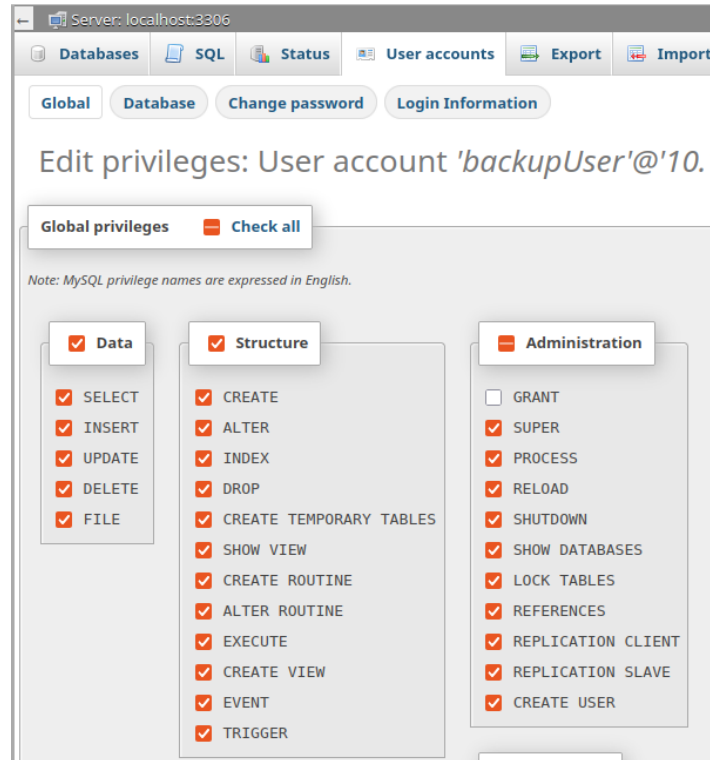
This change allows MySQL to accept connections from any IP address instead of only the local machine. Without this modification, the backup server would not be able to connect to the database.

The parameter mysqlx-bind-address was left as 127.0.0.1, since it is not required for this setup.

A dedicated database user was then created using phpMyAdmin:

- Username: **backupUser**
- Privileges: Full access to the project database

This user is used by the backup server to access the database. Creating a separate user ensures better control over permissions and avoids using the root account for automated processes.



Backup Server Setup

On the backup server, several system configurations were required to prepare it for communication and backup operations.

First, the OpenSSH server was installed and enabled. This allows remote access to the server and makes it possible to manage the system or transfer files from another machine.

```
backupadmin@stama314-backup:~$ sudo systemctl enable ssh
Synchronizing state of ssh.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable ssh
Created symlink /etc/systemd/system/ssh.service → /usr/lib/systemd/system/ssh.service.
Created symlink /etc/systemd/system/multi-user.target.wants/ssh.service → /usr/lib/systemd/system/ssh.service.
backupadmin@stama314-backup:~$ sudo systemctl start ssh
backupadmin@stama314-backup:~$ systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/usr/lib/systemd/system/ssh.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-04-07 20:54:35 CEST; 3s ago
 TriggeredBy: ● ssh.socket
   Docs: man:sshd(8)
        man:sshd_config(5)
   Process: 2024 ExecStartPre=/usr/sbin/sshd -t (code=exited, status=0/SUCCESS)
   Main PID: 2025 (sshd)
    Tasks: 1 (limit: 9375)
   Memory: 2.1M (peak: 2.4M)
      CPU: 44ms
   CGroup: /system.slice/ssh.service
           └─2025 "sshd: /usr/sbin/sshd -D [listener] 0 of 10-100 startups"

Apr 07 20:54:35 stama314-backup systemd[1]: Starting ssh.service - OpenBSD Secure Shell server...
Apr 07 20:54:35 stama314-backup sshd[2025]: Server listening on 0.0.0.0 port 22.
Apr 07 20:54:35 stama314-backup sshd[2025]: Server listening on :: port 22.
Apr 07 20:54:35 stama314-backup systemd[1]: Started ssh.service - OpenBSD Secure Shell server.
```

The hostname was configured in the `/etc/hosts` file to define a local system identity:

```
sudo nano /etc/hosts
```

This helps with system identification and can simplify network communication in certain configurations.

```
127.0.0.1 localhost
127.0.1.1 STAMA314-backup

# The following lines are desirable for IPv6 capable hosts
::1      ip6-localhost ip6-loopback
fe00::0  ip6-localnet
ff00::0  ip6-mcastprefix
ff02::1  ip6-allnodes
ff02::2  ip6-allrouters
```

Next, a static IP address was configured using Netplan:

```
sudo nano /etc/netplan/00-installer-config.yaml
```

A static IP is required so that the database server can reliably identify and communicate with the backup server. If a dynamic IP were used, the address could change, breaking the connection.

```
GNU nano 7.2 /etc/netplan/00-installer-config.yaml
network:
  ethernets:
    ens33:
      dhcp4: false
      addresses:
        - 10.12.2.28/24
        #- 192.168.5.45/23
  version: 2
```

After editing the configuration, the file permissions were restricted and configuration was then applied. This ensures that only the system administrator can modify the network configuration, improving system security.

```
backupadmin@stama314-backup:~$ sudo chmod 600 /etc/netplan/00-installer-config.yaml
backupadmin@stama314-backup:~$ sudo netplan apply
```

The MySQL client was installed to allow communication with the database server. This tool is necessary because it allows the backup server to connect to the remote MySQL database and execute commands such as **mysqldump**.

```
sudo apt install mysql-client
```

To verify connectivity, a connection test was performed:

```
backupadmin@STAMA314-backup:~$ mysql -h 10.12.2.39 -u backupUser -p
```

This command attempts to connect to the database server using the created backup user. A successful connection confirms that network configuration, user permissions, and MySQL settings are correctly configured.

Backup Directory Setup

Two directories were created to organize and store backup data. Along with permissions set:

```
backupadmin@STAMA314-backup:~$ sudo mkdir -p /var/backups/portableindoorfeedback_db
backupadmin@STAMA314-backup:~$ sudo chmod 755 /var/backups/portableindoorfeedback_db
backupadmin@STAMA314-backup:~$ sudo mkdir -p /var/backups/full_system
backupadmin@STAMA314-backup:~$ sudo chmod 755 /var/backups/full_system
```

The directory `/var/backups/portableindoorfeedback_db` is used specifically to store the database backup files created by the backup script. These files contain exported `.sql` data from the project database.

The directory `/var/backups/full_system` was prepared for potential full system backups. This allows future expansion of the backup strategy to include not only the database but also system files and configurations.

The permission setting 755 allows the owner full access (read, write, execute), while other users can read and access the directories. This ensures that services such as FTP can access the backup files, while still maintaining a basic level of security.

Backup Script

A shell script was created to automate the process of backing up the project database. The script is responsible for connecting to the database server, exporting the database, storing it as a file, and managing old backups.

The script was created and made executable:

```
backupadmin@STAMA314-backup:~$ sudo nano /usr/local/bin/portableindoorfeedback_db_backup.sh
backupadmin@STAMA314-backup:~$ sudo chmod +x /usr/local/bin/portableindoorfeedback_db_backup.sh
```

Full script:

```
GNU nano 7.2 /usr/local/bin/portableindoorfeedback_db_backup.sh
#!/bin/bash
# Backup of Database Script
# backs up the project database from the web/database server

# variables
BACKUP_DIR="/var/backups/portableindoorfeedback_db"
DATE=$(date +"%Y%m%d_%H%M%S")
BACKUP_FILE="$BACKUP_DIR/portableindoorfeedback_db_$DATE.sql"
LOG_FILE="/var/log/backup.log"
DB_HOST="10.12.2.39"
#DB_HOST="192.168.5.44"
#DB_USER="backupUser"
#DB_PASSWORD="1@Fuksik2!"
#DB_NAME="portableindoorfeedback"

# create backup directory if it doesnt exists
mkdir -p $BACKUP_DIR

# execute backup
mysqldump --login-path=databaseServer -B --single-transaction $DB_NAME > $BACKUP_FILE 2>> $LOG_FILE

# check if backup was made successfully
if [ $? -eq 0 ]; then
    echo "$(date): Backup successful - $BACKUP_FILE" >> $LOG_FILE

    # delete backup after 7 days
    find $BACKUP_DIR -name "portableindoorfeedback_db_*.sql" -type f -mtime +7 -delete

    echo "$(date): Cleaned up backups older than 7 days" >> $LOG_FILE
else
    echo "$(date): ERROR - Backup failed!" >> $LOG_FILE
fi
echo "$(date): Backup process completed" >> $LOG_FILE
```

Variables

The script starts by defining variables:

- BACKUP_DIR** - location where backup files are stored
- DATE** - current timestamp used to create unique filenames
- BACKUP_FILE** - full path of the backup file
- LOG_FILE** - file used to store logs

Database connection details:

- DB_HOST** - IP address of the database server
- DB_USER** - database user created for backups
- DB_PASSWORD** - password for authentication
- DB_NAME** - name of the database to back up

Directory Creation

```
mkdir -p $BACKUP_DIR
```

Creates the backup directory if it does not already exist. The **-p** option ensures that no error is shown if the folder already exists.

Database Backup

```
mysqldump --login-path=databaseServer -B --single-transaction $DB_NAME >  
$BACKUP_FILE 2>> $LOG_FILE
```

This command:

- connects to the remote database server using MySQL login profile
- creates an export of the project database as a .sql file
- stores the backup in the defined backup directory
- writes any error messages into the log file

Backup Verification

```
if [ $? -eq 0 ]; then
```

Checks if the backup command was successful:

- if it is 0 then it means success
- any other value means error

Automatic Cleanup

```
find $BACKUP_DIR -name "portableindoorfeedback_db_*.sql" -type f -mtime +7 -delete
```

Deletes backup files older than 7 days to:

- prevent storage overflow
- keep only recent backups

Logging

```
echo "$(date): Backup successful - $BACKUP_FILE" >> "$LOG_FILE"
```

This allows monitoring of:

- successful backups
- errors
- cleanup actions

Automation with Cron

The backup script was scheduled using cron:

```
backupadmin@STAMA314-backup:~$ sudo crontab -e_
```

Entry added:

```
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 8 * * * /usr/local/bin/portableindoorfeedback_db_backup.sh
```

This cron entry means:

- **0** → Execute at minute **0**
- **8** → Execute at hour **8** (08:00 AM)
- ***** → Every day of the month
- ***** → Every month
- ***** → Every day of the week
- And the path to the shell script

This makes it so that a backup is executed automatically every day at 08:00.

Secure MySQL Authentication with MySQL Config Editor

To improve security, database credentials were not stored directly inside the shell script. Instead, the **MySQL Config Editor** was used to securely store the login information in an encrypted MySQL login file.

```
backupadmin@STAMA314-backup:~$ mysql_config_editor set --login-path=databaseServer --host=10.12.2.39 --user=backupUser --password
Enter password:
```

This command was used to create the login profile which will store the database server IP address, the backup user profile name to access the database and database password to enter. Providing advantages for sensitive data such as usernames and passwords are not visible inside the backup script and unauthorized users cannot easily read database credentials from plain text files.

To verify the stored login profile, display the saved login configuration I used:

```
backupadmin@STAMA314-backup:~$ mysql_config_editor print --all
[databaseServer]
user = "backupUser"
password = *****
host = "10.12.2.39"
```

FTP Configuration

To allow access to the created backup files from another machine, an FTP server was configured on the backup server. This enables transferring backup files over the network in a simple and controlled way.

The FTP server was installed:

```
backupadmin@STAMA314-backup:~$ sudo apt install vsftpd -y
```

An FTP user was created and restricted to the backup directory (FTP jail):

User: ftpuser

Home directory: /var/backups/portableindoorfeedback_db

```
backupadmin@STAMA314-backup:~$ sudo useradd -m -d /var/backups/portableindoorfeedback_db ftpuser
backupadmin@STAMA314-backup:~$ sudo passwd ftpuser
New password: ftp123
```

The user was assigned ownership of the backup directory:

```
backupadmin@STAMA314-backup:~$ sudo chown ftpuser:ftpuser /var/backups/portableindoorfeedback_db
backupadmin@STAMA314-backup:~$ sudo chmod 755 /var/backups/portableindoorfeedback_db
```

Basic configuration was done in vsftpd config file to jail FTP user inside the directory:

```
backupadmin@STAMA314-backup:~$ sudo nano /etc/vsftpd.conf_
```

```
# Restrict FTP user to backup directory
user_sub_token=$USER
local_root=/var/backups/portableindoorfeedback_db

# Passive mode (firewall compatibility)
pasv_enable=YES
pasv_min_port=30000
pasv_max_port=31000

# You may restrict local users to their home directories. See the FAQ for
# the possible risks in this before using chroot_local_user or
# chroot_list_enable below.
chroot_local_user=YES
chroot_list_enable=NO
allow_writeable_chroot=YES

# Activate logging of uploads/downloads.
xferlog_enable=YES

# You may override where the log file goes if you like. The default is shown
# below.
xferlog_file=/var/log/vsftpd.log

# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
#
# Uncomment this to allow local users to log in.
local_enable=YES
#
# Uncomment this to enable any form of FTP write command.
write_enable=YES
#
# Default umask for local users is 077. You may wish to change this to 022,
# if your users expect that (022 is used by most other ftpd's)
local_umask=022
```

After configuration, the FTP service was started and enabled:

```
backupadmin@STAMA314-backup:~$ sudo systemctl restart vsftpd
backupadmin@STAMA314-backup:~$ sudo systemctl enable vsftpd
```

This ensures the FTP server runs automatically and is available after system reboot.

To verify the setup, a connection was tested from another device (for example using the database server) using:

```
stama@stama-VMware-Virtual-Platform:~$ ftp 10.12.2.28
Connected to 10.12.2.28.
220 (vsFTPD 3.0.5)
Name (10.12.2.28:stama): ftpuser
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> ls
229 Entering Extended Passive Mode (|||30347|)
150 Here comes the directory listing.
-rw-r--r-- 1 0 0 405020 Apr 27 14:59 portableindoorfeedback_db_20260427_145957.sql.gz
-rw-r--r-- 1 0 0 405020 Apr 27 15:00 portableindoorfeedback_db_20260427_150015.sql.gz
-rw-rw-r-- 1 1000 1000 0 Apr 27 15:00 portableindoorfeedback_db_20260427_150042.sql.gz
-rw-r--r-- 1 0 0 405020 Apr 27 15:19 portableindoorfeedback_db_20260427_151902.sql
-rw-r--r-- 1 0 0 405650 Apr 27 15:29 portableindoorfeedback_db_20260427_152915.sql
-rw-r--r-- 1 0 0 203 Apr 27 15:49 portableindoorfeedback_db_20260427_154933.sql
226 Directory send OK.
```

After logging in with the FTP user credentials, access to the backup directory was confirmed. And I could confirm that the ftp user was inside the backup directory and couldn't get out from it.

If we want to extract those SQL files to restore our database in case of database being erased, stolen or corrupted. We use the **get** command to download a file from the backup server to our database server. Which then we can Export using the phpMyAdmin GUI to restore the contents of the database.

```
ftp> get portableindoorfeedback_db_20260427_154933.sql
local: portableindoorfeedback_db_20260427_154933.sql remote: portableindoorfeedback_db_20260427_154933.sql
229 Entering Extended Passive Mode (|||30693|)
150 Opening BINARY mode data connection for portableindoorfeedback_db_20260427_154933.sql (203 bytes).
100% |*****| 203 167.15 KiB/s 00:00 ETA
226 Transfer complete.
203 bytes received in 00:00 (60.42 KiB/s)
```

Reflection

During this sprint, the setup of the backup server was partially successful. The system configuration, directory setup, and automation using cron were completed. But essentially later, I was able to finally make the backup function work even in both ways (saving and restoring).

REPIF1-2: Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: *27 September – 07 June 2026*

(WORKSHOP PART)

REPIF1: SPRINT A1 Report

PVC Holder

PVC Holder Fabrication and Assembly

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 10 November – 5 December 2025

Contents

Sprint 1 of the Project (Workshop work).....	82
Goal	82
Objective	82
Components/Tools	83
<i>PVC Components</i>	83
<i>Tools Used</i>	83
Schematics and Measurements	85
Work Steps	87
Results	89
<i>Final Result</i>	89

Sprint 1 of the Project (Workshop work)

Goal

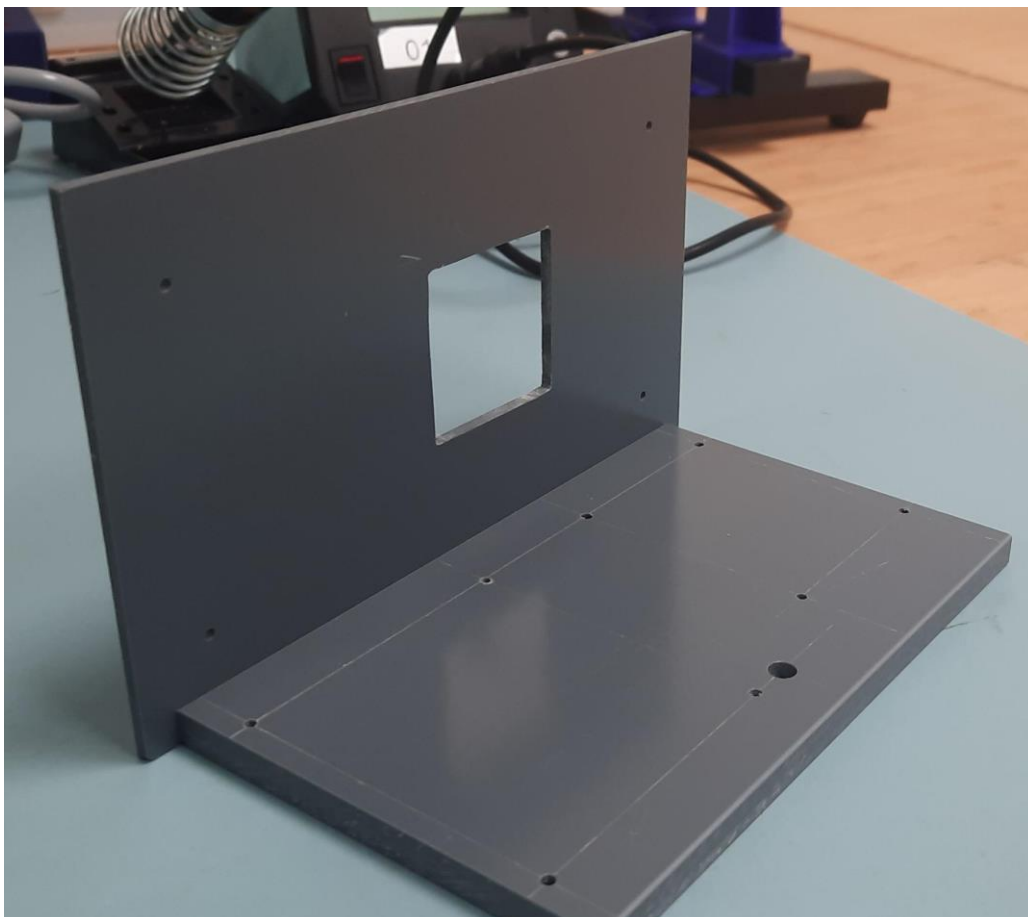
The goal of this Sprint in the workshop was to prepare the PVC holder that serves as the basic internal support structure for the touch display, Raspberry Pi, and circuit board.

Two plastic PVC plates were processed and prepared so they could later be mounted together to form the core internal frame of the measuring station.

Objective

The objective of this Sprint was to mark, drill, and prepare both PVC plates according to the required measurements.

This included preparing all mounting holes, creating a square opening, forming countersinks for the screw heads, and tapping threads into the designated holes. By completing these mechanical preparations, the plates became ready for the later assembly of the display unit, Raspberry Pi, and the internal electronics.



Components/Tools

PVC Components

The PVC holder consists of two separate plates that form the internal mounting structure for the display and electronic components. Both plates were prepared according to the official technical drawings (see *Schematics section*), including hole placement, dimensions, countersinks, and threaded holes:

- **Front Plate (Display_Halterung Frontplatte)**
Used to mount the touch display and to provide the square cutout for the micro-controller components.
- **Bottom Plate (Display_Halterung Bodenplatte)**
Used to hold the Raspberry Pi, the circuit board, and threaded standoffs for mounting.

Tools Used

The following tools were used to accurately mark, drill, countersink, and thread the PVC plates:

Standard Height Gauge

Used to scribe straight, parallel reference lines on the PVC plates, ensuring that hole positions and the display cutout were aligned correctly.



Center Punch

Used with a hammer, to mark drilling points so that the drill bit enters precisely without slipping.



Hand Files and Needle Files

Hand files were used to begin shaping the rectangular display cutout after creating the initial opening. While needle files were used for fine shaping and adjusting the display opening to match the drawn measurement lines.



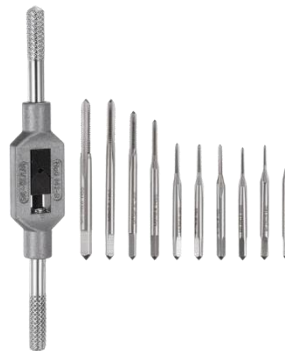
Countersink Drill Bit

Used to create conical recesses for countersunk screw heads on the specified holes.



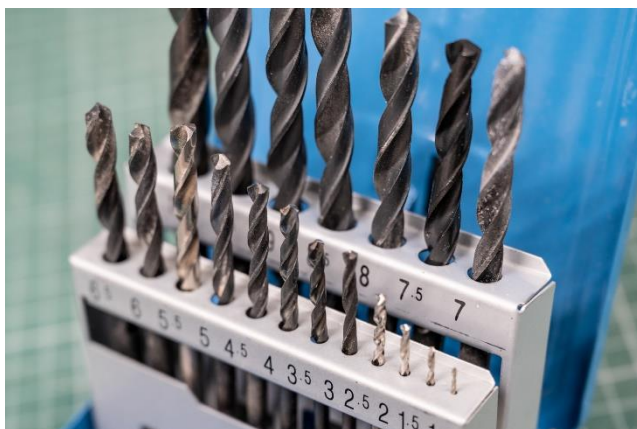
Thread Tap (M2.5) with T-Handle Tap Wrench

Used to cut internal threads into the mounting holes along the edge of the plate. Because the material is plastic, only a single finishing tap was required.



Drill Press (Bench Drill)

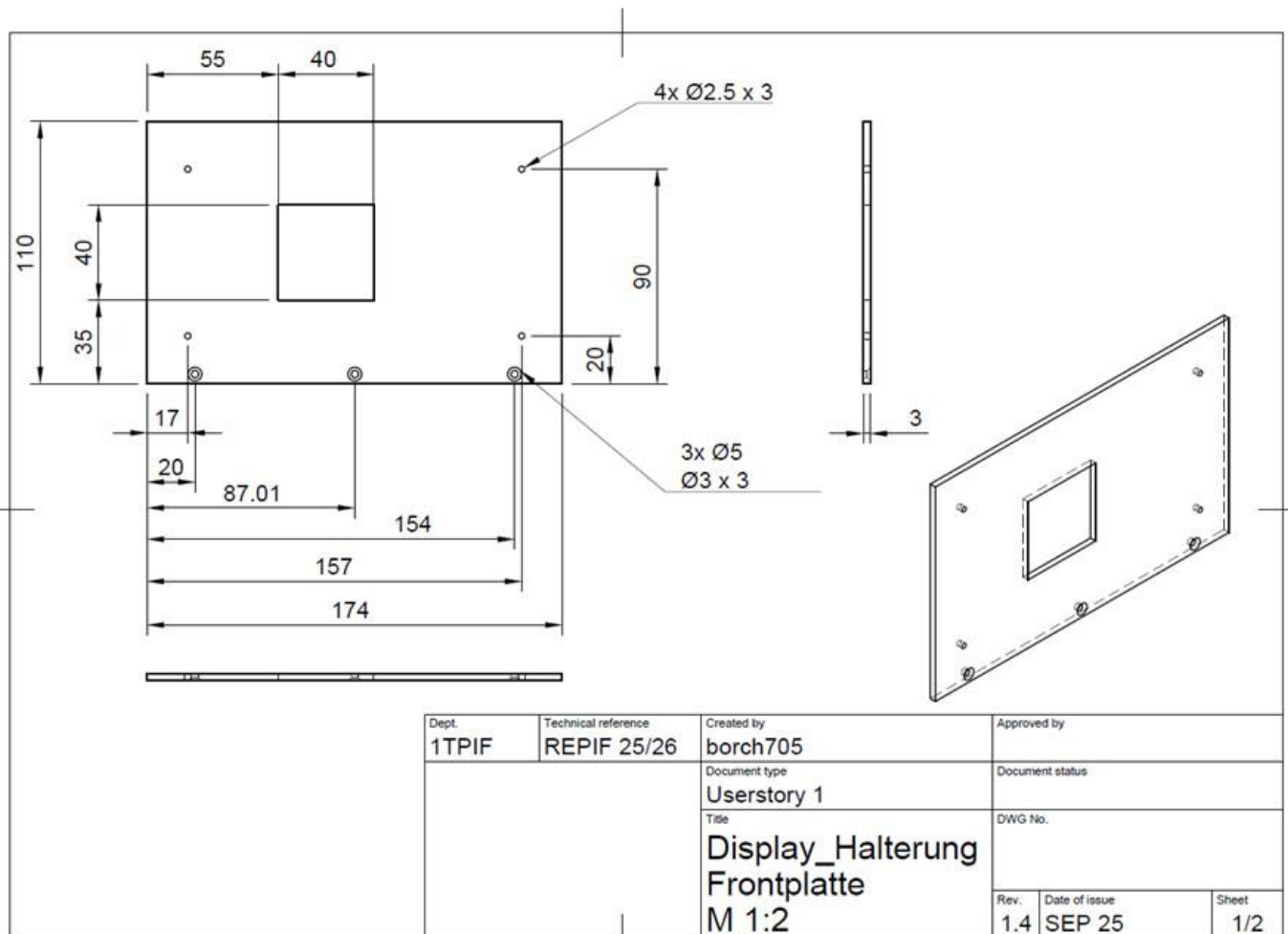
Used to drill all required holes with perpendicular accuracy, including Ø2.5 mm, Ø3 mm, Ø4 mm, and Ø5 mm holes.



Schematics and Measurements

The work process for both PVC plates was carried out according to the official technical drawings shown below. These schematics contain all relevant measurements required to accurately mark, drill, countersink and prepare the plate. The dimensions include hole diameters, hole spacing, plate thickness, and the exact outline of the opening. By following these drawings, each feature on the PVC plates could be transferred precisely before cutting or drilling.

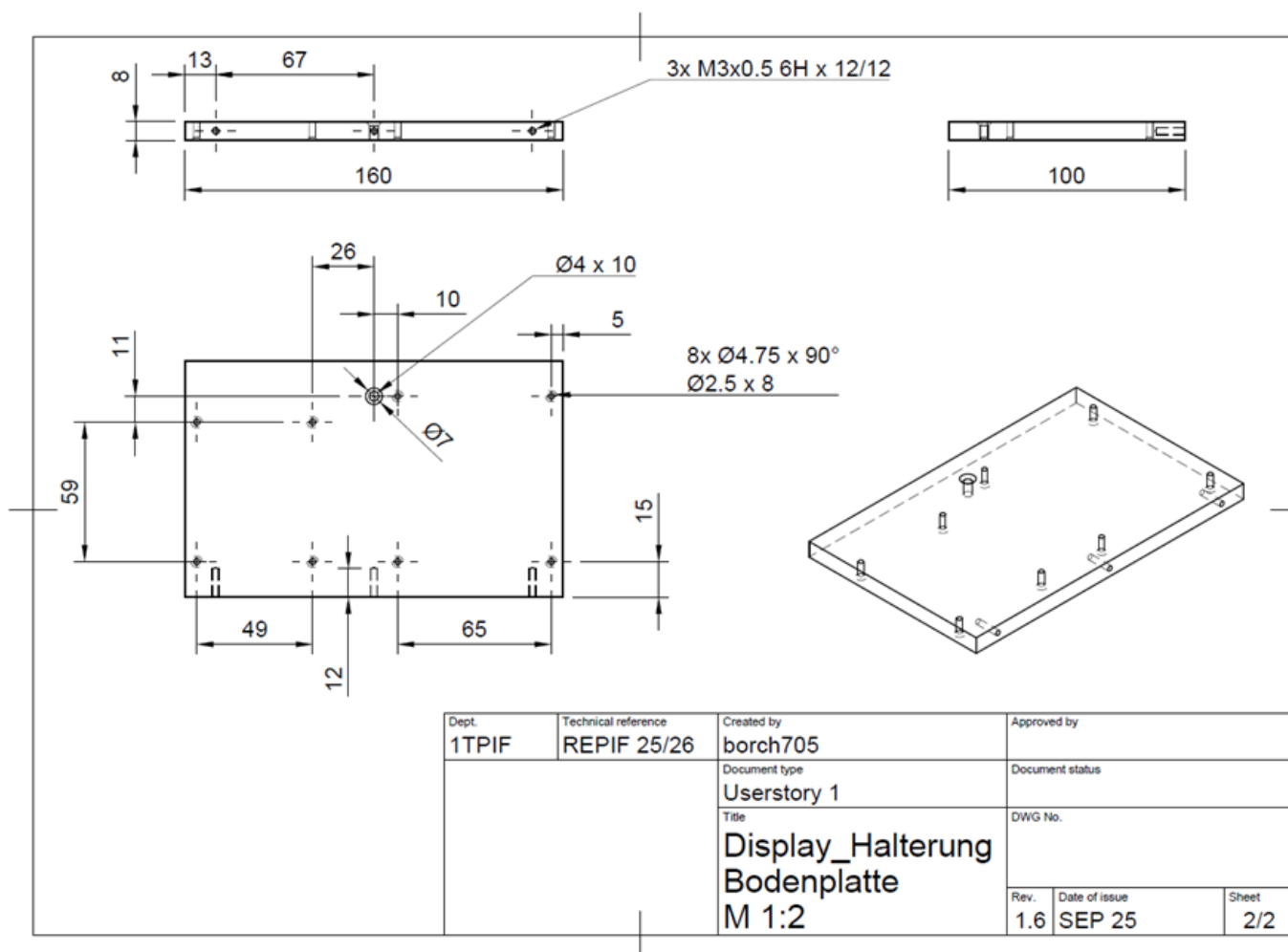
Front Plate (Display_Halterung Frontplatte)



The front plate contains:

- A centered **square cutout** for the touch display
- **Four Ø2.5 mm holes**
- **Three Ø3 mm holes**
- **Three countersunk mounting holes** with **Ø5** profile on the **Ø3** holes
- Precise spacing between the display window, mounting holes, and outer edges
- Total plate thickness of **3 mm**, as shown in the side view

Bottom Plate (Display_Halterung Bodenplatte)



The bottom plate includes:

- One **countersunk hole** for support screw (**Ø7 profile** on the **Ø4 hole**)
- **Threaded holes (M3 × 0.5)** for mounting the Raspberry Pi
- **Eight Ø2.5 mm holes** for attaching the PCB and standoffs
- Additional **Ø2.5 mm holes** on the side for attaching the other PCV plate
- A plate thickness of **8 mm** (side view)
- Alignment holes that match the front plate spacing

Work Steps

The work process of the PVC plates went according to the schematics and required a combination of precise marking, drilling, shaping, and thread cutting.

The following work steps describe the process performed during this sprint and adjustments that had to be done.

1. Marking Reference Lines

The first step involved transferring all important measurements from the technical drawings onto the PVC plates.

A standard height gauge was used to scribe straight parallel reference lines across the surface. These lines served as guides for both the square display cutout and the positions of the drilled holes. To ensure accuracy, the plates were aligned against the metal blocks and on a perfectly flat stone plate, allowing lines to be drawn consistently and in parallel with the plate edges.

2. Marking Hole Positions

After the reference lines were scribed, the individual hole positions were identified by marking cross points.

A center punch and hammer was then used to create by punching small starter points at each hole location. This prevented the drill bit from slipping and ensured that every hole would be drilled exactly at its intended position.

3. Drilling Holes Using the Drill Press

All required holes, ranging from Ø2.5 mm to Ø5 mm, were drilled using a drill press to maintain 90 degree alignment.

The drill press made it possible to achieve clean, accurate holes for:

- Display mount holes
- Alignment holes for connecting both plates
- Countersink preparation holes
- Threaded hole bases (Ø2.5 mm)

Almost each hole was drilled according to the dimensions specified in the schematics. One of the main holes for the Bottom Plate had a countersunk hole drill in too deep since its height on the press was only for the Front Plate thickness. Fortunately, the small mistake didn't cost an entire plate replacement. Instead, additional screw supports will be added in to adjust the screw.
(remark for both drilling holes and countersink steps)

4. Creating the Square Display Cutout

To form the square opening for the touch display, a starter hole in the middle was first drilled inside the cutout area.

Using this as an entry point:

- **Hand files** were used to begin removing large amount of the material
- **Needle files** were used afterward to fine-shape the edges

The filing continued until the cutout matched the exact outline marked on the plate.

5. Countersinking the Screw Holes

Several holes on both plates needed to accommodate countersunk screws.

A **countersink drill bit** was used to create a 90° conical recess at the surface of each designated hole. Then adjusted with a hand countersink tool to even the screw in.

This ensured that the screw heads would sit flat with the plate surface once assembled.

6. Cutting Threads Into the Smaller Holes

The Ø2.5 mm drilled holes on the side of the Bottom Plate required internal threading for attaching components.

Using a thread tap (M2.5) together with a T-handle tap wrench, each hole was tapped manually. Because the plates were made of PVC, a single finishing tap was sufficient to cut clean threads without pre-tapping.

The tapping process allowed:

- Screws to fit securely
- Components to be mounted without using nuts
- Proper alignment between the upper and lower plate

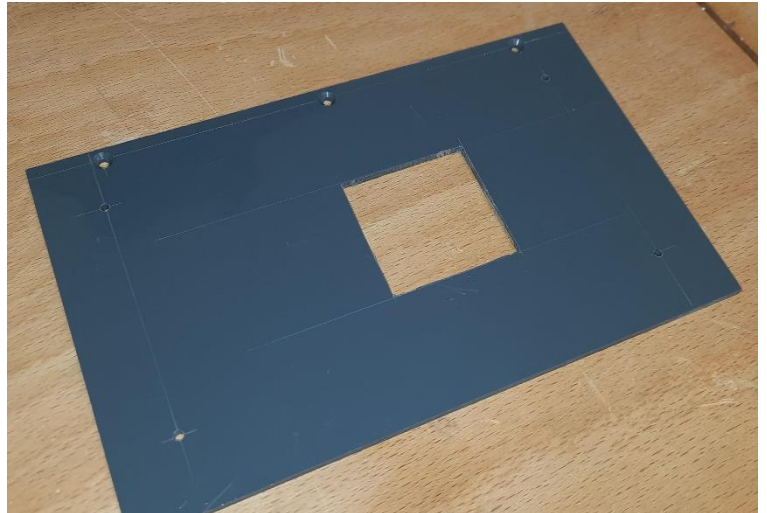


Results

The PVC plates were machined according to the schematics, with all holes, threads, countersinks, and the display cutout completed successfully. The plates fit together properly and form a stable holder for mounting the display and electronics in the next project steps.



Bottom Plate with traced lines and drilled holes

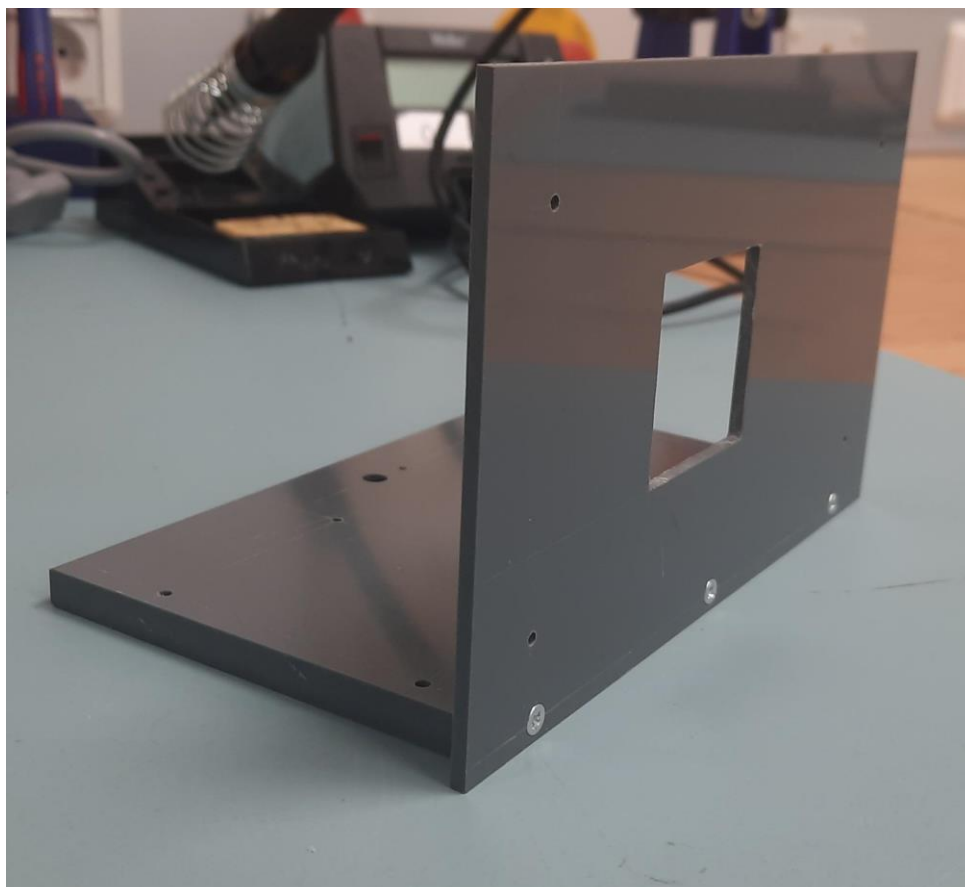
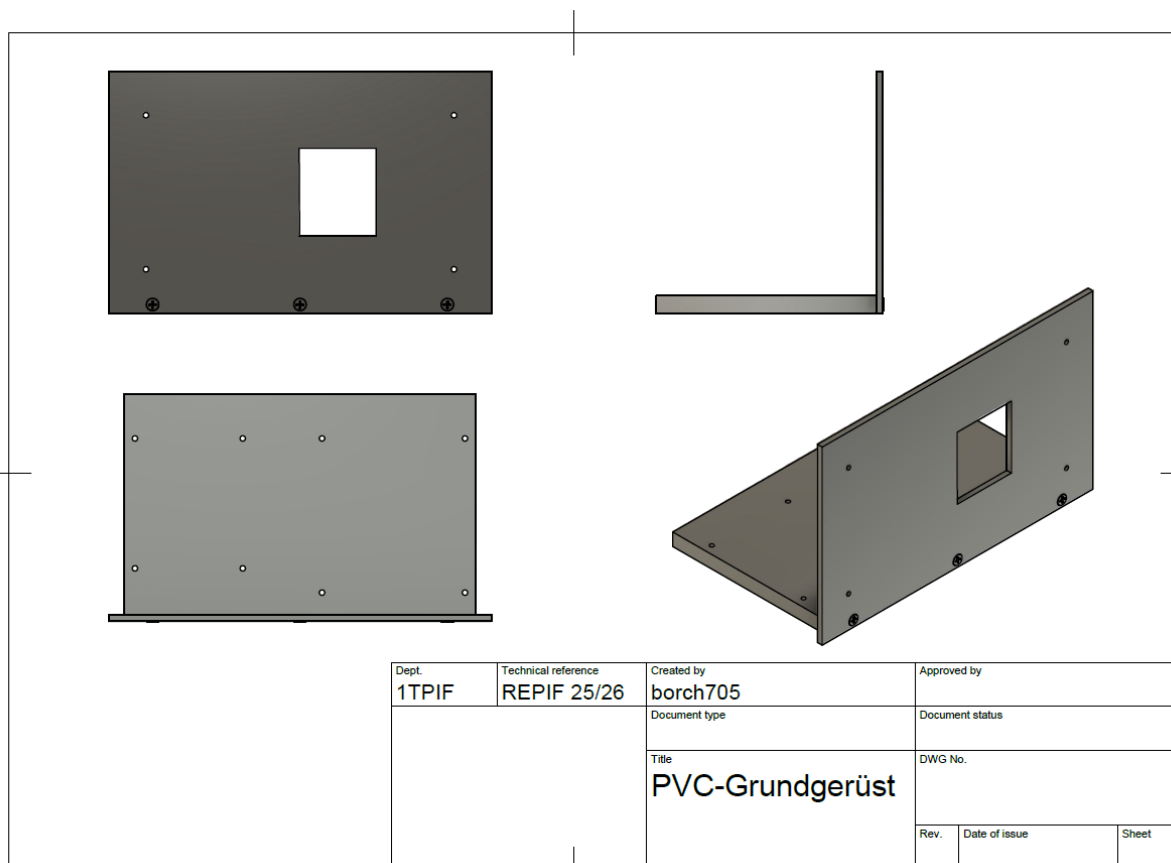


Front Plate traced lines, drilled holes and mad



Bottom Plate side drilled holes with threading

Final Result



REPIF1: SPRINT A2 Report

Printed Circuit Board

Printed Circuit Board Assembly

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 8 December – 21 December 2025

Contents

Sprint 2 of the Project (Workshop work).....	93
Goal	93
Objective	93
Components/Tools	93
<i>Tools Used</i>	93
<i>PCB Components</i>	94
Schematics and Layout.....	95
Work Steps	97
Result.....	98

Sprint 2 of the Project (Workshop work)

Goal

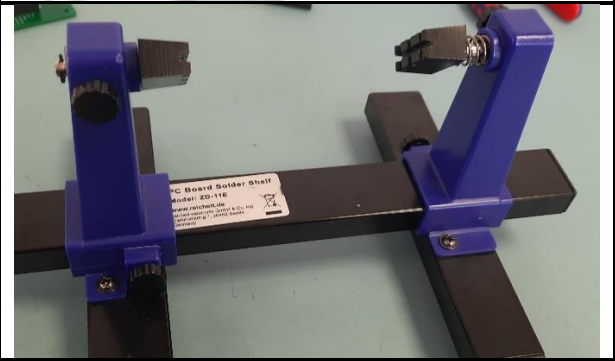
The goal of this sprint was to assemble the printed circuit board required for the project by correctly placing and soldering all electronic components according to the provided circuit design. The completed PCB serves as a hardware foundation for later system integration.

Objective

The objective was to identify the required electronic components based on the given circuit board schematic and to assemble them onto the PCB using proper soldering techniques, while ensuring functional reliability and avoiding damage to components or the board.

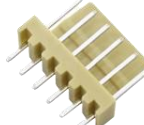
Components/Tools

Tools Used

Soldering station and Solder wire	 A digital soldering station with a black base and a digital display showing 350°C. A soldering iron with a black handle and a silver tip is plugged into the station. A spool of silver solder wire is visible in the background.
PCB solder shelf	 A blue metal PCB solder shelf with two adjustable clamps. A label on the shelf reads: "PC Board Solder Shelf Model: ZD-118".

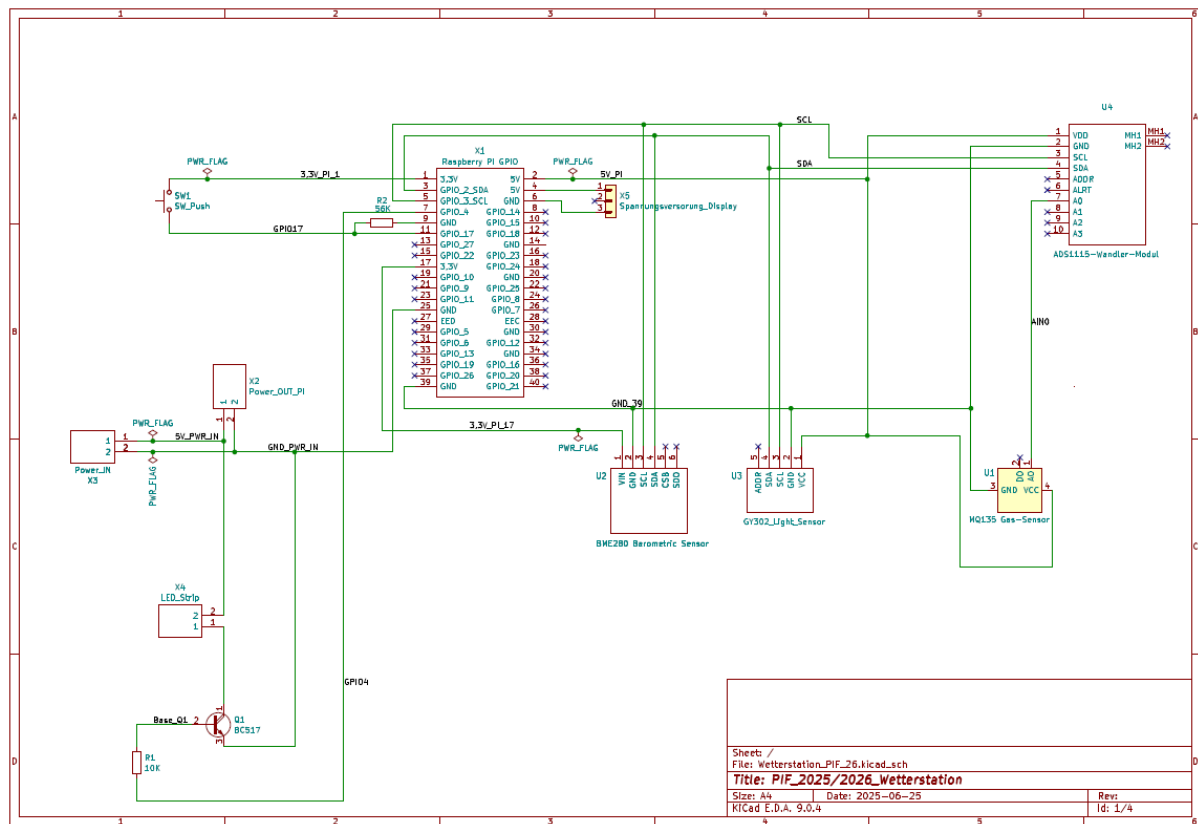
PCB Components

The following electronic components were used to populate the printed circuit board according to the circuit schematic:

Name	Qty.	Image
Printed circuit board (PCB)	1	
Resistor (10 kΩ)	1	
Resistor (56 kΩ)	1	
IDC ribbon cable box header	1	
5-pin PCB connector	1	
6-pin PCB connector	1	
3-pin straight male headers	1	
4-pin straight female header	1	
Transistor (BC517)	1	
2 pin low-power screw terminal block	2	
2 pin high-power screw terminal block	2	

Schematics and Layout

The PCB layout and circuit schematic were used as a reference to determine component placement, orientation, and correct pin connections before soldering.

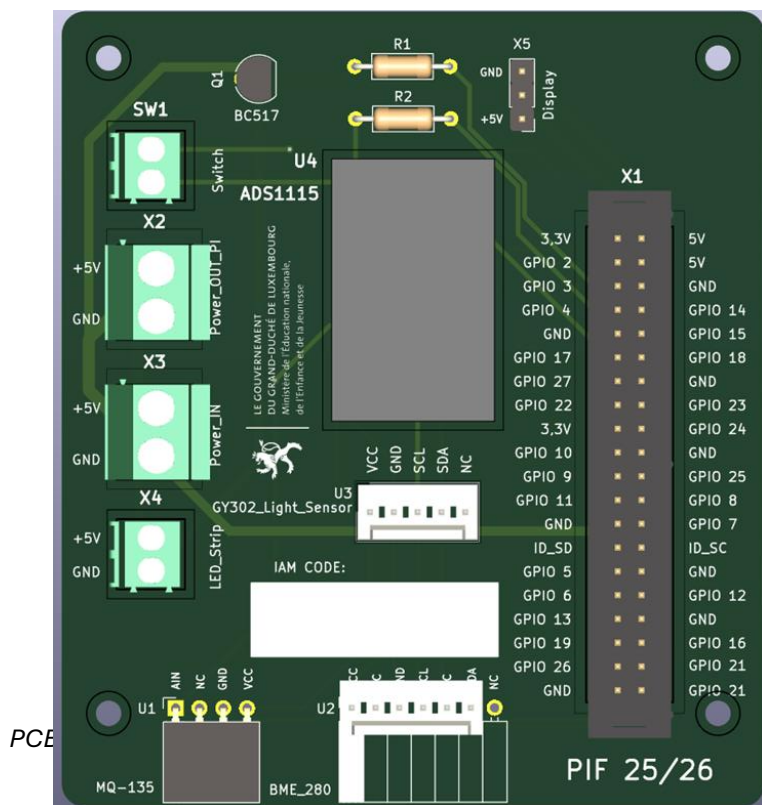


Circuit schematic

1TPIFI2-DI

REPIFI1

STAMA314



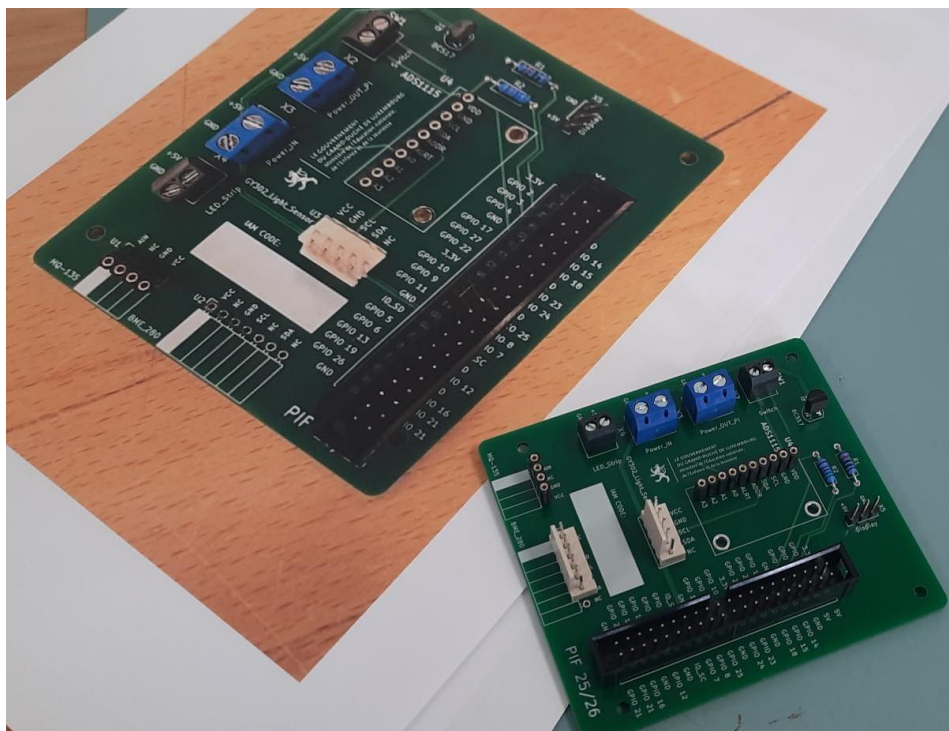
Work Steps

Based on the graphical schematics of the circuit board, the required components and the PCB were prepared. After setting up and heating the soldering station, the components were placed and soldered onto the board one by one.

During the soldering process, special attention was given to component orientation and pin placement. The **NC (Not Connected)** pin of the **U2** component was intentionally left unsoldered, as specified in the circuit design.

Safety and Quality Considerations

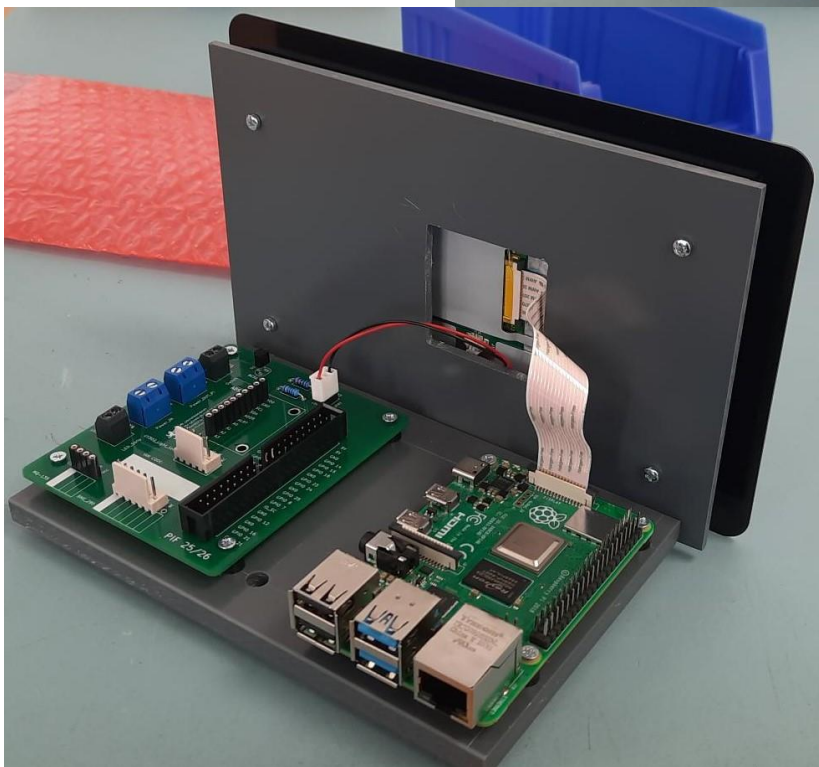
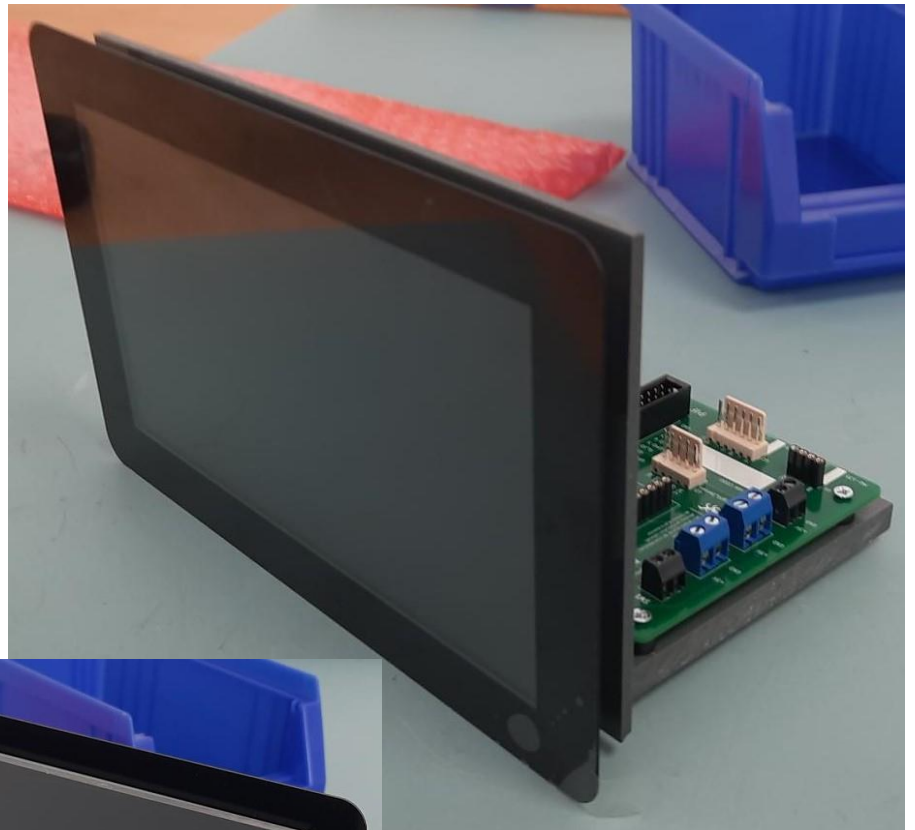
- Components were not overheated during soldering
- Excess solder was avoided to prevent short circuits
- Solder was prevented from leaking to the opposite side of the PCB
- The PCB surface was protected from prolonged heat exposure
- The transistor was not inserted too deeply into the board



Completed Printed Circuit Board

Result

After completing the soldering process, the assembled circuit board was mounted onto the finished PVC holder. The PCB is now mounted and ready for integration into the next step of the project.



REPIF1: SPRINT A3 Report

Power Supply

Power Supply (Modified)

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 5 January – 18 January 2026

Contents

Sprint 3 of the Project (Workshop work).....	101
Goal	101
Objective.....	101
Components/Tools	101
<i>Tools and Materials used</i>	101
Work Steps	102
<i>Safety and Handling Notes</i>	102
Result.....	103
Reflection.....	103

Sprint 3 of the Project (Workshop work)

Goal

The goal of this sprint was to prepare a safe and reliable power connection for the system by modifying the original Raspberry Pi power supply. Instead, we decided to keep the power supply and just demonstrate correct handling a separated cable by soldering electrical connection and using isolation techniques.

Objective

The objective was to connect two cable ends by properly stripping, soldering, and insulating all conductors using heat-shrink tubing. This ensures electrical continuity and sufficient insulation for demonstrating correct cable connection process.

Components/Tools

Tools and Materials used

Soldering station and Solder wire	
PCB solder shelf	
Multi-core power cable (4 core with Aluminium shielding net)	
Heat-shrink tubing (small diameter)	

Heat-shrink tubing (large diameter, transparent)	
Wire stripper	
Heat gun	
Side cutters	

Work Steps

The outer insulation of the cable was removed to expose the aluminium shielding mesh, which was then twisted into a single strand to ensure electrical continuity. After that, the inner insulation was stripped to expose the individual conductors.

Each conductor was stripped by approximately 5 mm. Before soldering, heat-shrink tubing was placed over the cables to allow insulation after soldering. The corresponding conductors were then soldered together carefully.

After soldering, the heat-shrink tubing was positioned over each joint and heated until fully insulated. The aluminium shielding net was also soldered together and insulated. Finally, a larger transparent heat-shrink tube was applied over the entire joint to provide protection and additional insulation.

Safety and Handling Notes

Cables were not overheated during soldering to prevent from shrinking the heat shrinking tube

Result

The power cable connection was successfully soldered and fully insulated. All conductors are securely connected and protected, making the cable suitable for later integration into the system without modifying the original Raspberry Pi power supply.



Reflection

During this sprint, soldering process was a bit troublesome and some parts of the process I had to repeat. I followed the instructions presented on the user story instead of teacher's instructions and cut the shielding mesh. At certain point when I was soldering cables, I have forgotten to place a heat-shrinking tube over the cables, so I had to re-do the process again.

In future tasks, I will pay closer attention to instructions and work steps, especially when handling electrical components and insulation materials.

REPIF1: SPRINT A4 Report

Sensors 1

Gas Sensor and Analog-to-Digital Conversion Integration

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 5 January – 1 February 2026

Contents

Sprint 4 of the Project (Workshop work).....	106
Goal	106
Objective.....	106
Components.....	106
<i>Modules</i>	106
<i>Pin Description (Overview)</i>	107
Work Steps	107

Sprint 4 of the Project (Workshop work)

Goal

The goal of this sprint was to integrate the required sensor and analog-to-digital converter modules into the system. These components are responsible for capturing environmental data and converting analog sensor signals into digital values that can be processed by the Raspberry Pi.

Objective

The objective was to prepare and install the MQ135 gas sensor module and the ADS1115 16-bit analog-to-digital converter module according to the provided circuit documentation. This included verifying the components, understanding their functionality, and correctly placing them on the printed circuit board.

Components

Modules

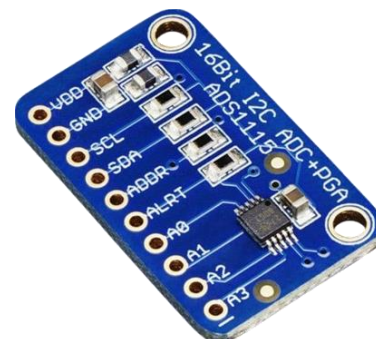
MQ135 Gas Sensor Module

The MQ135 is an air quality sensor used to detect gases such as ammonia, alcohol, benzene, smoke, and CO₂. It provides an analog output signal that varies depending on gas concentration.



ADS1115 – 16-bit Analog-to-Digital Converter (ADC)

The ADS1115 converts analog signals from sensors into high-resolution digital values. It communicates with the Raspberry Pi via the I²C interface and allows accurate measurement of low-voltage sensor outputs.



Pin Description (Overview)

MQ135 Sensor Pins

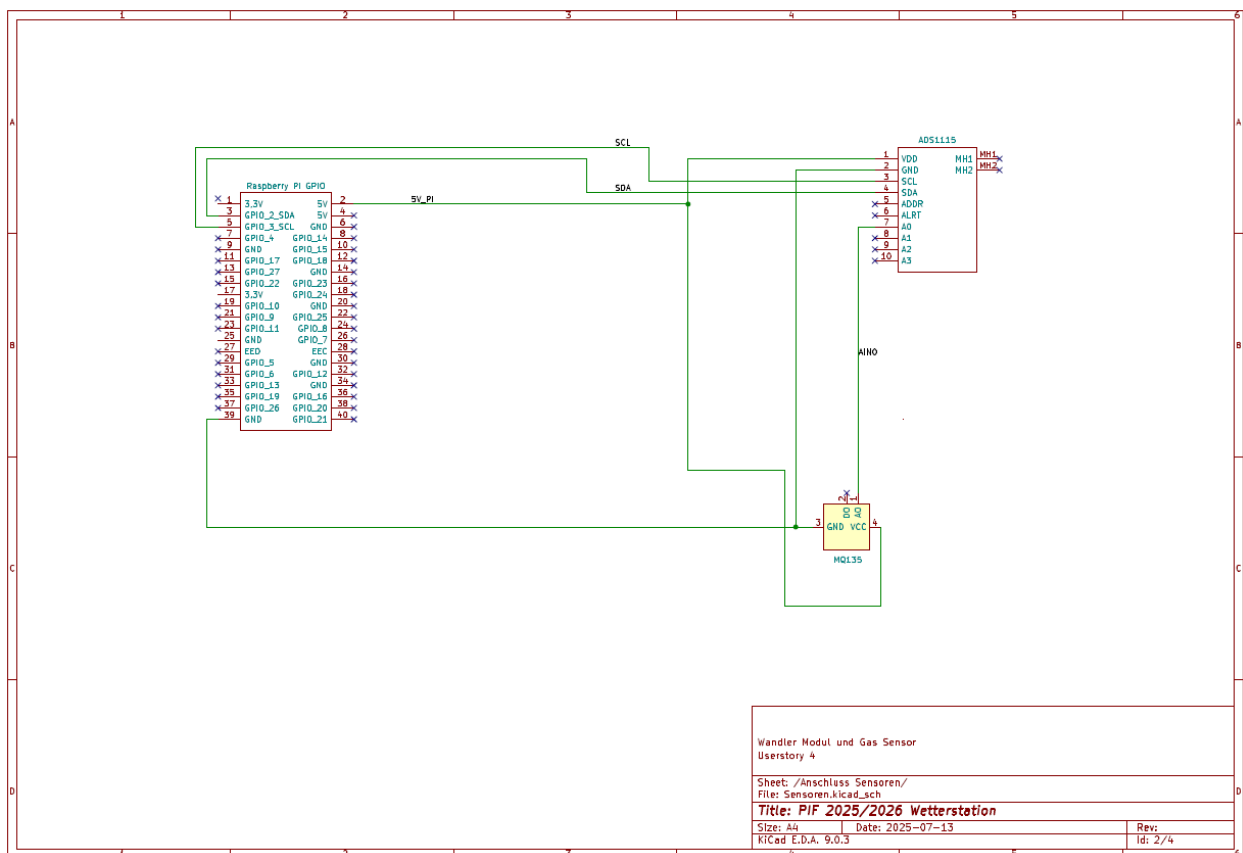
- **VCC** – Power supply
- **GND** – Ground
- **AO** – Analog output signal
- **DO** – Digital output (not used in this setup)

ADS1115 Converter Pins

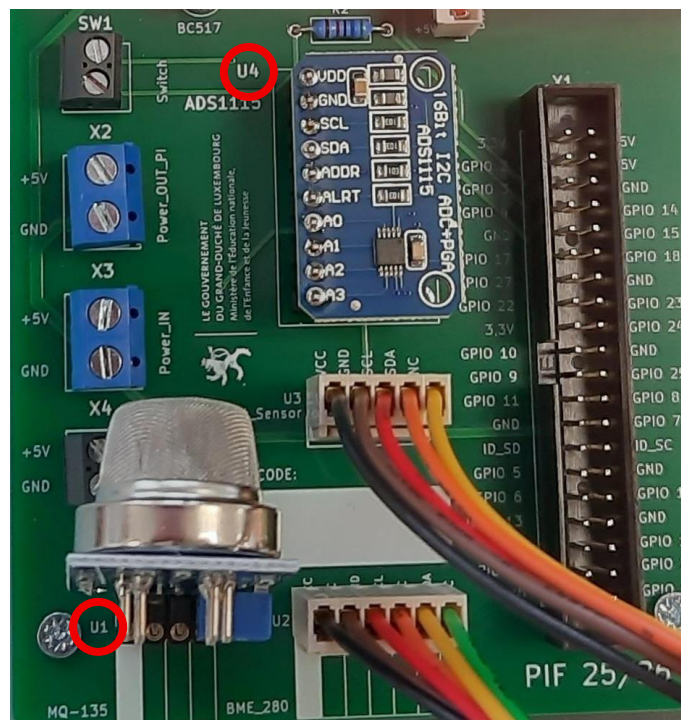
- **VDD** – Power supply
- **GND** – Ground
- **SCL** – I²C clock line
- **SDA** – I²C data line
- **A0–A3** – Analog input channels
- **ADDR** – I²C address selection pin (used to change the device address on the I²C bus)
- **ALRT** – Alert/Ready output pin (can be used as a comparator or conversion-ready signal)

Work Steps

Before assembly, the required components were checked with the work order and circuit documentation. Although soldering pin headers was part of the original task description, the provided MQ135 sensor and ADS1115 converter already had pin headers soldered in place.



Following the circuit datasheet, the **MQ135 sensor module** was placed into the designated **U1** position on the printed circuit board and aligned with the correct pins. The **ADS1115 converter module** was then inserted into position **U4**, also according to the PCB markings and schematic.



After placement, the modules were visually inspected to ensure correct orientation and proper seating in their respective slots.

REPIF1: SPRINT A5 Report

Sensors 2

Digital Environmental and Light Sensor Integration with I²C Configuration

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 2 February – 1 March 2026

Contents

Sprint 5 of the Project (Workshop work).....	111
Goal	111
Objective.....	111
Components	111
<i>Modules</i>	111
<i>Pin Description (Overview)</i>	112
Work Steps	112
I ² C Address Determination and Configuration	114
Functional Test (Prepared Procedure)	115
Reflection.....	115

Sprint 5 of the Project (Workshop work)

Goal

The goal of this sprint was to integrate the environmental sensor and light sensor into the system, establish proper I²C communication, and prepare the modules for functional testing using dedicated Python test programs.

Objective

This sprint focused on the electrical preparation and connection of the BME280 environmental sensor and the BH1750 (GY-302) light sensor. The sensors were connected to the printed circuit board using soldered cable connections and mounted into their designated PCB slots.

In addition, the I²C addresses of all connected modules (BME280, BH1750, and MQ135, ADS1115 from previous Sprint) were to be determined and configured to ensure correct communication with the Raspberry Pi. The functionality was intended to be verified using provided test scripts.

Components

Modules

BH1750 (GY-302) – Light Sensor

The BH1750 (GY-302) is a digital light intensity sensor used to measure ambient illumination in lux. It is commonly applied in environmental monitoring systems where light conditions need to be recorded accurately.

The sensor communicates via the I²C interface and provides direct digital measurement values. Because the output is already digital, no additional analog-to-digital conversion is required.



BME280 – Environmental Sensor

The BME280 is a combined environmental sensor capable of measuring:

- Temperature (°C)
- Humidity (%)
- Air pressure (hPa)

The module includes internal calibration and signal processing, providing stable and precise digital output values. Communication with the Raspberry Pi is performed via the I²C interface. The BME280 is widely used in weather monitoring systems due to its compact size and multi-parameter capability.



Pin Description (Overview)

BH1750 Pin Configuration

- **VCC** – Power supply
- **GND** – Ground
- **SCL** – I²C clock line
- **SDA** – I²C data line
- **ADDR** – I²C address selection

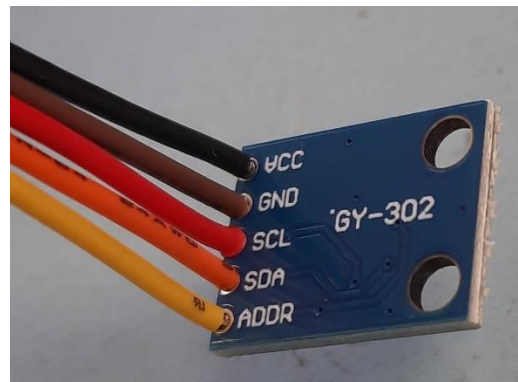
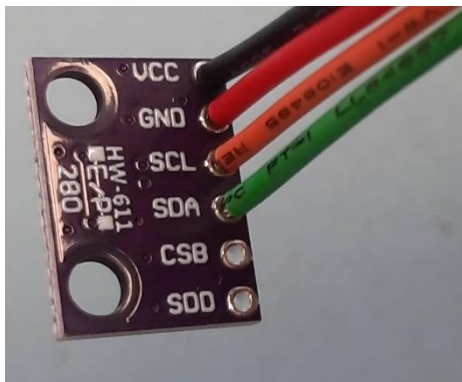
BME280 Pin Configuration

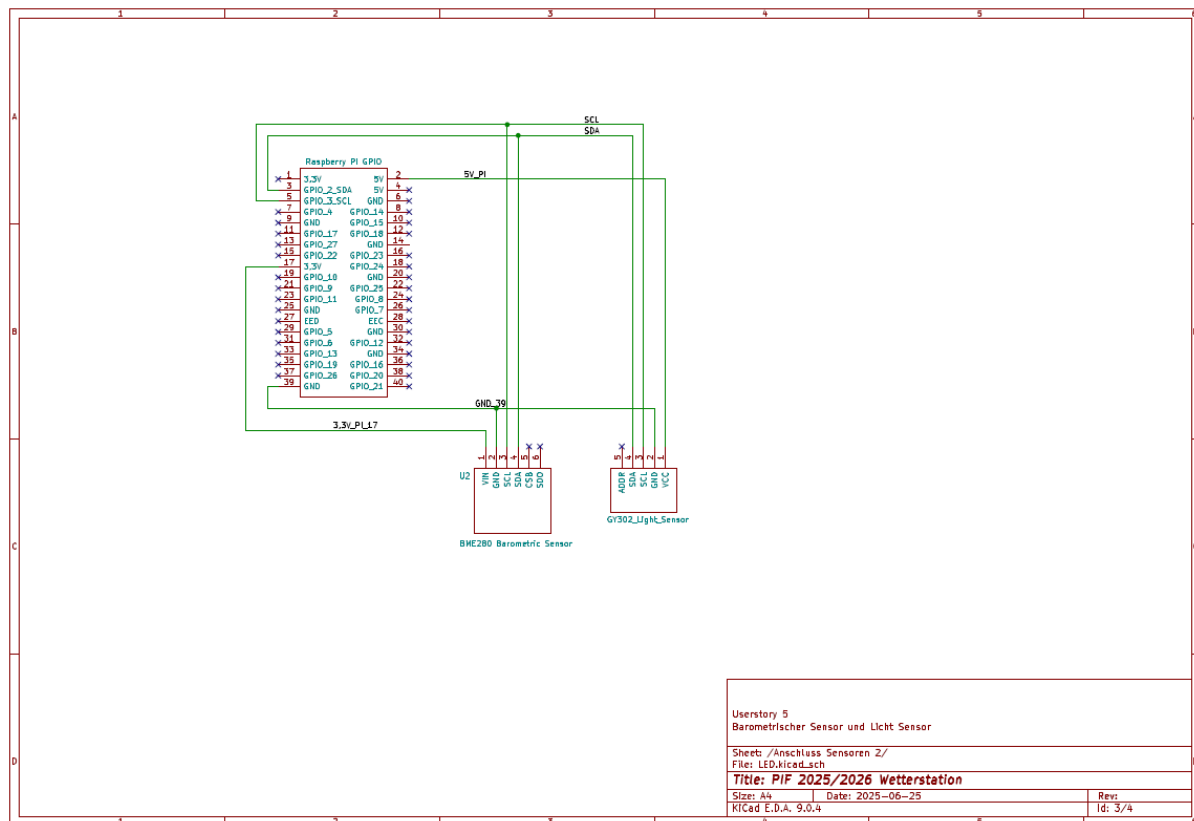
- **VCC** – Power supply
- **GND** – Ground
- **SCL** – I²C clock line
- **SDA** – I²C data line
- **CSB** – Interface selection (SPI/I²C)
- **SDO** – I²C address selection

Work Steps

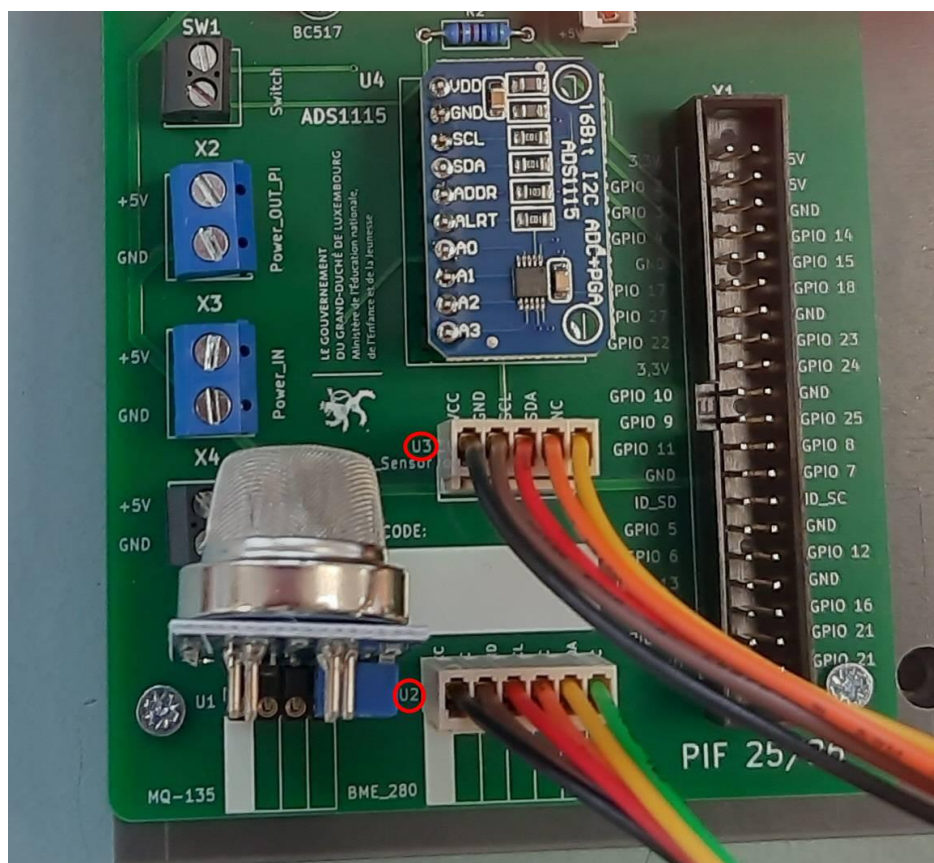
First, the required sensor modules and corresponding PCB connectors were checked to ensure compatibility and correct pin configuration. The BH1750 light sensor was prepared by soldering five cables to its pins (VCC, GND, SCL, SDA, and ADDR). Care was taken to produce clean solder joints and to avoid overheating the module.

For the BME280 environmental sensor, four cables were soldered to the pins VCC, GND, SCL, and SDA. The pins CSB and SDO were not connected, as the module was operated in standard I²C mode according to the project user story description.





After soldering, all pins were visually inspected to ensure proper connection and insulation. The **BH1750** module was then connected to the 6-pin PCB connector at position **U3**, while the **BME280** module was inserted into the 5-pin PCB connector at position **U2**.



I²C Address Determination and Configuration

In this section, I decided to take initiative to look up on the configuration process and how to make it work in theory since I didn't had access to the physical project and the sensors.

The following I²C-based components were present in the system:

- MQ135 (via ADS1115) (Sprint 4)
- ADS1115 16-bit converter (Sprint 4)
- BME280
- BH1750

Each I²C device requires a unique address to communicate on the shared bus.

Since all devices share the same I²C bus, each module must use a unique hexadecimal address. If two devices use the same address, communication conflicts occur and data transmission fails.

The address of connected devices can typically be determined using an I²C scanning command such as:

```
i2cdetect -y 1
```

The provided test files **test-bh1750.py**, **test-bme280.py**, and **test-mq135.py** contain a placeholder value for the I²C address:

```
ADDRESS = 0x00 # TODO: Change this
```

The value 0x00 does not represent the actual device address. It is only a temporary value that must be replaced before running the program.

After determining the correct I²C address using a scanning tool command(e.g., i2cdetect -y 1), the detected hexadecimal address must be inserted into the script. For example, if the scan shows the address 0x23, the line in the program must be changed to:

```
ADDRESS = 0x23
```

This step is necessary because each I²C device communicates through its unique hexadecimal address. If the correct address is not configured, the program will not be able to establish communication with the sensor.

Functional Test (Prepared Procedure)

The intended testing process was:

1. Determine the correct I²C address using an I²C scanning tool.
2. Insert the detected address into the respective test script.
3. Execute the script, for example:

```
python3 test-bh1750.py
```

4. The program should output measurement values such as:
 1. Light intensity (lux)
 2. Temperature (°C)
 3. Pressure (hPa)
 4. Humidity (%)
 5. CO₂ concentration (ppm)

The provided test scripts include basic error handling to detect communication failures or incorrect address configuration.

Reflection

During this sprint, the soldering process required careful attention, particularly when attaching the cables to the sensor pins. Precision was important to avoid cold solder joints or incorrect wiring. Since multiple sensors share the same I²C bus, correct pin alignment and address configuration were especially important.

One challenge was understanding why only specific pins of the BME280 sensor (VCC, GND, SCL, SDA) were connected, while CSB and SDO remained unused. While also trying to understand how the modules work on the circuit board while connected to the RaspberryPi. Reviewing the documentation helped clarify that these pins are not required in standard I²C mode.

Due to time constraints, the functional testing with live measurement values could not be fully completed. However, the configuration process and test scripts were prepared according to the project requirements.

In future tasks, I will allocate more time for final verification and testing to ensure that all components are fully validated before completing the sprint.

REPIF1: SPRINT A6 Report Assembly

Wooden Housing Assembly and Component Integration

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 2 March– 15 March 2026

Contents

Sprint 6 of the Project (Workshop work).....	118
Goal	118
Objective.....	118
Components	118
Component Placement.....	119
Work Steps	119
Preparation of the Wooden Housing	119
Drilling Holes for Button and LED Cables	120
Mounting the Sensors on the Rear Panel.....	121
Final Assembly of the Housing.....	123
Reflection.....	124

Sprint 6 of the Project (Workshop work)

Goal

The goal of this sprint was to assemble the wooden housing of the Portable Indoor Feedback station and integrate the main hardware components inside it. The housing serves as the structural enclosure that protects the electronics and provides mounting points for the sensors, push button, and LED cables.

Objective

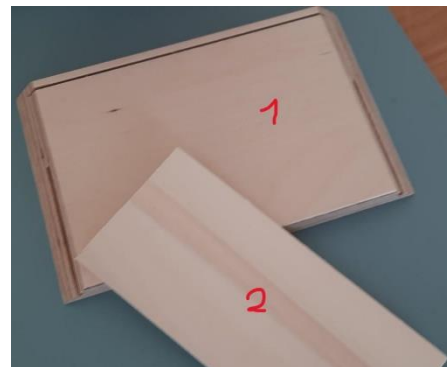
The objective was to prepare the wooden housing parts, drill the required openings for external components, mount the sensors on the rear panel, and assemble the housing together with the internal electronics. Special attention had to be given to component placement and cable routing so that the display unit, Raspberry Pi, and printed circuit board could be inserted and removed without obstruction.

Components

The following components and materials were used during this sprint:

Wooden housing parts

- Base plate (trapezoid-shaped) (1)
- Rear panel (2)
- Two side walls



Fastening materials

- M2.5 screws
- Nuts
- Metal washers
- Wooden locking pins

Tools

- Drill press
- Conical drill bit (approx. 30 mm)
- Wood drill bit with center spike
- 3 mm drill bit
- Measuring tool / ruler
- Sandpaper or file for smoothing edges

Component Placement

The housing was designed with guide slots in the base plate and side walls. These slots allow the display module with the Raspberry Pi and printed circuit board to slide into the box from the top.

The rear panel also slides into the housing through wider slots on the side walls. A small gap remains at the bottom of the rear panel to allow cables from the sensors to pass through.

The side corners of the housing components are angled ramp joints with small holes. These joints allow the parts to be locked together while using small pins, ensuring that it has stable structure.

Work Steps

Preparation of the Wooden Housing

For this sprint, four wooden parts were provided to build the housing of the Portable Indoor Feedback station. The housing consisted of:

- a trapezoid-shaped **base plate**
- **two side walls**
- **one rear panel**

The wooden components were designed with **precise cutouts** that allow the parts to slide into each other during assembly. The base plate and the side walls contain thin guide slots that allow the **display and the PVC holder with the electronics** to slide into the housing from the top.

Additionally, the side edges of the base plate and side walls contain **angled ramp cut sides with small cutout**. These cutouts are used for inserting small locking pins when two corner pieces are placed together. Once inserted, these pins secure the parts together and keep the housing firmly assembled without the need for glue.

Before the housing could be assembled, two holes needed to be drilled to allow the installation of the **push button and LED cables**.



Drilling Holes for Button and LED Cables

The holes for the push button and LED cables were drilled into one of the side walls. The location was chosen carefully to ensure that the cables could easily reach the printed circuit board inside the housing while also not obstructing any other cables inside.

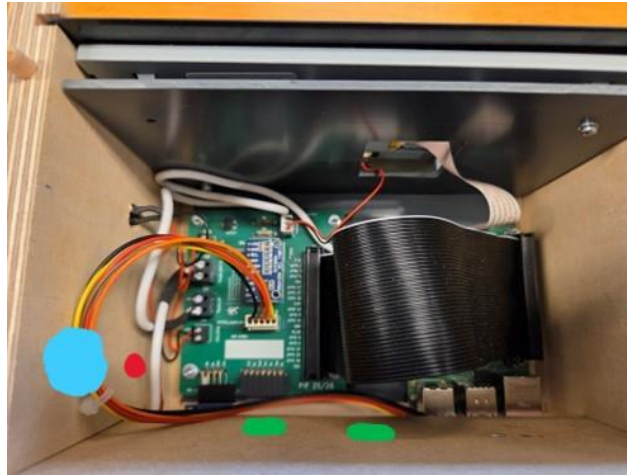
The selected side was closer to the **printed circuit board**, while the opposite side was partially blocked by the **Raspberry Pi and the GPIO extension cable**.

Before drilling, the hole positions were measured and marked approximately **3–5 cm from the edges** of the wooden plate. The holes were placed slightly higher on the wall in order to leave enough space for the components located near the bottom of the housing.

Two different drill bits were used to drill the holes:

- A **conical drill bit (approx. 3 cm diameter)** for the push button (marked in blue)
- A **wood drill bit with center spike** for the LED cable opening (marked in red)

Special attention was required when placing the wooden plate on the **drill press**, because the lower side of the wall also contains a ramp-shaped joint used for assembly. To prevent the wood from splitting, the plate was positioned so that the ramp side rested securely on the drill press table without putting too much pressure on it. The holes were then drilled carefully and slowly.



Drilling and Mounting the Sensors on the Rear Panel

The rear panel of the housing was used to mount the **light sensor** and the **environmental sensor**.

Before installation, small mounting holes were drilled using a **3 mm drill bit**. After drilling, the edges of the holes were smoothed to remove wooden splinters.

The sensors were already connected to the printed circuit board through soldered cables. These cables were guided through the small opening at the bottom of the rear panel.

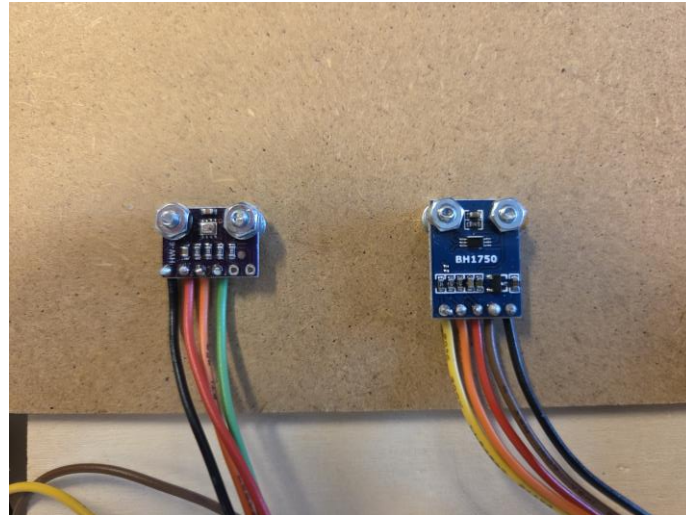
Due to space limitations inside the housing, the sensors had to be positioned carefully (marked in green):

- One sensor was placed **centrally behind the printed circuit board**
- The second sensor was mounted **between the PCB and the Raspberry Pi**



The sensors were fixed to the panel using:

- **M2.5 screws**
- **nuts**
- **metal washers**



Final Assembly of the Housing

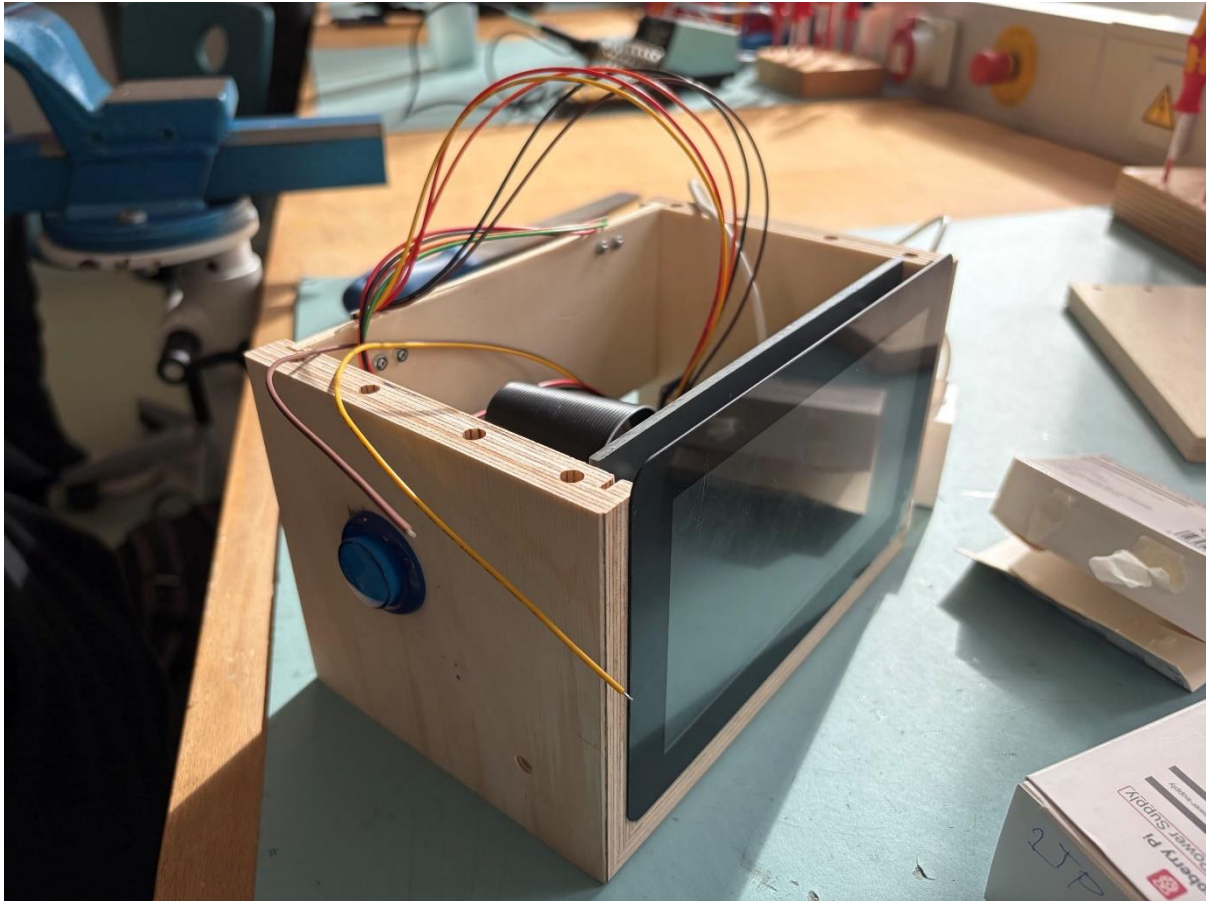
After all necessary holes were drilled and the sensors were mounted, the housing could be assembled.

First, the **base plate** was placed with the guide slot facing upward. The two **side walls** were then attached by sliding their ramp joints into the base plate. Small wooden locking pins were inserted into the corner holes to secure the structure.

Next, the **main component assembly** consisting of the display, Raspberry Pi, and printed circuit board was carefully slid into the guide slots of the housing.

The **push button** was then installed through the previously drilled hole and secured using the mounting ring on the inside of the wall.

Finally, the **rear panel** with the mounted sensors was inserted from the top into the side wall guide slots, closing the back side of the housing.



Reflection

The wooden housing was successfully assembled and the main electronic components were securely installed inside the enclosure. The sensors were mounted on the rear panel, and the push button and LED cables were properly routed through the drilled openings.

During this sprint, careful planning was required when deciding where to drill the holes for the push button and LED cables. The internal components of the device limited the available space, which required selecting the side of the housing closest to the printed circuit board.

Additionally, attention had to be given to the orientation of the wooden parts during drilling to avoid damaging the ramp joints used for assembly.

REPIF1: SPRINT A7 Report

Lighting

LED Lighting Installation and Button Integration

Indoor Weather Station Project

Student: Maciej Stankowski

Class: 1TPIFI2-DI

Date: 16 March– 29 March 2026

Contents

Sprint 7 of the Project (Workshop work).....	127
Goal	127
Objective.....	127
Components and Materials	128
Tools Used:.....	128
Schematics and Layout.....	129
Functionality of the LED System	130
Work Steps	131
Result.....	132
Reflection.....	132

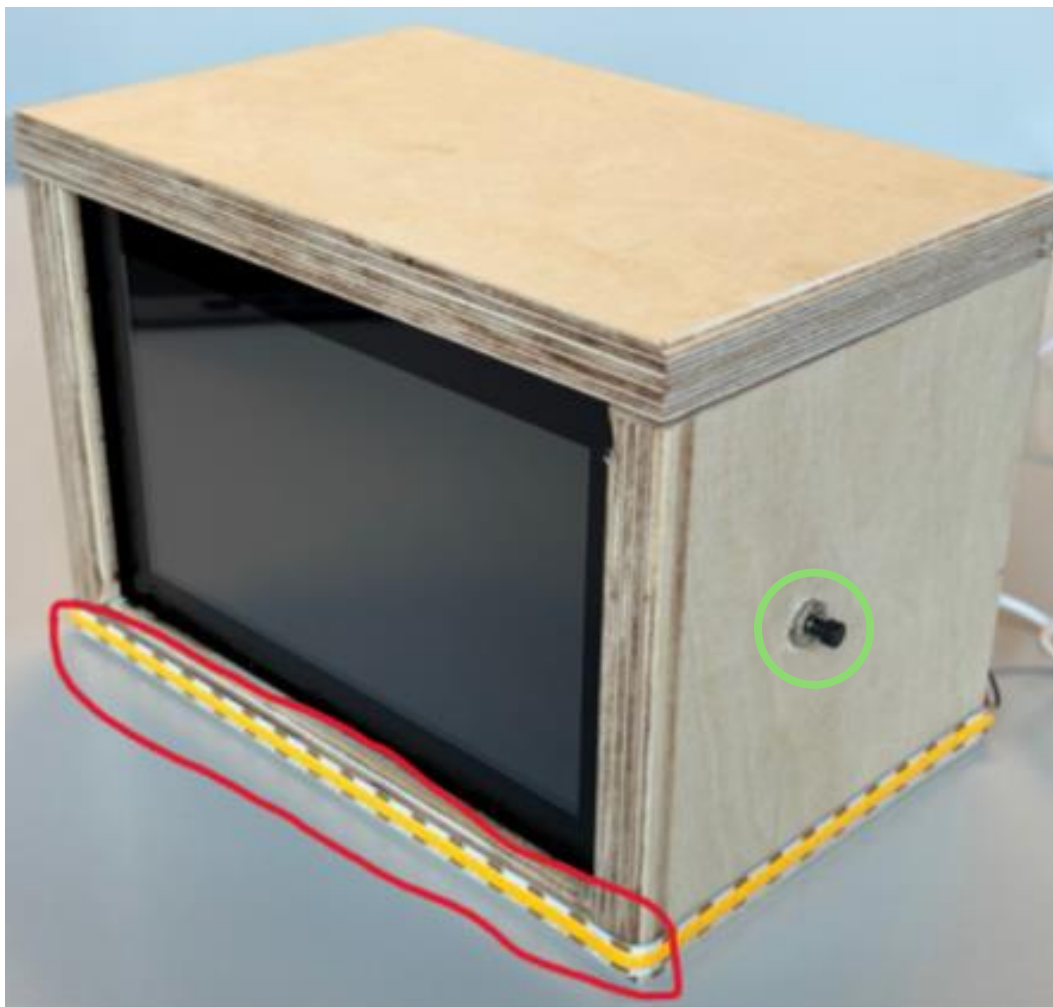
Sprint 7 of the Project (Workshop work)

Goal

The goal of this sprint was to install LED lighting into the project housing and integrate a control button. This would assure the usability and functionality of the station by providing visual illumination and manual control over it.

Objective

The objective was to mount an LED strip outside of the housing, connect the cables to the printed circuit board, and ensure proper cable routing through the drilled openings. Additionally, a button was installed and wired to allow manual interaction with the system. Furthermore, the system was designed so that the LED lighting can be controlled via the Raspberry Pi using GPIO pins. This includes processing the button input and switching the LED strip on or off through the circuit, enabling both hardware integration and software-based control.



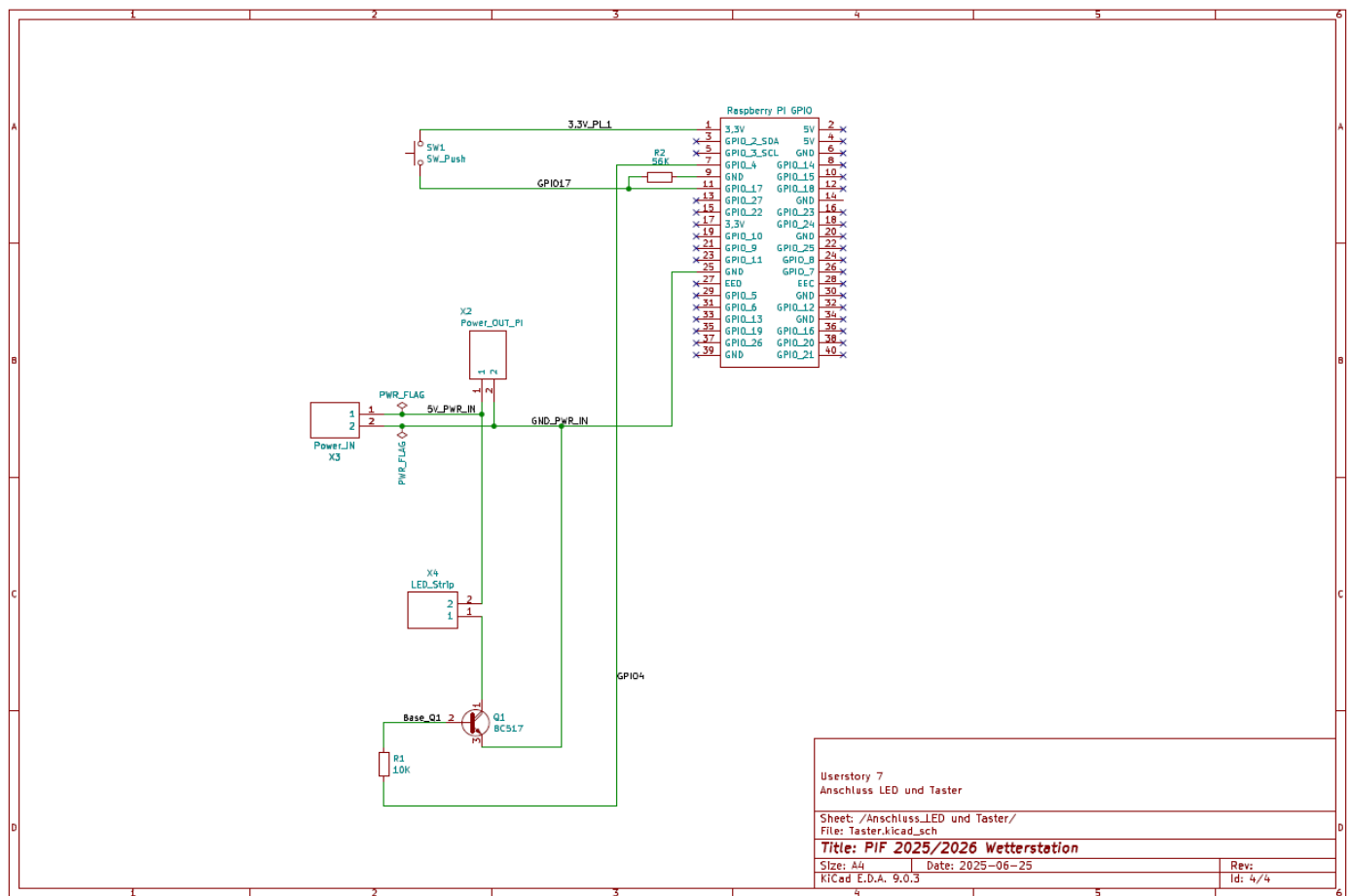
Components and Materials

Name	Qty.	Image
LED light strip	1	
Push button	1	
Electrical cables (power and signal wires)		
Heat-shrink tubing		
Double-sided adhesive tape		

Tools Used:

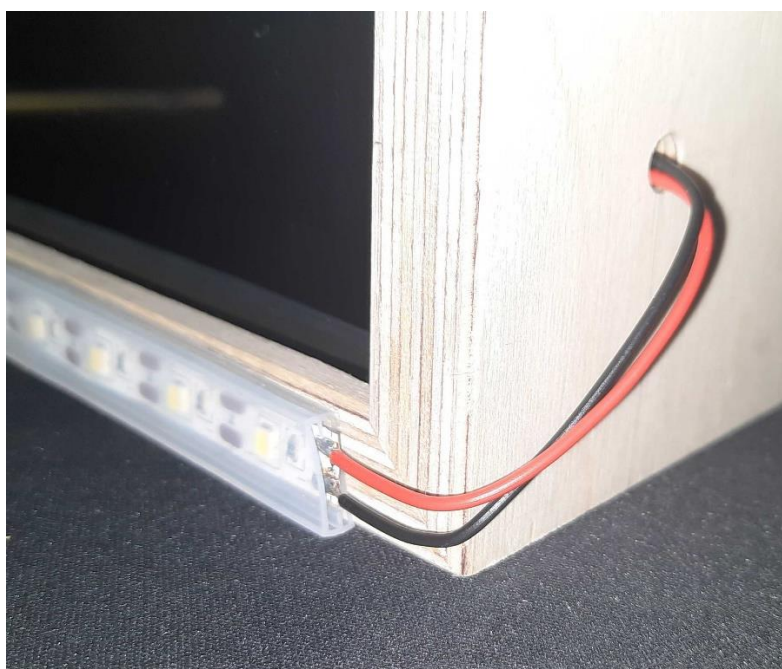
Name	Image
Soldering station (soldering iron + solder)	
Wire stripper (cable insulation removal tool)	
Multimeter (for testing button contacts)	

Schematics and Layout



Circuit diagram of LED strip and button connection

The LED strip was mounted at the **front bottom side of the housing**, below the screen. This position was chosen to avoid interference when removing the top part of the housing and to prevent stress on the cables.



The button was mounted on the **side wall**, using the pre-drilled hole from the previous sprint. Both components were connected to the PCB using appropriately routed cables.



Functionality of the LED System

The LED lighting system is controlled through a combination of hardware and software using the Raspberry Pi GPIO pins. According to the schematic, the LED strip is connected via connector **X4** and powered through the circuit's power input. The switching is performed by a transistor (**Q1 – BC517**), which acts as an electronic switch.

The transistor is controlled by **GPIO4** of the Raspberry Pi. When the Raspberry Pi sends a signal to this pin, the transistor allows current to flow and the LED strip turns on. When no signal is present, the transistor blocks the current and the LEDs remain off. This protects the Raspberry Pi, since its GPIO pins cannot directly provide enough current to power the LED strip.

The automatic switching of the LEDs is implemented through the **BH1750 light sensor** and the Python firmware script `measure.py`. The script continuously reads the ambient light level and changes the state of **GPIO4** between HIGH and LOW depending on the measured brightness. This allows the LEDs to switch on automatically when the surrounding environment becomes too dark.

The push button (**SW1**) is connected to **GPIO17** and is intended to provide manual control of the LED system. A resistor (**R2 – 56kΩ**) is used to ensure a stable input signal when the button is not pressed. Although the hardware connection of the button has been completed, the software functionality for manual button control has not yet been fully implemented. Therefore, at the current stage of the project, only the automatic light sensor control is functional.

The original Raspberry Pi power adapter was not modified in this project to avoid material waste. Instead, a bridge connection was added on the PCB to distribute the power supply to the LED circuit. Although the LED strip is rated for **5V** operation, testing showed that **3.3V** provided sufficient brightness for this prototype, making it suitable for the current implementation.

This would result in a system where:

- The **button provides input** to the Raspberry Pi
- The **Raspberry Pi processes the input**
- The **transistor switches the LED strip on or off**

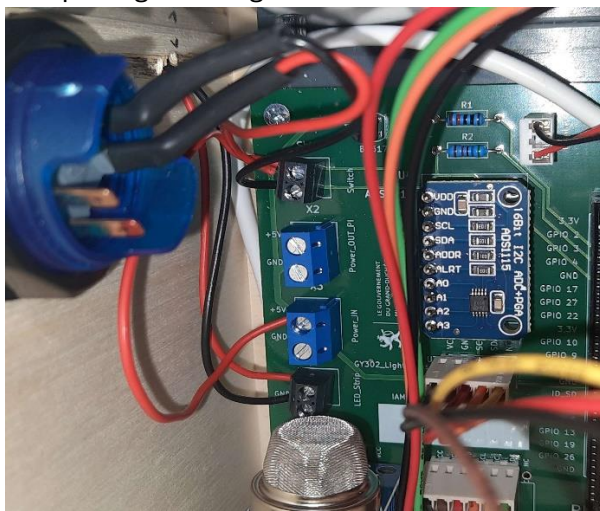
Work Steps

First, the required cables were prepared by removing insulation using a wire stripper. Two cables were connected to the X4 connector on the printed circuit board, which provides the power supply for the LED strip.

The cables were then guided through the previously drilled hole in the housing to the outside. At the end of the cables, the LED strip was prepared and soldered carefully, ensuring correct polarity (positive and negative connections). After soldering, the LED strip was attached to the bottom front edge of the housing using double-sided adhesive tape. This placement ensured that the strip remains fixed while also allowing easy removal of the upper housing components without pulling on the cables.

For the button installation, the component was first inserted into the larger drilled hole on the side wall. Before soldering, a multimeter was used to determine which contacts of the button close the circuit when pressed. The cables were then soldered to the correct button terminals. Heat-shrink tubing was placed over the soldered joints and heated to provide insulation and mechanical stability.

Finally, the other ends of the button cables were connected to the printed circuit board, completing the integration.

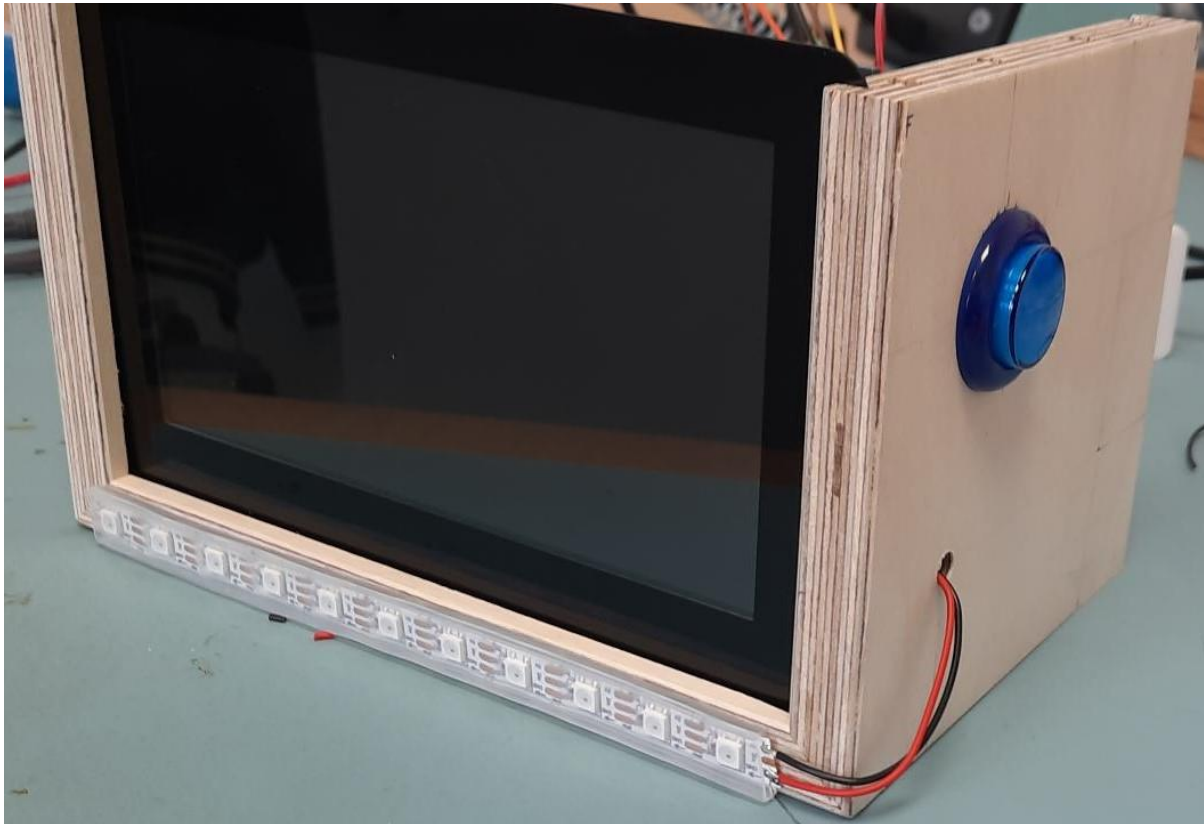


At the end we would be able to close up the housing with the top plate. We had to make sure that the plate's thin cutout is aligned with the front side that has the main screen before we could place the wooden plate. Additionally, on the side of the plate and the side walls, there were holes designated for the small wooden dowels to punch in. They were to ensure that the top stays tightly closed once the final work has been done.



Result

The LED strip and button were successfully installed and connected. The lighting system is securely mounted and properly wired, allowing stable operation without interfering with other components.



Reflection

During this sprint, careful planning of cable routing and component placement was important to avoid mechanical stress and interference with other parts of the system.

One challenge was cable management and proper connection. Since the X4 and SW1 on the circuit board were facing the inner walls, I was trying not to bend them and ensure that they are connected to the entries to avoid any disconnection of the cables to have proper transfer of power to the LEDs and through the button switch.

1TPIFI2-DI

REPIFI1

STAMA314